Kompanion

Walter Dal'Maz Silva

2026-03-23

Table of contents

Kompanion **7**
Foreword . . . 7
Installation . . . 8
Before you start . . . 8
Automatic sourcing . . . 9
Known limitations . . . 10
Building the notes . . . 10

I Computing **12**

1 Git version control **14**
1.1 Common activities . . . 14
1.2 Adding submodules . . . 14
1.3 Other tips . . . 15

2 Using SSH **16**
2.1 Port forwarding . . . 16
2.2 Key pair authentication . . . 16

3 Scientific programming **19**

4 Visual Studio Code **20**
4.1 VS Code extensions . . . 20

5 Working on Windows **22**
5.1 Command line basics . . . 22
5.2 Mount a network drive in WSL . . . 23
5.2.1 Finding a process handle . . . 23
5.3 Identifying a proxy PEM . . . 23

6 Using containers **24**
6.1 Containerfile creation . . . 24
6.2 Using Podman . . . 24
6.3 Podman with GPU . . . 25
6.4 Sidenote on Docker . . . 26
6.5 Using Apptainer . . . 27
6.6 Build workflow . . . 29

7 General Tips **30**
7.1 Running Jupyterlab from a server . . . 30
7.2 Downloading from YouTube . . . 30
7.3 Installing Python packages behind proxy . . . 31
7.4 Regular expressions . . . 31
7.5 Using nteract with a virtual environment . . . 31

8 Selection of tools **33**
8.1 Scientific publishing . . . 33
8.2 Scientific web apps . . . 33
8.3 API and databases . . . 33
8.4 Graphics . . . 34
8.5 Geometry . . . 34
8.6 Meshing . . . 34
8.7 Rendering . . . 34
8.8 Simulation . . . 34
8.9 Productivity . . . 35
8.10 Other . . . 36

9 Portable applications **37**
9.1 How does it work? . . . 37
9.2 Other possibly supported applications . . . 37
9.3 Software specific instructions . . . 38
9.3.1 DualSPHysics . . . 38
9.3.2 gmsh . . . 38
9.3.3 Graphviz . . . 38
9.3.4 MFiX . . . 38
9.4 Creating a portable environment . . . 39
9.4.1 Initial setup . . . 40
9.4.2 Programming languages . . . 40
9.4.3 MSYS2 . . . 42
9.5 Creating a portable launcher . . . 42

II Elmer Multiphysics **43**
Quick answers . . . 44
Limitations and issues . . . 44
Ongoing work . . . 44
Retrieving materials . . . 45

10 Elmer Fundamentals **46**
10.1 Preprocessing steps . . . 46
10.2 Using Elmer . . . 47
10.3 Parallel computing . . . 47
10.4 Tips and ideas . . . 47
10.5 Material properties . . . 48
10.6 Postprocessing . . . 49
10.7 User-defined functions . . . 50
10.7.1 Derived fields . . . 50
10.7.2 Saving data modules . . . 51
10.7.3 OpenFOAM coupling . . . 51

11 Solver Input Files (SIF) **52**
11.1 Sections . . . 52

12 MATC **53**
12.1 Declaring variables . . . 53
12.2 Control structures . . . 54
12.3 Operators . . . 54

12.4 Function definitions . . . 55
12.5 Built-in functions . . . 56
12.6 Usage . . . 59

III Ansys Fluent **60**
License usage . . . 61

13 Text user interface (TUI) **62**
13.1 Parameters . . . 63
13.2 Troubleshooting . . . 63
13.2.1 Terminal lacking interactivity . . . 63

14 User profile **65**

15 Setup from TUI **67**
15.1 Configuration . . . 67
15.2 Loading and checking a mesh . . . 67
15.3 Merging zones . . . 67
15.4 Handling named expressions . . . 67
15.5 Defining operating conditions . . . 68
15.6 Setting up some models . . . 68
15.7 Modify materials . . . 68
15.8 Setting boundary conditions . . . 68
15.9 Creating a DPM injection . . . 69
15.10 Saving the case . . . 71

16 Solution from TUI **72**
16.1 Report definitions . . . 72
16.2 Residuals . . . 72
16.3 Solver parameters . . . 72
16.4 Initialize flow and iterate . . . 73
16.5 Parallel settings . . . 73

17 Results from TUI **74**
17.1 Figures in graphical mode only . . . 74
17.2 Exporting for external processing . . . 75

18 Getting started with PyFluent **76**
18.1 Journaling . . . 76
18.2 First steps . . . 76

IV Reactor Engineering **77**

19 Acetylene pyrolysis reactor **78**
19.1 Toolbox . . . 78
19.2 Reference case . . . 78
19.3 Scan table . . . 79

20 Counter-current plug flow **81**
20.1 Introduction . . . 81
20.2 Shared functionalities . . . 81

20.3 Single plug-flow reactor . . . 82
20.3.1 Example - single PFR simulation . . . 82
20.4 Counter-current flow reactors . . . 84
20.4.1 Example - pair of coupled PFR . . . 84
20.5 Adding losses to the environment . . . 85
20.5.1 Example - environment coupling . . . 86
20.6 Registering flow for a single reactor . . . 87
20.6.1 Example - prototype with a single reactor . . . 87
20.7 Registering flow for a pair of reactors . . . 88
20.7.1 Example - extension to coupled system . . . 88

21 Fitting NASA9 data **92**
21.1 Helper utilities . . . 92
21.2 Nitrogen actual fit . . . 92
21.3 Enforcing constant specific heat . . . 93
21.4 Ensuring enthalpy continuity . . . 95
21.5 Application to a real material . . . 96
21.6 Generalizing approach . . . 99

V Finite Volume Method **105**

22 Finite Volume Method **106**
22.1 Diffusion equation . . . 106
22.2 Convection equation . . . 106

23 Diffusion in 1-D **107**
23.1 Import toolbox . . . 107
23.2 Introductory Fickean diffusion . . . 107
23.3 Internal discretization . . . 108
23.4 Boundary conditions . . . 112
23.5 Type 1 BC - Dirichlet . . . 113
23.6 Type 2 BC - Neumann . . . 115
23.7 Type 3 BC - Robin . . . 116
23.8 Arbitrary grid spacing . . . 118
23.9 Solution algorithm . . . 120
23.10 Python prototype . . . 120

24 Convection in 1-D **121**

VI Montagne **122**

25 Cartographie **123**
25.1 OruxMaps . . . 123
25.2 Liens utiles . . . 123
25.3 Visualisation des traces GPX . . . 124

26 Liens utiles **125**
26.1 Géneral . . . 125
26.2 Cartographie . . . 125
26.3 Mobilité douce . . . 125
26.4 Cyclisme . . . 126

26.5 Trail running 126
26.6 Neige et glace 126
26.7 Autour de Chamonix 126

References **127**

Kompanion

This it the *Kompanion companion* notes!

Kompanion is a portable setup toolbox for working in Scientific Computing under Windows. It is designed to be used in a laptop with no or minor administrator rights, and it allows you to install and use a wide range of tools and libraries for scientific computing without the need for a full installation.

For more details, please check our repository at GitHub. You might also be interested in Majordome, a project closely related to Kompanion (which acts as its build environment and provides installation of several packages it interacts with). Contributions and corrections are welcome, please reach me by mail.

Important: before using this, please be aware that your company/university probably forbids the use of portable tools outside sandboxed systems. It is your responsability to use this toolbox only in systems where you are allowed to. The developper does not hold any resposibility on your failure to respect your local regulations.

Foreword

For many years I have been trying to *work*... well, when you are in scientific computing you quickly discover that IT knows nothing about it and even in academic settings these guys are unaware of your needs. Those who are supposed to help you thrive are actually stopping you from even trying to do things. The *corporate* rules are too restrictive for any creative mind; *governance* means failure.

As a natural rebel, I learned everything to by-pass rules. Early in my career, when I was working only in Linux systems, that was pretty easy as you can compile your code and that is not against the rules at all in most settings. That allowed me to develop a solid background in compiling the code of others, what is not always trivial. When reading the *README* and *INSTALL* projects of major scientific computing projects, you quickly realize the inconsistencies in procedures and the available flags for compilation. When faced many and many times with this sort of issue, you are obliged to master at least `Makefiles` and `CMakefiles`. The classical `Makefiles` may be obscure with some macro expansions doing weird stuff, while `CMakefiles` can be so huge that it takes years to fully create a grasp of them.

Next, when moving into the corporate world, you will at some point be forced to go into Windows, and that was the case for me too. About 10 years ago, before the *big-bullshit era* of AI, the scientific computing for Windows was pretty poor. Even Python was quite limited, you most certainly needed to use Anaconda at that time. Most scientific libraries are harder to compile under Windows and you will probably learn about Cygwin and MSYS2 to try to overcome the issues, as getting a proper license of Windows compiler will probably be difficult unless that is your core job as a developer.

The other part of the nightmare concerns *PortableApps*: you can't simply trust those packages, and you don't want to get in trouble by bringing some viruses home. To be avoided at all costs. That's when you learn that many installers are simply zip packages and that you can extract them somewhat. It was that that gave rise to *Kompanion* (it had so many names and repositories over the years, but now we will stick with it): allow people to do scientific computing in a laptop with no or

minor (needed to run codes in parallel) administrator rights. I discuss about that in more detail in a dedicated section Chapter 9, and I strongly encourage you to read that to understand a bit what is happening under `kompanion.ps1` script without reading its thousands lines of *copy-paste* code.

Installation

Before you start

The minimal Kompanion setup includes:

- Visual Studio Code
- Python
- Git
- 7-Zip
- Curl

If that's all you need, simply open a PowerShell terminal (preferred over the legacy *Windows PowerShell*) at this directory and run `.\kompanion.ps1` (or `.\kompanion.ps1 -RebuildOnStart` if the first run fails) to install the defaults. In most cases this will work out of the box, as the links for these tools are from well known sources and are generally not filtered by corporate firewalls. If that fails, you might need to build Kompanion elsewhere and drop it in a USB drive to transfer to your workstation.

Installing the basic setup before tweaking the packages you need is a good choice, as generally you will have no proper code editor (other than the trashy Notepad) to configure `kompanion.json` file, as described in what follows. Also, if there are any errors, a minimal configuration will produce less messages and it will be easier to debug.

After succeding the preliminary installation, you can start tweaking the packages you need. For that, run `.\kompanion.ps1 -RunVsCode` from this directory. That will start VSCode with the current project open. Under `.kompanion` directory you will find a `kompanion-sample.json` which allows one to configure packages to be installed. Copy this file and rename it to `kompanion.json`, where you can play with the `true/false` flags for selecting packages. After saving the file run `.\kompanion.ps1 -RebuildOnStart` once again and wait for completion.

- General purpose tools:

Tool	Install
drawio	False
ffmpeg	False
imagemagick	False
inkscape	False
jabref	False
lessmsi	False
miktex	False
msys2	False
nteract	False
nvim	False
pandoc	False
poppler	False
quarto	False

Tool	Install
tabby	False
zettlr	False

- Programming languages:

Tool	Install
coq	False
elm	False
erlang	False
haskell	False
julia	False
node	False
racket	False
rlang	False
rust	True

- Scientific libraries:

Tool	Install
blender	False
dwsim	False
elmer	False
freecad	False
freefem	False
gmsh	False
meshlab	False
opencascade	False
paraview	False
prepomax	False
radcal	False
su2	False
tesseract	False

Note: please notice that if some package fails to download, *e.g.* firewall issues, you might need to manually clean the respective folder under `.kompanion\bin` before running again. Simply delete the directory named after the failed package. You may also need to delete the file named after that package under `.kompanion\temp` as there be a partial download there.

If you want to save some important disk space, after installation you can delete the whole `.kompanion\temp`.

Now you should be good to go!

Automatic sourcing

This section describes how to get Kompanion automatically sourced in your user profile for all PowerShell sessions. Start by creating an environment variable `KOMPANION_SOURCE` pointing to the *full-path* of file `kompanion.ps1`.

From Windows menu, search for *Environment variables for your account* and open it. Click on *New* create a variable named `KOMPANION_SOURCE`; use the *Browse file…* button to select the `kompanion.ps1` file to get the right path.

For instance, if you are called like me and cloned this repository at `GitHub\kompanion` under your profile, then its value should be `C:\Users\walter\GitHub\kompanion\kompanion.ps1`

Next, start PowerShell and identify the path to your user profile:

```
echo $PROFILE
```

If that file does not exist, create it with:

```
New-Item -ItemType File -Path $PROFILE -Force
```

Now you can run `notepad $PROFILE` to open it and append the following lines:

```
if (Test-Path $env:KOMPANION_SOURCE) {
    . $env:KOMPANION_SOURCE

    # Uncomment the following if you want the terminal to start at
    # the root of this repository; useful for development.
    # cd "$env:KOMPANION_DIR"
} else {
    Write-Host "KOMPANION_SOURCE not found at '$env:KOMPANION_SOURCE'"
}
```

Now all PowerShell sessions will automatically source Kompanion, and you can start using it right away. You can run `Kode` from any directory to open the root of this repository in VSCode.

Known limitations

- Windows MPI support (as required by SU2 in their distributed version) still requires administrator rights, as you need to install the Microsoft MPI.
- You cannot pin the editor to the taskbar (because of how sourcing environment works), but you can create a shortcut to `kode.vbs` in your desktop.

Building the notes

Notes are built using Quarto, and you can find the source code in this repository. To build the notes, you need to have that tool, Python and Rust enabled in Kompanion. Then you can follow these steps to create a virtual environment, install the required packages and build the notes:

```
# Create a virtual environment
python -m venv .venv

# On Windows, use
.venv\Scripts\Activate.ps1

# Update pip
python -m pip install --upgrade pip
```

```
# Install the required packages
python -m pip install -r requirements.txt

# Use this to generate all formats
quarto render

# Or use these to generate specific formats
quarto render --to html
quarto render --to pdf

# Finally, to publish the notes on GitHub Pages, use
# (you need to be an allowed project collaborator to do this)
quarto publish gh-pages --no-prompt --no-browser
```

Part I

Computing

Notes related to my learning and teaching interests in several fields related to scientific computing (mostly applied mathematics and machine learning) and related applications. This home page is the entry point and those interested in more content and interesting links navigate below between the multiple pages. It can also be used as a general guide for introducing scientific computing as it tries to introduce the minimal skill set any scientific computing engineer or scientist should have:

- Version control comes first, everything else is worthless without it, currently that means Git.
- Next comes software documentation with Doxygen, Sphinx, and/or Documenter.jl.
- A low(er) level programming language among Rust, C, C++, and Fortran, preferably all of them.
- Scripting languages, as of 2026, Python is mandatory, Julia highly recommended.
- Basic machine learning in one of the above scripts, everything is ML these days.
- Shell automation, basis of both Bash/other UNIX shell and PowerShell are required.
- Typesetting equations reports and presentations (beamer) in LaTeX.
- Domain specific skills related to the field of study (CFD, DFT, MD, ML, ...).

A sample setup of an operating system for scientific computing and practicing the above skills is provided in a section dedicated to Ubuntu.

Some technologies have been mainstream or important in the past, but nowadays, some of them have already died or are becoming too niche to be put in such a list. That is the case of SVN for version control. As for programming languages in science, that is the case of *matlabish* (MATLAB, Octave, Scilab) environments, which are still used by *controls and automation* people, but are mostly incompatible with good software practices and should be discouraged.

It is also worth getting familiar with high-performance computing (HPC); in the Top 500 page you can get to know the most powerful computers on Earth. The specification benchmarking page allows for the check of hardware specification, what is interesting when preparing investment in a computing structure. Lastly, when working in multi-user systems it is worth knowing about job management systems such as Slurm.

As a last word, I would like to remember that it is humanly impossible to master everything at once; even after more than 10 years in the field as of today I only have a grasp in the tools I do not use everyday. Software and methods evolve, and unless you keep using a specific tool you simply cannot afford to keep up to date with it. That should not be a roadblock for a scientist in the long term. As you get used to scientific software, getting back to a good level of some tool you used in the past is quick (but not extremely fast in some cases) and learning new tools for which you already know the science behind is trivial. Even exploring new fields become easy in some cases.

1 Git version control

1.1 Common activities

- Upon connecting for the first time in a computer, remmeber to configure:

```
git config --global user.email "walter.dalmazsilva@gmail.com"
git config --global user.name "Walter Dal'Maz Silva"

# Do not track file modes (rwx):
git config core.fileMode false

# (GitHub CLI for Linux optional)
gh auth login
```

- Some daily life reminders (you will repeat this so much it will impregnate in your brain):

```
git status

git add *

git commit -m "some message"

git checkout '<branch-name>'

git branch -d '<branch-name>'
```

- Line ending normalization: instructions provided in this thread; do not forget to add a `.gitattributes` file to the project with `* text=auto` for checking-in files as normalized. Then run the following:

```
git add --update --renormalize
```

- To create a GitHub pages (`gh-pages`) branch with no history do the following

```
git checkout --orphan gh-pages
git reset --hard
git commit --allow-empty -m "fresh and empty gh-pages branch"
git push origin gh-pages
```

1.2 Adding submodules

Generally speaking adding a submodule to a repository should be a simple matter of

```
git submodule add 'https://<path>/<to>/<repository>.git'
```

Nonetheless this might fail, especially for large sized repositories; I faced this issue which I tried to fix by increasing buffer size as reported in the link. This solved the issue but led me to another problem which could be solved by degrading HTTP protocol.

The reverse operation run:

```
git rm '<path-to-submodule>'
```

It seems that in modern git versions that is enough. In the past it could not be fully automated as discussed here. After running the above you manually remove the history with

```
rm -rf '.git/modules/<path-to-submodule>'

git config remove-section 'submodule.<path-to-submodule>'
```

For managing submodules, the following might be useful:

```
# You added the submodule in another computer, now sync it here:
git submodule update --init --recursive

# Sync submodule to head of remote:
git submodule update --remote --merge
```

1.3 Other tips

- Version control in Windows: for Windows users, TortoiseGIT adds the possibility of managing version control and other features directly from the file explorer.

2 Using SSH

There is something almost inevitable in scientific computing: you will need to connect to some remote machine at some point. Most of the time, that is a daily activity, whether you connect to a remote computer or HPC cluster. The most common protocol for such connections is SSH. This note provides some insights in common usage of SSH.

2.1 Port forwarding

When running applications in a remote server where you have no administration rights (often the case in a scientific cluster), you might need to redirect a port through an SSH tunnel so that it is accessible from your local workstation. The following snippet illustrates the command format as pseudo-code:

```
ssh -L \
    "<local_port>:localhost:<remote_port>" \
    "<REMOTE_USER>@<REMOTE_HOST>"
```

For example:

```
ssh -L 8080:localhost:8080 walter@192.168.1.10
```

2.2 Key pair authentication

This section guides you through the generation of a key pair generation and authentication. Additional steps to setup a connection through VS Code are provided.

Creating the keys: generate the SSH key pair locally (*i.e.* on your workstation); common options are:

- `-t rsa`: key type (RSA is widely supported)
- `-b 4096`: key length (more bits = stronger, recommended 4096)
- `-C` : comment (usually your email)

When running the command, accept defaults for storage at `~/.ssh/id_rsa[.pub]`; optionally add a passphrase for additional security (but then you will need to enter it each time you need to connect, so that's undesirable if the only reason you are creating the SSH key is to have quick access to the server).

```
ssh-keygen -t rsa -b 4096 -C "yourusername@your.server.com"
ssh-keygen -t ed25519 -b 4096 -C "yourusername@your.server.com"
```

If you have password access to the server and `ssh-copy-id` run the following:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub user@remote_host
```

Alternatively (in Windows PowerShell for instance but reformat it in a single line or replace the pipes by backticks) manually append to the `~/.ssh/authorized_keys`:

```
cat ~/.ssh/id_rsa.pub | \
    ssh yourusername@your.server.com \
    "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

As a last option do it by hand, but you risk breaking the format of `authorized_keys`.

Testing Linux server: before anything, try connecting with you identity:

```
ssh -i ~/.ssh/id_rsa yourusername@your.server.com
```

If that falls-back to your password connection, connect normally to the server and make sure the rights of both SSH directory and authorized keys file are right before trying again:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Maybe the server SSH has not been enabled for key authentication, which can be inspected without opening the actual configuration file through (requires `sudo` rights):

```
sudo sshd -T | grep pubkeyauthentication
```

If it is not enabled, you can edit the file (find and modify `PubkeyAuthentication yes`) as follows and restart the service:

```
sudo vim /etc/ssh/sshd_config
sudo systemctl restart sshd

# Additional step for SELinux only:
restorecon -Rv ~/.ssh
```

Test again; upon new failure, try the verbose mode of SSH connection on your workstation:

```
ssh -v yourusername@your.server.com
```

while simultaneously connected to the server (`sudo`) reading the logs:

```
# Debian-based:
sudo tail -f /var/log/auth.log

# Under RHEL/CentOS/Fedora:
sudo tail -f /var/log/secure
```

Adding the key to VS Code by perform the following steps:

- Install `Remote-SSH` extension
- Press `F1` and search for `Remote-SSH: Open SSH Configuration File`
- Add an entry like the following (modifying the host name and user):

```
Host myserver
    HostName your.server.com
    User yourusername
    IdentityFile ~/.ssh/id_rsa
```

If the above fails to fill in your right user name (sometimes Windows username will appear) you can try the following workaround to enforce user:

```
Host yourusername@your.server.com
    HostName your.server.com
    User yourusername
    IdentityFile ~/.ssh/id_rsa
```

About cluster usage: that is the single case you might want to store both public and private keys at the same `.ssh`; to navigate across nodes (assuming your `$HOME` directory is the same) you need both keys. Please keep in mind to use a different key pair than the one you use to connect to the cluster for security reasons.

3 Scientific programming

At its core, serious *scientific computing* requires *scientific programming*. To that end, this page provides access to programming learning materials and related links. If this subject is new to you, to be able to successfully follow the contents you might learn a bit about the environment we will use, VS Code 4 and the minimum about command prompt on Chapter 5.

It is not worth learning any programming before being introduced to the good practices. Many programmers I know write *garbage that works for them only*. It is impossible to have a healthy collaboration if code is not standardized, reason why I place this highly biased introduction here.

One of the reasons Guido van Rossum created Python is because he wanted code to be readable. You should be able to guess what some code is doing even without specific technical knowledge about the language. This is probably the mean feature that made its creation so popular in the scientific world.

Although they are applicable to Python, the practices recommended in the famous PEP8 can be extended to other languages, including Julia. You should **read PEP8 religiously**. That document describes how to write clean and maintainable Python code. When transposing that to Julia, the minimum you are expected to do is:

- lines are limited to 79 characters
- use spaces around all operators
- consistent indentation with spaces
- blank lines around structural blocks
- lower case variable names
- Pascal-case structure names
- use underscore to separate words
- document functions properly

When you code, remember that most of the time what you are doing will be reviewed/used by somebody else and that person might not be in the mood to decipher the cryptic code you wrote; it that person is myself, **I will promptly refuse to help you** with badly written code. For newcomers, it is always better to talk about this before you write your first lines because once you stick to bad practices you will hardly ever leave them. Before you write something new, check if your ideas are also consistent with PEP20.

Julia has its own stylistic conventions that are simpler than PEP8; the main differences are the way to name functions (it recommends to *glue* words and use no underscore) and the *exclamation mark !* indicating a function modifies it(s) argument(s). For function naming you may chose to stick to PEP8 recommendation, what is my personal choice. The detailed document is found here.

Python code documentation is generally done with Sphinx. Julia has its own syntax which can be used to generate package documentation with help of Documenter.jl.

Important: Julia supports Unicode input, but its use is highly discouraged in modules. Unicode characters are better suited to write application scripts such as notebooks (in Pluto or Jupyter).

4 Visual Studio Code

If you are reading this, you are probably using VS Code for the first time or need a refresher! VS Code is Microsoft's open source text editor that has become the most popular editor in the past decade. It is portable (meaning it works in Windows, Linux, and Mac) and relatively light-weight (it won't use all you RAM as some proprietary tools would do). If you are doing Data Science or wish a more interactive environment, Posit has made a spin-off VS Code called Positron that is very similar to use and comes bundled with a few extra functionalities.

There are a few shortcuts you might want to keep in mind for using this tool in an efficient manner:

- `Ctrl+J`: show/hide the terminal
- `Ctrl+B`: show/hide the project tree
- `Ctrl+Shift+V`: display this file in rendered mode
- `Ctrl+Shift+P`: access the command pallet
- `Ctrl+K Ctrl+T`: change color theme
- `Alt+Z`: toggle column wrapping

A few more tips concerning the terminal:

- `Ctrl+L` gives you a clean terminal (also works inside Julia prompt)
- `Ctrl+D` breaks a program execution (*i.e.* use to quit Julia prompt)

If you copied a command from a tutorial, you **CANNOT** use `Ctrl+V` to paste it into the terminal; in Windows simply right-click the command prompt and it will paste the copied contents. Linux users can `Ctrl+Shift+V` instead.

Notice that `Ctrl+M` will toggle the visibility of the integrated terminal; if you accidentally press it, autocompletion will stop working in terminal. Just press it again and normal behavior will be recovered.

4.1 VS Code extensions

VS Code supports a number of extensions to facilitate coding and data analysis, among other tasks. Local (user-created) extensions can be manually installed by placing their folder under `%USERPROFILE%/.vscode/extensions` or in the equivalent directory documented here. Below you find my recommended extensions for different purposes and languages.

The following extensions are recommended.

- General purpose:
  - ms-vscode.PowerShell
  - ms-vscode-remote.remote-ssh
  - GitHub.copilot
  - ukoloff.win-ca
  - vscode-icons-team.vscode-icons

  - ritwickdey.LiveServer
  - dnut.rewrap-revived

- Python development:
  - ms-python.python
  - ms-toolsai.jupyter
  - tamasfe.even-better-toml
  - Pyrefly
  - Ruff

- Other languages support
  - C/C++
  - Erlang
  - F#
  - Gmsh
  - Julia
  - LaTeX Workshop
  - Matlab
  - Modern Fortran
  - Quarto
  - Ruby

- Data inspection:
  - Data Wrangler
  - Excel Viewer
  - Rainbow CSV

- VS Code themes:
  - lakshits11.ayu-monokai
  - cameronbieganek.julia-color-themes
  - GitHub.github-vscode-theme

I have also developed a few (drag-and-drop) extensions; in the future I plan to provided them through the extension manager.

- wallytutor/elmer-sif-vscode: VS Code extension for Elmer Multiphysics SIF

A few tips for installing extensions:

- If installing local extensions (such as *elmer-sif-vscode*), consider using Developer: Install Extension from Location instead of placing it under `.vscode/extensions` manually.
- For installing extensions from within a VSCode terminal, rather than the main executable one must use `%VSCODE_HOME%\bin\Code.cmd`. Currently, Visual Studio Code's command-line interface doesn't directly support installing extensions from a local directory without packaging them into .vsix files. The recommended method is to package your extension into a .vsix file using vsce and then install it.

5 Working on Windows

5.1 Command line basics

If this is you first time using the command prompt you might be interested by this section. The command prompt (often referred to as *terminal* in Linux world) is your interface to interact with the operating system and many available tools. To learn any useful scientific computing skills it is useful to get a grasp of its use because it is there that we will launch most applications. The illustrations below assume you are working under Windows, but the introductory commands are common to most operating systems.

Now let's launch a terminal. If you are working under VS Code you can use the shortcut to display the terminal `Ctrl+J`; the bottom of your window should display something as

```
PS D:\Kompanion>
```

The start of this line displays you *path* in the system; depending on your configuration that could not be the case and you can ask the OS to give you that with `pwd` (print working directory)

```
PS D:\Kompanion> pwd

Path
----
D:\Kompanion
```

If you are invited to move to directory `src` you may which to use command *change directory*, or `cd` in the system's language

```
PS D:\Kompanion> cd .\bin\
PS D:\Kompanion\bin>
```

Now that you reached your destination, you might be interested at inspecting the contents of this directory, *i.e.* listing its contents; that is done with `ls` as follows

```
PS D:\Kompanion\bin> ls


    Directory: D:\Kompanion\bin


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         1/31/2025  11:11 AM                apps
d-----          2/3/2025   9:19 AM                data
d-----         1/30/2025   2:34 PM                downloads
d-----          2/3/2025  11:50 AM                pkgs
d-----         1/31/2025   9:33 AM                scripts
```

```
d-----         1/30/2025   9:58 AM                tests
-a----         1/31/2025   9:33 AM           2697 activate.bat
-a----         1/30/2025   9:58 AM            161 code.bat
-a----         1/30/2025   9:58 AM            132 kode.bat
-a----         1/30/2025   9:58 AM            131 kpip.bat
```

Oops! It was not the directory you wanted to go to! No problems, you can navigate *one-level-upwards* using the special symbol `..` (two dots) and change directory again

```
PS D:\Kompanion\bin> cd ..\docs\
PS D:\Kompanion\docs>
```

This is the minimum you need to know: navigate, know your address, inspect contents.

5.2 Mount a network drive in WSL

Here we assume we will mount drive `Z:` at `/mnt/z`:

```
# Create the mount point (if required):
sudo mkdir /mnt/z

# Mount the network drive in WSL:
sudo mount -t drvfs Z: /mnt/z
```

Actually the same procedure can be used to access a SMB drive from within WSL temporarily with:

```
sudo mount -t drvfs '\\path\to\smb' /mnt/<mount-point>/
```

For automatic remount, consider adding the following to your `/etc/fstab`

```
//path/to/smb /mnt/<mount-point>/ drvfs auto,rw,nosuid,exec,uid=1000,gid=1000 0 0
```

5.2.1 Finding a process handle

This is useful when Windows won't let you move a file or folder because *it is already open somewhere*. First, download and extract Handle; from PowerShell run the following:

```
./handle.exe -u -nobanner "C:\Path\To\File.txt"
```

Notice that your file might have started another process and some research might be required.

5.3 Identifying a proxy PEM

```
# List all root certs:
Get-ChildItem -Path Cert:\LocalMachine\Root

# Export a specific one:
$cert = Get-ChildItem -Path Cert:\LocalMachine\Root | Where-Object {
    $_.Subject -like "*<company-name-generally>*"
}
Export-Certificate -Cert $cert -FilePath "proxy.pem"
```

6 Using containers

Containers allow for reproducible workflows that can be shared among different users and computing environments. They encapsulate applications and their dependencies, ensuring consistent behavior across various systems. This is particularly useful in scientific computing, where complex software stacks are common. Keeping the focus on open source tools, this section introduces two popular containerization technologies: Podman and Apptainer.

For isolating applications and still allow a nice user-space interaction, mixing both tools is great approach; one first creates and dump a container with `podman`, then converts it to Singularity format with `apptainer` for portable deployment. Installation instructions for both tools can be found on their respective websites; for `podman` the package manager is often the way to go, while for `apptainer` downloading pre-compiled binaries is usually the easiest path. The tool started at Lawrence Berkeley National Laboratory can be downloaded for several Linux systems and deployed locally.

Under Debian (or its variants, as Ubuntu), navigate to the download directory and install with the following, taking care to replace the right version (check the repository above for the right version and possible name changes or alternative versions):

```
version="1.4.4"
url="https://github.com/apptainer/apptainer/releases/download"
pkg="apptainer_${version}_amd64.deb"

wget "${url}/v${version}/${pkg}"
sudo apt install fakeroot
sudo dpkg -i ${pkg}
```

Below we summarize the most relevant commands and workflows for daily use.

6.1 Containerfile creation

WiP: a section on how to create a Containerfile should be added here.

6.2 Using Podman

In the open source community podman takes the place of docker for the creation of application containers. It mimics the commercial software to allow developers work with both tools *almost* interchangeably. It is not uncommon to find people creating aliases of `docker` in their sandbox environments to point to their `podman` executable (some Linux distributions even have packages dedicated to this automatic override). One must be aware that although the command interfaces are very similar, they are not the *exactly* same and advance usage requires mastering each of them individually.

The following summarizes some daily life commands with `podman`.

- List available images in a local machine:

```
podman images
```

- Run image <img> interactively using bash:

```
podman run -it '<img>' /bin/bash
```

- Run image exposing port <container> to host at <host>:

```
podman run -p '<container>:<host>' '<img>'
```

- Dump image <img> to <img>.tar for portability:

```
podman save -o '<img>.tar' '<img>'
```

- List all available containers (there might be external/hidden, so use -a):

```
podman container ls -a
```

- Remove a given container by ID (only the first 2-3 characters of ID are required):

```
podman container rm '<ID>'
```

- Remove a given image by ID:

```
podman rmi '<ID>'
# podman image rm '<ID>'
```

- Clean all the cache:

```
podman builder prune
```

6.3 Podman with GPU

> ⚠ WSL not supported
>
> Notice that podman GPU support will not work on WSL. In that case you need to use the actual docker; please refer to the next section for details.

For supporting GPU (NVidia) you need to install `nvidia-container-toolkit` first:

```
NVIDIA_BASE="https://nvidia.github.io/libnvidia-container"
NVIDIA_GPG="/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg"

curl -fsSL "${NVIDIA_BASE}/gpgkey" | sudo gpg --dearmor -o ${NVIDIA_GPG}
curl -s -L "${NVIDIA_BASE}/stable/deb/nvidia-container-toolkit.list" \
    | sed "s#deb https://#deb [signed-by=${NVIDIA_GPG}] https://#g"  \
    | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```

The Achiles heel is the configuration below, that might break from times to times:

```
sudo nvidia-ctk runtime configure --runtime=podman
sudo systemctl restart podman
```

Some tips for fixing it

First inspect if podman is using crun instead of runc; the NVIDIA hook will not attach to crun.

```
podman info --format '{{.Host.OCIRuntime}}'
```

If that is the case edit /etc/containers/containers.conf by changing the engine as follows:

```
[engine]
runtime = "runc"
```

You mignt also need to install runc before restarting podman:

```
sudo apt-get install -y runc
sudo systemctl restart podman
sudo systemctl status podman
```

Maybe also consider reinstalling nvidia-container-toolkit:

```
sudo apt-get purge nvidia-container-toolkit
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit
```

Now you can test with an image matching your driver version (change below):

```
podman run --rm -it \
  --security-opt=label=disable \
  --hooks-dir=/usr/share/containers/oci/hooks.d \
  docker.io/nvidia/cuda:12.6.3-cudnn-runtime-ubuntu24.04 \
  bash
```

6.4 Sidenote on Docker

As we saw in the previous section, sometimes we are obliged to fall back to docker. In general the CLI of both docker and podman are quite similar (at least with regards to what was presented in the introduction). When working with docker a first thing you might want to to is enable *rootless* containers by adding your own user name to the docker group:

```
sudo usermod -aG docker $USER
```

Once done, you will need to close and restart WSL or your shell session (in case of native Linux). That will allow containers to be run without rights elevation.

Below we illustrate the use of --gpus flag that allows exposing a device to the container (as a fallback to what we cannot do with podman in WSL). Before testing, please keep in mind that the image IMAGE must have the same driver version as your local machine, so locally run nvidia-smi to check for that then visit NVidia docker page to identify the image you need.

```
IMAGE="docker.io/nvidia/cuda:12.6.3-cudnn-runtime-ubuntu24.04"
docker run --gpus all ${IMAGE} nvidia-smi
```

Sometimes you might encounter a situation where there is no possibility to change the host of an application running in a container, *i.e.* it binds only to `localhost/127.0.0.1`, so port forwarding in *bridge* mode is not an option. For instance, you need to build ParticleAnalyzer yourself to have it modified to run from a container. In that case you can add the `--network host` option without port mapping so that the same local network is shared. That option is a security issue and must be avoided.

> ⚠ Docker on Rocky Linux 9
>
> For installing `docker` on Rocky Linux 9, follow the steps below:
>
> ```
> dnf check-update
> dnf config-manager \
>  --add-repo https://download.docker.com/linux/centos/docker-ce.repo
>
> dnf install docker-ce docker-ce-cli containerd.io
>
> systemctl start docker
> systemctl status docker
> systemctl enable docker
>
> # Optional : allow non-root user to run docker commands
> # usermod -aG docker $USER
> ```
>
> To enable GPU support with `docker` on Rocky Linux 9, install Nvidia container toolkit as follows:
>
> ```
> toolkit="https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-toolkit.
> curl -s -L $toolkit | tee /etc/yum.repos.d/nvidia-container-toolkit.repo
>
> dnf install -y nvidia-container-toolkit
>
> nvidia-ctk runtime configure --runtime=docker
> systemctl restart docker
> ```

6.5 Using Apptainer

Using `podman` locally is great, but packaging redistributable containers for reuse in HPC is much smoother with Apptainer. Although Apptainer has its own image scripting system through *definition files*, personal experience has shown that the workflow is much smoother by generating container files and then converting them to Singularity format as explained above.

There reason is that container files generate intermediate check-points from which they will continue the build if some failure is encountered, *i.e.* each `RUN` command in a container will generate a partial image.

When working with Apptainer definition files, failures imply full rebuild of the image, what might become extremely boring when trying to compile new code. A workaround is to use a sequence of definition files, one importing from the dump of the previous one, but that not only will generate a large size of temporary dumps as it will become difficult to manage.

After getting excited by the Apptainer definition files because they do not need chaining of commands with a && \ to make a shell block, I personally gave up on them after loosing a few days of my life recompiling again and again… so for now I stick with the container creation and conversion workflow discussed in more detail below.

- Converting a local podman *tar-dump* into a Singularity image:

```
apptainer build "<img>.sif" "docker-archive://<img>.tar"
```

- Running an apptainer image as a non-root user is as simple as:

```
apptainer run '<img>.sif'
```

- Another option with Apptainer is to run instances on the background, as follows:

```
# Provide the instance name:
instancename='<file-name>'

# Start instance on the background:
apptainer instance start -B $PWD "${instancename}.sif" "${instancename}"

# Enter instance in shell mode:
apptainer shell "instance://${instancename}"

# Stop instance after working:
apptainer stop "${instancename}"
```

Other useful/relevant commands in this context are `apptainer instance list` and `apptainer instance stop <instance-name>`. In case additional packages may be required after the instance creation, one can use a temporary file system with `--writable-tmpfs`. For configuring its maximum size, check the docs before modifying `/etc/apptainer/apptainer.conf` parameter `sessiondir max size`.

Since `apptainer` makes use of *user space*, sourcing of applications is not done as root, so one must edit add to their `~/.bashrc` if path configuration is desired and re-source that file when activating a container. For instance, the required environment variables for #OpenFOAM are provided by `FOAM_SOURCE` file given below; in the *host system* outside the container it does not exist, so adding a test in `~/.bashrc` is required. Once you activate the container with `apptainer run <image-name>.sif`, by calling `source ~/.bashrc` the environment will be properly set.

```
FOAM_SOURCE=/opt/openfoam13/OpenFOAM-13/etc/bashrc

[[ -f ${FOAM_SOURCE} ]] && source ${FOAM_SOURCE}
```

Another approach is to execute the SIF image once, source the required variables required in the container, dump `env > draft.env`, edit the file as required and then wrap a call with contextualized environment as:

```
function openfoam12() {
    FOAM_NAME=$HOME/Applications/openfoam12-rockylinux9
    apptainer run --cleanenv --env-file ${FOAM}.env ${FOAM}.sif
}
```

6.6 Build workflow

Using both tools can be roughly automated by generating a `podman` image, dumping it into a portable format, then converting to Singularity format. Below we illustrate the workflow for an arbitrary container file; this is summarized in a bash script containerfile.sh which makes use of Containerfile.

Now you can move the SIF image to another computer (for instance, you prepared this in a PC with access to the Internet for later using it in an isolated HPC), launch a terminal and run:

```
apptainer run -B $PWD '/path/to/project/image.sif'
```

Notice that `apptainer` does not resolve symbolic links, so `$PWD` above will fail if trying to run from a path that contains a link; navigate to the actual directory containing the project before running the image to get your files visible.

Note: use `apptainer run` when you want to execute the container's default application or task; on the other hand, use `apptainer shell` when you need an interactive session to explore or debug the container.

7 General Tips

7.1 Running Jupyterlab from a server

Before running the server it is a good idea to generate the user configuration file:

```
jupyter-lab --generate-config
```

By default it will be located at `~/.jupyter/jupyter_lab_config.py`. Now you can add your own access token that will simplify the following steps (and allow for reproducible connections in the future).

```
c.IdentityProvider.token = '<YOUR_TOKEN>'
```

The idea is illustrated in this thread; first on the server side you need to start a headless service as provided below. Once Jupyter starts running, copy the token it will generate if you skipped the user configuration step above.

```
jupyter-lab --no-browser --port=8080
```

On the host side (the computer from where you wish to edit the notebooks) establish a ssh tunel exposing and mapping the port chose to serve Jupyter. Instructions are provided in Chapter 2. Now you can browse to `http://localhost:8080/` and add the token you copied earlier or your user-token you added to the configuration file. Notice that you need to keep the terminal you used to launch the port forwarding open while you work.

7.2 Downloading from YouTube

#programming/python/tips

Retrieving a video or playlist from YouTube can be automated with help of yt-dlp.

To get the tool working under Ubuntu you can do the following:

```
# Install Python venv to create a local virtual environment:
sudo apt install python3-venv

# Create an homonymous environment:
python3 -m venv venv

# Activate the local environment:
source venv/bin/activate

# Use pip to install the tool:
pip install -U --pre "yt-dlp[default]"
```

NOTE: alternative applications as youtube-dl and pytube are now considered to be legacy as discussed in this post.

7.3 Installing Python packages behind proxy

#programming/python/tips

To install a package behind a proxy requiring SSL one can enforce trusted hosts to avoid certificate hand-shake and allow installation. This is done with the following options:

```
pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org <pkg>
```

7.4 Regular expressions

Regular expressions (or simply *regex*) processing is a must-have skill for anyone doing scientific computing. Most programs produce results or logs in plain text and do not support specific data extraction from those. There *regex* becomes your best friend. Unfortunately during the years many flavors of regex appeared, each claiming to offer advantages or to be more formal than its predecessors. Due to this, learning regex is often language-specific (most of the time you create and process regex from your favorite language) and sometimes even package-specific. Needless to say, regex may be more difficult to master than assembly programming.

Useful tools:

- regex101
- regexr

Matching between two strings: match all characters between two strings with lookbehind and look ahead patterns. Notice that this will require the enclosing strings to be fixed (at least under PCRE). For processing `WallyTutor.jl` documentation I have used a more generic approach but less general than what is proposed here.

Match any character across multiple lines: as described here with `(.|\n)*`.

Regex in Julia: currently joining regexes in Julia might be tricky (because of escaping characters); a solution is proposed here and seems to work just fine with minimal extra coding.

7.5 Using nteract with a virtual environment

> ⚠ Warning
>
> nteract project underwent a major restructuration and the following instructions might not work anymore.

```
# Activate the virtual environment:
. .venv\Scripts\Activate.ps1

# Give the kernel an unique name:
$kName = "your-kernel-name"

# Install the kernel:
python -m ipykernel install --user `
    --name $kName --display-name "Python ($kName)"
```

```
# Run nteract:
nteract.exe
```

8 Selection of tools

8.1 Scientific publishing

The following tools might be of interest for creating scientific content with embedded code.

- Jupytext documentation
- Jupyter Book
- Weave.jl - Scientific Reports Using Julia
- Franklin - Building static websites in Julia
- Pluto.jl - interactive Julia programming environment
- Quarto

8.2 Scientific web apps

Some practical tools for creating web applications for scientific applications:

- Gradio
- Panel
- Streamlit
- Trame
- Voilà

8.3 API and databases

Python:

- FastAPI
- Pydantic
- TinyDB

Rust (frameworks):

- Actix
- Axum
- Rocket

Rust databases:

- redb
- ReefDB
- sled

8.4 Graphics

- Fiji
- gnuplot
- Graphviz

8.5 Geometry

- blender: the most powerful 3D (in the general sense) open source modeling tool; allows geometries to be exported to STL, which is compatible with most meshing software.
- FreeCAD: contrarily to blender, this is the most mature open source modeling tool in the technical sense. It supports both 3D conception and detailed drawing, among other features.
- cadquery: a simple parametric geometry tool.
- Salome: not just grometry, but also a large meshing toolbox provided by EDF.

8.6 Meshing

- gmsh: the *to-go* meshing tool for 2D geometries and visualization of many formats of 3D meshes; before trying to produce reliable structured meshes and geometry in 3D some ninja skills need to be developed. Its own scripting language makes parametric meshing easy.
- MeshLab: allows to manipulate triangulated grids generated in CAD; helpful for preparing patches for use with #OpenFOAM/snappyHexMesh.

8.7 Rendering

- ParaView: the *de facto* post-processing tool for many fields of application.
- trame: rendering results in web-applications.

8.8 Simulation

- Cantera
- CasADi
- DualSPHysics
- DWSIM
- Elmer: multiphysics FEM toolkit (see dedicated topic); download it here
- FreeFem++
- LAMMPS
- MFiX

- OpenFOAM: general purpose FVM CFD toolkit (see dedicated topic).
- OpenModelica
- RadCal
- SU2
- TRUST Platform: the basis for TrioCFD code by CEA.

8.9 Productivity

Obsidian is the *de facto* solution for note-taking and *second brain* management, but it is not free for commercial ends and that has become a problem for my intended work use. Looking for alternatives for this tool which is my main productivity setting, I came across the following packages. Testing was done with *I want it to be the same* mindset and if after a few minutes I was not convinced by the application, it was automatically discarded. In summary, I liked both *Joplin* and *Zettlr* but will pursue the use of the latter only as *Joplin* does not meet by criteria. *StandardNotes* is a false open-source package and *logseq* is still too raw for any production setting.

Software	Pros	Cons
laurent22/joplin	Rich interface with many features; excellent tool if you are not seeking version control as a target. Available as portable executable.	Counterintuitive interface and files are not directly stored as `.md`; file system synchronization requires and absolute path. After closing the executable, a process was kept alive.
logseq/logseq	Available as portable executable.	Poor UI at first sight. Fast but *dumb* in the sense it will only support *one file* with a given title, what is incompatible with my way of organizing directories. Pages are not organized as in the folder view.
standardnotes/app	Interface is cleaner/less cluttered than Joplin. Available as portable executable.	Stopped using it as received the first notification that smart tags require a paid plan. Also only plain text files are supported in free mode.
Zettlr/Zettlr	Support to YAML frontmatter. UI gets better as you open files. Integration to BibTeX.	Poor UI at first sight. Not available as portable executable; but installation can be done in any user folder, what is also fine. It took a long time to import my existing *second brain* and sometimes it glitches/has some lags.

8.10 Other

- protobuf: for parsing #OpenFOAM dictionaries from Python.

9 Portable applications

9.1 How does it work?

Kompanion is intended to be used as a portable environment with *almost* all batteries included. Applications are placed under `.kompanion/bin/` directory and all configuration of paths and other required environment variables are provided by *kompanion.ps1* script.

It mostly revolves around VS Code. You launch the editor through a dedicated script and everything else is available in its terminal. That said, the whole of the ecosystem is command line based - but this is scientific computing, if you don't know command line you are in the wrong job. If you followed the instructions to source Kompanion from $PROFILE, then you can also launch the terminal from the start menu and everything will be available there as well.

It is intended to leave minimal track on host system (probably a few files and directories on your user home directory or under *AppData*), but this is not enforced in its development, so take care if you are not allowed to execute some software in a given computer.

9.2 Other possibly supported applications

The following packages are not currently included in Kompanion, but they are known to be somewhat portable and they might be added in the future. Those displayed in italics cannot be currently automated, but they can be installed in a machine then copied to Kompanion (ongoing investigation on how to automate that). Also check the Chapter 8 for a software selection that might be of interest for you and maybe not listed here.

- DualSPHysics
- Dyssol
- Fiji
- FileZilla
- *FreeFEM++*
- gnuplot
- Graphviz
- iperf
- Ipopt
- Jamovi
- JASP
- lazarus
- lite-xl
- LAMMPS
- MFiX
- MSYS2
- MUSEN
- Notepad++
- Octave

- OpenCALPHAD
- OpenModelica
- Orange3
- Portable LISP
- puTTy
- Ruby
- Rufus (requires admin privileges to be used though)
- SALOME (needs to be manually downloaded)
- *Scilab*
- strawberry-perl
- VMD
- ZeroBraneStudio
- Zettlr
- Xpra

9.3 Software specific instructions

9.3.1 DualSPHysics

For DualSPHysics one might also want to install this FreeCAD addon and this Blender addon.

9.3.2 gmsh

Because it was chosen that SDK version of gmsh was more adapted to integrate Kompanion, if you want to be able to pin the executable to your taskbar you need to copy the DLL from its `lib/` directory to `bin/`. If you think you will not need Python/Julia interfaces of gmsh, then you can move the DLL instead to save ~80MB of disk (and you can always install gmsh with pip for those cases).

9.3.3 Graphviz

If the first time you run `dot` you get a message as

```
There is no layout engine support for "dot"
Perhaps "dot -c" needs to be run (with installer's privileges) to register the plugins?
```

Simply run `dot -c` as suggested and it should work fine (without admin privileges).

9.3.4 MFiX

This package is installed in a separate environment and not added to the toolbox path; that is because other Python environments could interact with each other and lead to unpredictable behavior. The following instructions are provided:

- Install miniforge3 under a dedicated directory.
- Add a script with the following content to launch miniforge console:

```
%windir%\system32\cmd.exe "/K" ^
    %~dp0\miniforge3\Scripts\activate.bat ^
    %~dp0\miniforge3
```

- Go to the downloads page, login and copy the personal installation command that looks like the following:

```
conda create -n mfix-<version> ^
    mfix==<version>            ^
    mfix-doc==<version>        ^
    mfix-gui==<version>        ^
    mfix-solver==<version>     ^
    mfix-src==<version>        ^
    -c conda-forge -c          ^
    https://mfix.netl.doe.gov/s3/<personal-token>//conda/dist
```

- To run the software activate the created environment and call its executable:

```
conda activate mfix-<version>
mfix
```

9.4 Creating a portable environment

This section illustrates the generalities of how do create a portable software toolbox. Before proceding with any installation, make sure to read all the elements provided below; simply trying to follow instructions in a step-by-step fashion will certainly lead to failure and frustration.

For each of the applications you will install, make sure to perform the following generic steps:

1. Download the portable version of the application, generally a `.zip` compressed file, and save it to some temporary location. Sometimes the normal installer of an application already supports portable mode.
2. Extract the compressed file to a dedicated directory. From there, you can move the content to the final location. See the notes below for details about compression practices.
3. Configure the application in a dedicated script or add a routine to the application script that manages all the paths and settings. The latter approach is recommended for putting everything together and is how Kompanion currently works.

Important: You need to be aware that there are two common practices of compressing (zip) a portable software: (1) some editors place everything in a directory and compress the directory, while others (2) pack everything under the root of the compressed file. Windows built-in system allows your to navigate compressed files as part of the filesystem, so inspect which of the above methods was used. If the later is detected, take care to select the right option to ensure the contents are extracted to a new directory. Several packages are stored directly at *zip* root and that may be messy to clean if extracted without the above precautions.

Important: since setting up the environment may require editing batch scripts, be aware that to open them you cannot *click* the file, but *right-click* and *edit*, as Windows see these as executables. In these files, lines starting by `@REM` are seem as comments and generally explain something about what follow them. If you are starting from scratch and do not master batch scripting, prefer learning PowerShell instead, as it is more modern and powerful, and you can easily call PowerShell scripts from batch scripts. It is how the current version of Kompanion is implemented and it is the recommended way to go.

9.4.1 Initial setup

These are the required steps to get your system working for the first time:

1. Go to VS Code download page and get the `.zip` version for `x64` system (`Arm64` is not supported here). Extract it to the binaries directory and copy the path of VS Code directory the set the variable `VSCODE_DIR`.
2. Go to Git download page and select *64-bit Git for Windows Portable*, download it. Notice that this is not a compressed file *per se*, but it is desguised as an executable. Double-click it and accept the default `PortableGit` installation directory. After extraction finishes, move it to binaries directory. Add it to the path by setting the variable `GIT_DIR` to the path of the directory.

Important: After installing `VS Code`, do **NOT** enable its portable mode because the VBS file for launching it will point to a specific user-data folder.

Important: these notes are quite general and skip details. Refer to `kompanion.ps1` and look for the functions `Invoke-Configure*` and `Invoke-Install*` for the exact steps to configure each application.

9.4.2 Programming languages

The following provides instructions for some languages you might wish to have fully integrated to your environment. For lower level languages, please consider using MSYS2 as described in its dedicated section.

9.4.2.1 Python

Go to WinPython download page and find the latest version tagged with a *Latest* green flag (avoid *Pre-release* versions on top). Expand the *Assets* and download the `.exe` version (also a disguised compressed file). Follow steps similar to Git above. See `Invoke-ConfigurePython` in `kompanion.ps1` for the configuration of paths and environment variables.

9.4.2.2 Julia

Go to Julia download page and select the latest stable Windows 64-bit portable version. See `Invoke-ConfigureJulia` in `kompanion.ps1` for the configuration of paths and environment variables.

Download rustup-init following the installation. See `Invoke-ConfigureRust` in `kompanion.ps1` for the configuration of paths and environment variables.

9.4.2.3 Octave

Go to Octave download page and identify the `.zip` package for `w64`. As the main use of Octave is through its user interface, you can extract the content directly to `bin/octave`.

9.4.2.4 Ruby

Go to Ruby download page and identify the `Devkit` package for `x64`. During installation change the path to point to binaries and unselect the options to associate extensions and adding to the path.

9.4.2.5 LISP

LISP may be installed through Portacle. Instead of right-clicking and installing the executable, Shift-click and select to extract the contents; place the resulting `portacle` directory. You may choose not to add it to the path if using through its EMACs interface.

9.4.2.6 LaTeX and related

Although LaTeX is not mandatory, it is highly encouraged; otherwise, what is the point of doing any scientific computing and not publishing its results?

- Start by following the instructions provided here to install MikTeX. Point the installation to a directory `miktex-portable` under `apps` and select *on-the-fly* installation of new packages.
- Consider installing the following additional software:
  - JabRef
  - pandoc
  - inkscape
- Globally install `pip install Pygments` for enabling syntax highlight in LaTeX using `minted`; that is the most flexible highlighting method for adding code snippets to your documents.
- Also consider installing extension LaTeX Workshop for automatic compilation in editor with built-in PDF visualization.

To append to `TEXMF` variable one can use the MiKTeX Console graphical interface and under `Settings > Directories` navigate and select the local path.

If you prefer a dedicated LaTeX editor, you may wish to install texstudio; some configuration of paths with the application may be required as it is not directly integrated to this environment.

Warning: for now having both Rust and Octave fully operational in command-line is not working. This should not be a problem as Octave can be launched with its user-interface and that remain the most popular/recommended way of using it.

Warning: For a fully operational *Jupyter notebook* environment, other the LaTeX support discussed in its dedicatet section, you also need both `pandoc` and `inkscape`.

Important: It is strongly recommended to install Quarto as a supporting tool for publishing reports from Julia and Python.

9.4.3 MSYS2

MSYS2 environment is useful for developing native Windows applications and programming in languages as C, C++, Fortran, and Rust. Kompanion VS Code configuration will automatically include this environment to its available terminal lists, what might fail if you do not install it. To get it working do the following:

- Go to MSYS2 page and save the installer to `downloads`.
- Run the installer and change installation path to point to a local `msys64` directory instead of the default `C:/msys64` (requiring admin rights).
- Once finished, launch an UCRT64 environment as proposed in the installation guide. Install at least the following:

```
pacman -S \
    mingw-w64-ucrt-x86_64-toolchain \
    mingw-w64-ucrt-x86_64-binutils \
    mingw-w64-ucrt-x86_64-gcc \
    mingw-w64-ucrt-x86_64-gcc-fortran
```

- Full package list is provided here.

9.5 Creating a portable launcher

A simple way to create a portable launcher requiring to source extra variables is by writing a simple batch script exporting or calling another script with the definitions:

```
@echo off

@REM Add variables to be sourced here such as
@REM set PATH="/path/to/some/dir";%PATH%
@REM ... or call another shared script doing so.
@REM call %~dp0\env

MyCode.exe
```

Because a batch script will keep a console window open, create a VB file with the following

```
Set oShell = CreateObject ("Wscript.Shell")
Dim strArgs
strArgs = "cmd /c MyCode.bat"
oShell.Run strArgs, 0, false
```

In the example we assume the program is called `MyCode.exe` and the batch script has been named in an analogous way `MyCode.bat`.

Part II

Elmer Multiphysics

Elmer is a multiphysics finite element method (FEM) solver mainly developed by CSC and maintained at GitHub. Several resources can be found in is official webpage and in the community portal and in the forum. There is also an YouTube channel with several tutorials and illustration of the package capabilities.

The goal of this page is not to supersede the documentation, but to make it (partially) available as a webpage where search and navigation become more intuitive. *Notice that this will be fed according to my personal projects and learning, so any contribution to accelerate the process is welcome.* Here you find a *user-guide-style* page with more details are provided below in the selected notes of Elmer documentation.

Quick answers

- Is Elmer the adequate tool for my projects? In this document you find a short introduction to what Elmer can do and the main executables.
- How do I start learning Elmer? Simply put, Elmer does not require basic users to master all the fundamentals of FEM, so following the *getting started* guide seems a good starting point. There you learn how to install, configure, and run the software.
- Where do I get the binaries of Elmer? If willing to run in Windows, the previous link provides the compiled binaries; there are also instructions for installing directly in Ubuntu as well as all the documentation and other test and sample cases.
- I feel alone, where do I find other users? The forum seems to be moderately active, so you can go there to chat with other users and developers if you are not in a hurry.

Limitations and issues

- Currently the GUI is not able to import SIF files generated manually because it stores its state in a XML file; to be able to re-run cases from the GUI users need to create the equivalent case (eventually using the free text fields) in the GUI itself before regenerating a SIF file. Notice that this will overwrite the SIF file, so keep in mind to backup the file in another directory; that is especially required for highly customized cases.
- When exporting meshes from `gmsh`, consider using the extension `.msh` and not `.msh2` as is often seen as a reminder of format 2 mesh; Elmer GUI is unable to render the mesh in this case. Notice that this has apparently no effect if running from command line.

Ongoing work

- Development of a VS Code syntax highlight extension with help of data provided in SOLVER.KEYWORDS.

Retrieving materials

Because there are plenty of interesting materials in Elmer public directory, it is worth downloading it all and selecting what to keep later. In a Linux terminal one could run the following command. If you also want to retrieve the animations, binaries, and virtual machines, consider removing and/or modifying the `-X` options.

```bash
#!/usr/bin/env bash

URL="https://www.nic.funet.fi/pub/sci/physics/elmer/"

wget -r -l 20 --no-parent           \
    -X /pub/sci/physics/elmer/anim/ \
    -X /pub/sci/physics/elmer/bin/  \
    -R "index.html*"                \
    ${URL}
```

10 Elmer Fundamentals

10.1 Preprocessing steps

- Other than the native `ElmerGrid`, several other software can be used for conceiving a geometry and mesh generation workflow. Users are encouraged to use external tools such as gmsh or Salome to generate computational meshes, built-in support in Elmer bein limited to very simple geometries only. For complex geometries FreeCAD is a standout alternative. For more consider reading *Geometry and Preprocessing*.
- Not all `gmsh` standard mesh formats are supported, but only version 2 (as for OpenFOAM); that said, users are encouraged to export mesh in UNV format to avoid compatibility issues, and also because both proposed tools support it. You can control the element orders in command line using option `-order <N>`; gmsh operates this way so that the same script can generate any supported element type.
- For importing meshes one uses `ElmerGrid <input-format> <output-format>`, where the format arguments are documented in the manual sections 1.3 and 1.4. The UNV input is given by number 8 while standard `ElmerSolver` output by number 2, so that the conversion command would start with `ElmerGrid 8 2 <other-arguments...>`. Always verify the number of nodes remain untouched after conversion - or if it changed when using option `-merge <dist>`, merging nodes that are closer than the user-defined `<dist>`.
- Remember to use `Coherence;` with OpenCASCADE factory in gmsh scripts to automatically strip internal faces; that might not be enough for complex cases, see below. **Note:** I tried doing so and unless `Physical Surfaces` naming the external boundaries are provided every interface is dumped and imported by Elmer; maybe I misunderstood the use of command!
- Because faces are not named in Elmer, *i.e.* no matter what `Physical Surface` names you provide in gmsh, even for the advanced user working from command line it might be interesting to use the interactive zone grouping capabilities of `ElmerGUI`. That might even become a *requirement* as geometric complexity grows.

Software	Notes
FreeCAD	Geometry only for now, probably the best in the list; good parametric modeling support. Has native Elmer integration.
Salome	Can export UNV (8) meshes readable by Elmer. An extension to call Elmer directly from Salome is under development.
gmsh	Can export MSH2 (14) and UNV (8) formats readable by Elmer.
netgen	Is able to write native Elmer linear meshes; can be used as a plug-in.
tetgen	Can be used as a plug-in.

10.2 Using Elmer

- Newcomers might be interested in `ElmerGUI`; although very intuitive, the interface is quite limited and for complex programs running from command line is the preferred mode.
- Most users will finally end setting up an workflow employing both to `ElmerSolver` (to run the simulations) and `ElmerGrid` (to prepare the grid and setup parallelization).
- There is also `ViewFactors` which might be useful in special cases involving radiation and other executables but they are not mentioned here because they fall in the legacy code family.
- Users must be aware that Elmer has no default unit system; one must take care that units are coherent across the different models and materials.
- Support to mathematical operations in SIF through MATC, which has its own syntax and documentation. It can be used, *e.g.* for computing temperature dependent properties, what can be helpful for simple expressions (instead of writing Fortran 90 code for extensions).

10.3 Parallel computing

- Before running in parallel a working case running in serial is required; using `ElmerGUI` this can be enabled in `Run > Parallel settings...`. Notice that after running postprocessing of `.pvtu` needs to be done in ParaView directly.
- To partition the mesh from command line one needs to run `ElmerGrid 2 2 <mesh-name> -partdual -metiskway <N>`, which will convert `<mesh-name>` from Elmer mesh format (2) into itself (thus the `2 2` in the command) and dump the resulting mesh in `partitions.<N>`, with `<N>` being the number of physical cores to run the simulation.
- Parallel cases can be run with `mpiexec -n <N> ElmerSolver_mpi`. Notice that under Linux the MPI runner is called `mpirun` instead of `mpiexec`.

10.4 Tips and ideas

- Use `Coordinate Mapping` to rotate meshes with oriented particles
- Scaling of a single direction can be done with `Coordinate Scaling`
- Time step can be changed with a list of elements in `Timestep Intervals`, *e.g.*

```
! Run 10 time-steps of 0.1 s, then 100 with 1.0 s.
Timestep Intervals(2) = 10 100
Timestep Sizes(2) = 0.1 1.0
```

- Take care with `Linear System Abort Not Converged = True` for physical consistency; generally continuing a simulation after a failed step is worthless unless one is pseudo-stepping towards a difficult (highly nonlinear) steady-state.

10.5 Material properties

Material properties can be specified as:

- Constant: just the default numeric input in SIF files
- Tabulated linearly or using a cubic spline, *e.g.*

```
! Linear interpolation
Viscosity = Variable Temperature
  Real
    298.15 1.0
    ! ... more data here
    373.15 2.0
End

! Cubic spline interpolation
Viscosity = Variable Temperature
  Real cubic
    298.15 1.0
    315.15 1.1
    345.15 1.5
    ! ... more data here
    373.15 2.0
End
```

- Arrays: for representing anisotropic bodies, *e.g.*

```
Heat Conductivity(3,3) = 1 0 0\
                         0 1 0\
                         0 0 2
```

- Using MATC as explained below in this page. Notice that sourcing files in MATC is the recommended way to get reusable code; coding MATC in SIF files requires to escape all lines and quickly becomes messy.
- User-defined functions (UDF) can also be provided in Fortran; notice that even when MATC can be used, this may lead to a speed-up of calculations with the inconvenient of needing more code. So for cases that are intended to be reused, it is important to consider writing proper extensions in Fortran. The following example illustrates a temperature dependent thermal conductivity function which is evaluated by Elmer at all nodes. In most cases a simple `USE DefUtils` is enough to get the required Elmer API to write the extension.

```
FUNCTION conductivity(model, n, time) RESULT(k)
    !*************************************************************
    ! Load Elmer library.
    !*************************************************************

    USE DefUtils
    IMPLICIT None

    !*************************************************************
```

```
    ! Function interface.
    !*************************************************************

    TYPE(Model_t) :: model
    INTEGER       :: n
    REAL(KIND=dp) :: time, k

    !*************************************************************
    ! Function internals.
    !*************************************************************

    TYPE(Variable_t), POINTER :: varptr
    REAL(KIND=dp) :: T
    INTEGER :: idx

    !*************************************************************
    ! Actual implementation
    !*************************************************************

    ! Retrieve pointer to the temperature variable.
    varptr => VariableGet(model%Variables, 'Temperature')

    ! Access index of current node.
    idx = varptr%Perm(n)

    ! Retrieve nodal temperature.
    T = varptr%Values(idx)

    ! Compute heat conductivity from NodalTemperature, k=k(T)
    k = 2.0 - T * (2.5e-03 - 1.0e-06 * T)
END FUNCTION conductivity
```

In order to compile the above assume it is written to `properties.f90` file; then one can call `elmerf90 properties.f90 -o properties` to generate the required shared library that is loaded in runtime by Elmer. Below we illustrate the use of `Procedure` to attach this library to a given material; first one provides the name of the shared library then the name of the function. A single library can in fact contain several functionalities.

```
Material 1
  Name = "Solid"
  Heat Conductivity = Variable Time
    Procedure "properties" "conductivity"
  Heat Capacity = 1000
  Density = 2500
End
```

10.6 Postprocessing

For postprocessing the recommended way is by using external tools as ParaView and PyVista, both handling well the VTK format of outputs. Nonetheless there are a some in-solver processing utilities

that are worth knowing, especially in what concerns extracting and filtering data from certain regions, creating new fields, and computing fluxes.

- Creating a new scalar

The keyword set of materials is actually not fixed; one can, for instance, create composition field in different units with MATC, as illustrated below (case here):

```
MoleFraction = Variable Concentration
  Real MATC "carbonmolefraction(tx)"
```

Then in solver `SaveMaterials`, this new name `MoleFraction` can be used as a variable:

```
Solver 1
  Equation = SaveMaterials
  Parameter 1 = Concentration Diffusivity
  Parameter 2 = MoleFraction
  Procedure = "SaveData" "SaveMaterials"
  Exec Solver = After Timestep
End
```

Another situation that can be frequently found is unit conversion for temperature. It was chosen to implement it in this case because it is multi-material; that is a reminder that in such cases the new variable needs to be created for all materials (as this is a tweak, since the temperature is not a material property, but a global field). If forgotten in one material, an error will show up in ParaView telling you that the field is not available in some regions.

10.7 User-defined functions

Upcoming

10.7.1 Derived fields

- StreamSolver
- FluxSolver

#elmer/models/flux-solver

- VorticitySolver
- DivergenceSolver
- ScalarPotentialSolver
- ArtificialCompressibility
- FluidicForce
- ElectricForce
- RigidBodyReduction

- FilterTimeSeries
- DataToFieldSolver
- ProjectToPlane
- StructureProjectToPlane
- FindOptimum

10.7.2 Saving data modules

Note: coupling to OpenFOAM is discussed separately in another section.

- SaveData
- SaveScalars

#elmer/models/save-scalars

- SaveLine

#elmer/models/save-line

- SaveMaterials
- SaveBoundaryValues
- ResultOutputSolve
- ResultOutputSolver

#elmer/models/result-output-solver

- SaveGridData
- SaveGridData
- Isosurface
- IsosurfaceSolver

10.7.3 OpenFOAM coupling

...

11 Solver Input Files (SIF)

Once you get serious with Elmer it is a natural evolution to prefer to work with *SIF* files instead of the GUI most of the time; the GUI remains useful for raw case generation, avoiding writing boilerplate setups and to get default values right. This is also true in a majority of scientific computing software. The documentation of *SIF* is spread over the whole documentation of Elmer and this page tries to summarize it. For the beginner, this video is a good starting point, this page being more of a reference manual.

Because syntax highlighting is important for productivity, I am continuously developing a minimalistic extension for VS Code. Its partial development can be found here. You can clone that repository and copy the `sif/` directory under `%USERPROFILE%/.vscode/extensions` or in the equivalent directory documented here; later it will probably be packaged as a standard VS Code extension.

11.1 Sections

- `Header`
- `Simulation`
- `Constants`
- `Material <n>`
- `Body <n>`
- `Solver <n>`
- `Equation <n>`
- `Initial Condition <n>`
- `Boundary Condition <n>`

12 MATC

Elmer provides a few extension methods. For complex models you might be prompted to use directly Fortran 90. For simpler things, such as providing temperature dependent thermophysical properties, it has its own parser for use in SIF, the metalanguage MATC. Expressions provided in MATC can be evaluated when file is read or during simulation execution.

Because it is quite concise, I summarized the whole of MATC syntax and functions in this page. For the official documentation please refer to this document.

12.1 Declaring variables

Variables in MATC can be matrices and strings; nonetheless, both are stored as double precision arrays so creating large arrays of strings can represent a waste of memory. For some weird reason I could not yet figure out, MATC language can be quite counterintuitive.

Let's start by creating a variable, say `x` below; attribution, such a in Python, creates the variable. It was stated that everything is a matrix or string; since `x` is not a string, it is actually a 1×1 matrix and in this case we can omit the indexing when allocating its memory.

```
x = 1
```

So far nothing weird; when declaring a string as `k` below we are actually creating a 1×5 row matrix, what is not unusual in many programming languages (except for being double precision here). Furthermore, we use the typical double-quotes notation for strings.

```
k = "hello"
```

Matrix indexing is zero-based, as in C or Python. Matrix slicing is done as in many scripting languages, such as Python, Julia, Octave, ..., and we can reverse the order of the first *six* elements of an array `y` as follows:

```
y(0, 0:5) = y(0, 5:0)
```

Notice above the fact that matrix slicing is *last-inclusive*, as in Julia, meaning that all elements from index zero to five *inclusive* are included in the slice. Weirdness starts when you do something like

```
z(0:9) = 142857
```

which according to the documentation will produce an array `1 4 2 8 5 7 1 4 2 8`; I could not verify this behavior yet. Additionally, the following produces another unexpected result:

```
z(9, 9) = 1
```

If matrix `z` does not exist, this results in a 10×10 matrix with all zeros except the explicitly declared element. The size of variables are dynamic, so in the above if `z` already existed but wa smaller, it would be padded with zeros instead.

Finally, logical indexing is also allowed:

```
x(x < 0.05) = 0.05
```

12.2 Control structures

MATC provides conditionals and loops as control structures. Below we have the `if-else` statement, which can be declared inline of using a C-style declaration using braces.

```
if ( expr ) expr; else expr;

if ( expr )
{
    expr;
    ...
    expr;
} else {
    expr;
    ...
    expr;
}
```

Loops can be declared using both `for` or `while`. There is no mechanism of generating values within a `for` loop and one must provide a vector of indexes for repetition. *In the official documentation there is no reference to a `continue` or `break` statement and I could not verify their existence yet.*

```
for( i=vector ) expr;

for( i=vector )
{
    expr;
    ...
    expr;
}

while( expr ) expr;

while( expr )
{
    expr;
    ...
    expr;
}
```

12.3 Operators

Assume the following definitions:

- `a`, `b` and `c` ordinary matrices
- `l`, `t` and `r` logical matrices
- `s`, `n` and `m` scalars

`b = a'`	is transpose of matrix a.
`b = @a`	evaluate content of a string variable a as a MATC statement.
`t = ~l`	elementwise logical not of if x is not zero.
`b = a ^ s`	if a is a square matrix and s is integral, a matrix power is computed, otherwise an elementwise power.
`c = a * b`	if a and b are compatible for matrix product, that is computed, otherwise if they are of the same size or at least one of them is scalar, an elementwise product is computed.
`c = a # b`	elementwise multiplication of a and b.
`c = a / b`	is fraction of a and b computed elementwise.
`c = a + b`	is sum of matrices a and b computed elementwise.
`c = a - b`	is difference of matrices a and b computed elementwise.
`l = a == b`	equality of matrices a and b elementwise.
`l = a <> b`	inequality of matrices a and b elementwise.
`l = a < b`	true if a is less than b computed elementwise.
`l = a > b`	true if a is greater than b computed elementwise.
`l = a <= b`	true if a is less than or equal to b computed elementwise.
`l = a >= b`	true if a is greater than or equal to b computed elementwise.
`a = n : m`	return a vector of values starting from n and ending to m by increment of (plus-minus) one.
`r = l & t`	elementwise logical and of a and b.
`l = a \\| b`	elementwise logical or of a and b.
`c = a ? b`	reduction: set values of a where b is zero to zero.
`b = n m % a`	resize a to matrix of size n by m.
`b = a`	assigning a to b.

12.4 Function definitions

The syntax of the function definition is similar to that of Julia and is given below. The function body is enclosed by braces. Instead of using a `return` statement, the resulting value is attributed to a variable named after the function with a leading underscore. Notice the `!` denoting comments in the description of the function.

```
function name(arg1, arg2, ...)
!
! Optional function description (seen with help("name"))
!
import var1, var2
export var3, var4
{
    expr;
     ...
    expr;

    _name = value
}
```

Functions have their own list of variables. Global variables are not seen in this function unless imported by `import` or given as arguments. Local variables can be made global by the `export` statement.

Functions, if returning matrices, behave in many ways as variables do. So if you have defined function `mult` as follows

```
function mult(a, b)
{
   _mult = a * b;
}
```

You can get element (3,5) of the `a` times `b` matrix with `mult(x,y)[3,5]` or the diagonal values of the same matrix by `diag(mult(x, y))`.

12.5 Built-in functions

- C-style math

The following listing provides a series of mathematical functions which follow a their meaning in C. The only exceptions are `ln` denoting the natural logarithm and `log` used here for base 10 logarithms.

```
r = sin(x)

r = cos(x)

r = tan(x)

r = asin(x)

r = acos(x)

r = atan(x)

r = sinh(x)

r = cosh(x)

r = tanh(x)

r = exp(x)

r = ln(x)

r = log(x)

r = sqrt(x)

r = ceil(x)

r = floor(x)

r = abs(x)

r = pow(x,y)
```

- General utilities

`funcdel(name)`	Delete given function definition from parser.
`funclist(name)`	Give header of the function given by name.
`env(name)`	Get value of environment variable of the operating system.
`exists(name)`	Return true (non-zero) if variable by given name exists otherwise return false (=0).
`source(name)`	Execute commands from file given name.
`format(precision)`	Set number of digits used in printing values in MATC.
`r = eval(str)`	Evaluate content of string str. Another form of this command is `@str`.
`who`	Give list of currently defined variables.
`help` or `help("symbol")`	First form of the command gives list of available commands. Second form gives help on specific routine.

- String and I/O functions

`str = sprintf(fmt[,vec])`	Return a string formatted using `fmt` and values from `vec`. A call to corresponding C-language function is made.
`vec = sscanf(str,fmt)`	Return values from `str` using format `fmt`. A call to corresponding C-language function is made.
`str = fread(fp,n)`	Read `n` bytes from file given by `fp`. File pointer fp should have been obtained from a call to `fopen` or `freopen`, or be the standard input file stdin. Data is returned as function value.
`vec = fscanf(fp,fmt)`	Read file `fp` as given in format. Format fmt is equal to C-language format. File pointer fp should have been obtained from a call to `fopen` or `freopen`, or be the standard input.
`str = fgets(fp)`	Read next line from fp. File pointer fp should have been obtained from a call to fopen or freopen or be the standard input.
`n = fwrite(fp,buf,n)`	Write n bytes from buf to file fp. File pointer fp should have been obtained from a call to fopen or freopen or be the standard output (stdout) or standard error (stderr). Return value is number of bytes actually written. Note that one matrix element reserves 8 bytes of space.
`n = fprintf(fp,fmt[,vec])`	Write formatted string to file fp. File pointer fp should have been obtained from a call to fopen or freopen or be the standard output (stdout) or standard error (stderr). The format fmt is equal to C-language format.
`fputs(fp,str)`	Write string str to file fp. File pointer fp should have been obtained from a call to fopen or freopen or be the standard input (stdin).
`fp = fopen(name,mode)`	Reopen file given previous file pointer, name and access mode. The most usual modes are `"r"` for reading and `"w"` for writing. Return value fp is used in functions reading and writing the file.
`fp = freopen(fp,name,mode)`	Reopen file given previous file pointer, name and access mode. The most usual modes are `"r"` for reading and `"w"` for writing. Return value fp is used in functions reading and writing the file.
`fclose(fp)`	Close file previously opened with fopen or freopen.
`save(name,a[,ascii_flag])`	Close file previously opened with `fopen` or `freopen`.
`r = load(name)`	Load matrix from a file given name and in format used by `save` command.

- Numerical utilities

`r = min(matrix)`	Return value is a vector containing smallest element in columns of given matrix. r = min(min(matrix)) gives smallest element of the matrix.
`r = max(matrix)`	Return value is a vector containing largest element in columns of given matrix. `r = max(max(matrix))` gives largest element of the matrix.
`r = sum(matrix)`	Return vector is column sums of given matrix. `r = sum(sum(matrix))` gives the total sum of elements of the matrix.
`r = zeros(n,m)`	Return n by m matrix with elements initialized to zero.
`r = ones(n,m)`	Return n by m matrix with elements initialized to one.
`r = rand(n,m)`	Return n by m matrix with elements initialized with random numbers from zero to one.
`r = diag(a)`	Given matrix return diagonal entries as a vector. Given vector return matrix with diagonal elements from vector. `r = diag(diag(a))` gives matrix with diagonal elements from matrix a, otherwise elements are zero.
`r = vector(start,end,inc)`	Return vector of values going from start to end by inc.
`r = size(matrix)`	Return size of given matrix.
`r = resize(matrix,n,m)`	Make a matrix to look as a `n` by `m` matrix. This is the same as `r = n m % matrix`.
`r = where(a)`	Return a row vector giving linear index to a where a is not zero.
`r = matcvt(matrix, type)`	Makes a type conversion from MATC matrix double precision array to given type, which can be one of the following: `"int"`, `"char"` or `"float"`.
`r = cvtmat(special, type)`	Makes a type conversion from given type to MATC matrix. Type can be one of the following: `"int"`, `"char"` or `"float"`.

- Linear algebra

`r = trace(matrix)`	Return value is the sum of matrix diagonal elements.
`r = det(matrix)`	Return value is determinant of given square matrix.
`r = inv(matrix)`	Invert given square matrix. Computed also by operator $^{-1}$
`r = tril(x)`	Return the lower triangle of the matrix x.
`r = triu(x)`	Return the upper triangle of the matrix x.
`r = eig(matrix)`	Return eigenvalues of given square matrix. The expression `r(n,0)` is real part of the n:th eigenvalue, `r(n,1)` is the imaginary part respectively.
`r = jacob(a,b,eps)`	Solve symmetric positive definite eigenvalue problem by Jacob iteration. Return values are the eigenvalues. Also a variable eigv is created containing eigenvectors.
`r = lud(matrix)`	Return value is LUD decomposition of given matrix.
`r = hesse(matrix)`	Return the upper hessenberg form of given matrix.
`r = eye(n)`	Return n by n identity matrix.

12.6 Usage

Although the language is pretty fast for most simple uses, it is much slower than Fortran extensions, so use with case when scalability is needed. Another point is overuse of MATC; do not use it with simple numerical expressions, *e.g.* `OneThird = Real $1.0/3.0` is much faster than its MATC counterpart `OneThird = Real MATC "1.0/3.0"`.

- Direct expression coding

One thing that in my opinion lacks in the documentation are examples of use in conjunction with SIF. For instance, for setting the thermal conductivity of a material as temperature-dependent one could use the following snippet and modify the string to match the desired expression. An example of its usage is provided in this case.

```
Heat Conductivity = Variable Temperature
  Real MATC "1.0 - tx * (2.5E-03 - 1.2E-06 * tx)"
```

- Sourcing functions from user modules

Models can become too complex to code in a single line. Hopefully MATC provides functions which can be declared in external modules. I avoid coding MATC directly in SIF because their syntax is different and that can quickly lead to unmaintainable code. An example of such external sourcing is provided in this case. You need to remember to call `source("module")` in `Simulation` section of SIF so that the functions can be used elsewhere. The call of a function become something as

```
Concentration Diffusivity = Variable Concentration
  Real MATC "diffusivity(tx)"
```

if independent variable is concentration or for time

```
Mass Transfer Coefficient = Variable Time
  Real MATC "masstransfercoef(tx)"
```

You can even use multiple variables, *e.g.*

```
Mass Transfer Coefficient = Variable Time, Temperature
  Real MATC "masstransfercoef(tx(0), tx(1))"
```

PS: *I managed to use a single `source` in SIF, although the documentation does not state that many sources are forbidden; for some reason multiple sources work when sourcing from a file.*

For more complex cases such as this one it is worth writing actual MATC function modules; since there is no syntax highlighter available for MATC in VS Code, the `.ini` extension seems to provide better readability to the code. The problem was split in two parts: the models which take care of sourcing the conditions, so that basic users could only edit the latter and run their variant calculations with no coding skills. Notice that the symbols that are used in SIF are exported from this line instead of being set as global variables.

Part III

Ansys Fluent

License usage

```
ansys_root=/path/to/ansys/installation
ansys_version=version-number

export LMUTIL="${ansys_root}$/ansys_inc/${ansys_version}/licensingclient/linx64/lmutil"

"${LMUTIL}" lmstat -c 1055@<ip> -a | grep "Users of" | \
    awk -F"[ ]" "{printf "%-25s %3s /%3s\n", $3, $13, $7}"
```

13 Text user interface (TUI)

An extremelly opinionated introduction.

Using Ansys Fluent can be a huge nightmare; the software is simply *bad*. There's no way someone should commercialize such a garbage of user experience, shame on you! It happens that as CFD engineers we often need to use it and in some cases it surprisingly works as its solver is not as bad as the graphical interface. One just needs to refrain themselves from using the graphical interface and go straight to the text user interface, hereafter our friend TUI.

If you are beginning in the CFD world, stop here and go learn some good code, as OpenFOAM or SU2; continue only if you really need Fluent because of your company or job opportunities require it. If you started learing with the GUI, I have the good-bad news that you lost your time and to be productive with this garbage tool you really need to use the TUI.

For what follows let's keep in mind the following:

- When aiming at creating a new model, it is worth entering manually commands in the TUI terminal of Fluent running in graphical mode; one just need to keep in mind to store those commands in a plain text *journal* file for later automating the workflow.
- Commands issued in the TUI are some sort of Scheme dialect; it is possible to code functions directly in journals for pre-processing case parameters. Unfortunately, Fluent's manuals list available TUI commands, but doesn't expose the full Scheme interpreter internals.
- Knowing that, if you save your journals with `.scm` extension instead of the traditional `.jou` most text editors will provide you with syntax highlighting.
- For learing Scheme one find good resources in Racket; just keep in mind that Fluent's Scheme is not Racket (it is some Ansys proprietary implementation), so just the main ideas will apply.
- If you check the project tree in Fluent, you will notice that there are three main sections, *Setup*, *Solution*, and *Results*; when I create a case I split journals in blocks corresponding to these sections and add a *Parameters* section beforehand. Most of my `model.scm` journal files will look like:

```
; Here goes plain Scheme code and parameter definitions:
/file/read-journal "0-params.scm"   ()

; One file per Fluent project section:
/file/read-journal "1-setup.scm"    ()
/file/read-journal "2-solution.scm" ()
/file/read-journal "3-results.scm"  ()

; Do not forget this if working in a shared cluster/HPC:
/exit ()
```

- It's worth mentioning that the above approach drastically reduces the use of user-defined expressions, as values that are computed in `0-params.scm` can replace initialization UDE's.

- Forget about Fluent built-in post-processing; it is too cumbersome and limited in the TUI and will require you to run in graphical mode for some commands to be available. Instead, export results as CGNS or other format and automate the workflow with PyVista.

13.1 Parameters

In the aforementioned `0-params.scm` file I often start by declaring a few constants that are often useful in case setup. The following snippet illustrates the basics of what you need to know about Scheme to do something useful in Fluent: commands are declared within parenthesis, as Scheme is of LISP's family, and parameter definitions follow a `(define <symbol> <value>)` syntax.

```
; Pi
(define pi 3.141592653589793)

; Gravity magnitude [m/s^2]
(define gravity-acceleration 9.81)

; Atmospheric pressure [Pa]
(define pressure-atm 101325.0)

; Universal gas constant [J/(mol*K)]
(define gas-constant 8.31446261815324)
```

One thing about Scheme that might be not familiar for people coming from other languages (or too young to have used an HP graphical calculator) is Polish notation for numerical operations: you preprend the operator to the operands. That has the advantage that `(+ 1 2 3)` is valid code and will add up all numbers.

```
; Compute some angle [deg] -> [rad]
(define theta (/ (* pi 15.0) 180.0))

; Gravity components [m/s^2]
(define gx 0.0)
(define gy (* -1 gravity-acceleration (cos theta)))
(define gz (* -1 gravity-acceleration (sin theta)))
```

Side-note: you might be asking yourself why the hell I project gravity acceleration components over the different axes. In some cases it is very useful if a geometry has corners or something that resembles a rectangle to keep it horizontal and apply body forces as components. For instance, when conceiving a geometry with Gmsh it will avoid a lot of trigonometry and round-off errors if you draw the body oriented as the screen. For some post-processing is is also useful to have horizontal bodies.

13.2 Troubleshooting

13.2.1 Terminal lacking interactivity

You can use `rlwrap` tool that adds readline functionality to any program:

```bash
# Install rlwrap (if not already installed)
sudo yum install rlwrap  # RHEL/CentOS
sudo apt install rlwrap  # Debian/Ubuntu

# Run Fluent with readline support
rlwrap fluent 3d -g

# Or create an alias
alias fluent-rl='rlwrap fluent'
```

14 User profile

The real power of TUI is the ability to introduce user-defined commands from a configuration file. This example setup showcases what kind of functionalities can be achieved. The following chapters may need elements exported here.

To get going, save this file to a case root or to your user home directory as `.fluent`; the next time you launch fluent, check the transcript for something as `Loading "/home/<user>/.fluent"`. You can define new functions here that best suit your typical CFD workflow.

```
; ------------------------------------------------------------------------------
; Utilities
; ------------------------------------------------------------------------------

; Configure number of blank lines for screen clear
(define m-nblanks 50)

; Clear screen (by printing a bunch of new lines)
(define (cls)
  (do ((i 0 (+ i 1)))
      ((= i m-nblanks))
    (newline))
  (display "...\n"))

; Alias of Linux list command
(define (ls)
  (system "ls -alt"))

; ------------------------------------------------------------------------------
; Manipulate zone info
; ------------------------------------------------------------------------------

; Get all zone info (list of zone-info)
(define (m-all-zones)
    (get-zones-info))

; Select zone name from zone info (1)
(define (m-get-zone-name zone-info)
  (symbol->string (car zone-info)))

; Select zone id from zone info (2)
(define (m-get-zone-id zone-info)
  (cadr zone-info))

; Select zone type from zone info (3)
(define (m-get-zone-type zone-info)
```

```
  (symbol->string (caddr zone-info)))

; Select cell flag from zone info (4)
(define (m-is-cell-zone? zone-info)
  (cadddr zone-info))

; ---------------------------------------------------------------------------
; DPM zone selection
; ---------------------------------------------------------------------------

; Zone types to ignore in DPM reports
(define M-DPM-IGNORE-ZONE-TYPES
  (list "fluid" "symmetry" "periodic" "interior"))

; Select zones for DPM reporting (not in ignore list and not cell zones)
(define (m-filter-dpm-zones zone-list)
  (filter
    (lambda (zone-info)
      (and (not (member (m-get-zone-type zone-info) M-DPM-IGNORE-ZONE-TYPES))
           (not (m-is-cell-zone? zone-info))))
    zone-list))

; Get names of DPM zones for reporting
(define (m-dpm-zones-names)
  (map m-get-zone-name (m-filter-dpm-zones (m-all-zones))))

; ---------------------------------------------------------------------------
; Solution
; ---------------------------------------------------------------------------

; Alias for programatically iterating flow
(define (m-solve-iterate n-iterations)
  (ti-menu-load-string (format #f "/solve/iterate ~a" n-iterations)))

; Alias for programatically updating DPM
(define (m-solve-dpm-update)
  (ti-menu-load-string "/solve/dpm-update"))

; ---------------------------------------------------------------------------
; EOF
; ---------------------------------------------------------------------------
```

15 Setup from TUI

15.1 Configuration

```
/file/set-batch-options
    no ; Do you want to confirm overwrite? [yes]
    no ; Do you want to exit on error? [no]
    no ; Do you want to hide questions? [no]
```

15.2 Loading and checking a mesh

```
; This is a comment (starting by a semi-colon).
/file/read-case "domain/tunnel.msh.h5" ()
/mesh/check
/mesh/mesh-info 0

; Write the case (or overwrite if existing)
/file/write-case "tunnel-model.cas.h5" ()
```

15.3 Merging zones

```
/mesh/modify-zones/merge-zones
    block
    block:2
    block:3
    ()
```

15.4 Handling named expressions

These are really bad examples; as stated in the beginning, for constant values it is better to use plain Scheme definitions. *TODO* add actual useful examples here:

```
/define/named-expressions/add "diam"  definition "0.01 [m]"       parameter yes ()
/define/named-expressions/add "dens"  definition "1.29 [kg/m^3]"  parameter no  ()
/define/named-expressions/add "visc"  definition "1.84 [Pa*s]"    parameter no  ()
/define/named-expressions/add "theta" definition "25 [deg]"       parameter no  ()
/define/named-expressions/export-to-tsv "expressions.tsv" ()
```

15.5 Defining operating conditions

```
/define/operating-conditions/gravity yes
    "0 [m/s^2]"
    "-cos(theta) * 9.81 [m/s^2]"
    "-sin(theta) * 9.81 [m/s^2]"
```

15.6 Setting up some models

```
; Select a viscous model
/define/models/viscous/ke-realizable yes
/define/models/viscous/kw-sst yes

; Activate DPM calculations
/define/models/dpm/interaction/coupled-calculations yes
/define/models/dpm/interaction/dpm-iteration-interval 10
/define/models/dpm/options/two-way-coupling yes
```

15.7 Modify materials

```
(define gas-rho 1.29)

/define/materials/change-create "air" "air"
    yes                   ; change Density? [no]
    constant gas-rho      ; Density
    no                    ; change Cp (Specific Heat)? [no]
    no                    ; change Thermal Conductivity? [no]
    yes                   ; change Viscosity? [no]
    expression "gas_mu"   ; Viscosity
    no                    ; change Molecular Weight? [no]
    no                    ; change Thermal Expansion Coefficient? [no]
    no                    ; change Speed of Sound? [no]
```

15.8 Setting boundary conditions

```
/define/boundary-conditions/set/mass-flow-inlet
    "inlet"  () mass-flow no 1.0  ()

/define/boundary-conditions/set/mass-flow-inlet
    "inlet"  () mass-flow no "mdot_inlet"  ()

/define/boundary-conditions/set/wall "gluish-wall" ()
    dpm-bc-type yes trap ()

/define/boundary-conditions/modify-zones/zone-type
    "environment-entry" wall ()
```

15.9 Creating a DPM injection

```
(define xc 0.0)
(define yc 1.0)
(define zc 0.2)

(define fiber-mdot (/ 200 3600))
(define fiber-diam 0.0005)
(define fiber-streams 50)
```

- Using group:

```
/define/models/dpm/injections/create-injection/fibers
    no            ; Particle type [inert]: Change current value? [no]
    yes           ; Injection type [single]: Change current value? [no]
    group         ; Injection type [single]>
    no            ; Injection Material [anthracite]: Change current value? [no]
    fiber-streams ; Number of Streams [2]
    yes           ; Stochastic Tracking? [no]
    yes           ; Random Eddy Lifetime? [no]
    1             ; Number of Tries [1]
    0.15          ; Time Scale Constant [0.15]
    no            ; Modify Laws? [no]
    no            ; Set user defined initialization function? [no]
    no            ; Cloud Tracking? [no]
    no            ; Rosin Rammler diameter distribution? [no]
    -0.01         ; First Point X-Position (in [m]) [0]
    0.01          ; Last Point X-Position (in [m]) [0]
    yc            ; First Point Y-Position (in [m]) [0]
    yc            ; Last Point Y-Position (in [m]) [0]
    zc            ; First Point Z-Position (in [m]) [0]
    zc            ; Last Point Z-Position (in [m]) [0]
    0             ; First Point X-Velocity (in [m/s]) [0]
    0             ; Last Point X-Velocity (in [m/s]) [0]
    0             ; First Point Y-Velocity (in [m/s]) [0]
    0             ; Last Point Y-Velocity (in [m/s]) [0]
    0             ; First Point Z-Velocity (in [m/s]) [0]
    0             ; Last Point Z-Velocity (in [m/s]) [0]
    fiber-diam    ; First Point Diameter (in [m]) [0.0001]
    fiber-diam    ; Last Point Diameter (in [m]) [0.0001]
    fiber-mdot    ; First Point Flow Rate (in [kg/s]) [9.999999999999999e-21]
    fiber-mdot    ; Last Point Flow Rate (in [kg/s]) [9.999999999999999e-21]
```

- Using solid-cone:

```
/define/models/dpm/injections/create-injection/fibers
    no            ; Particle type [inert]: Change current value? [no]
    yes           ; Injection type [single]: Change current value? [no]
    solid-cone    ; Injection type [single]>
    no            ; Injection Material [anthracite]: Change current value? [no]
    fiber-streams ; Number of Streams [2]
    yes           ; Stochastic Tracking? [no]
```

```
    yes            ; Random Eddy Lifetime? [no]
    1              ; Number of Tries [1]
    0.15           ; Time Scale Constant [0.15]
    no             ; Modify Laws? [no]
    no             ; Set user defined initialization function? [no]
    no             ; Cloud Tracking? [no]
    no             ; Rosin Rammler diameter distribution? [no]
    xc             ; X-Position (in [m]) [0]
    yc             ; Y-Position (in [m]) [0]
    zc             ; Z-Position (in [m]) [0]
    fiber-diam     ; Diameter (in [m]) [0.0001] 454.28e-06
    0              ; Azimuthal Start Angle (in [deg]) [0]
    360            ; Azimuthal Stop Angle (in [deg]) [360.0000005729578]
    0              ; X-Axis [0]
    -1             ; Y-Axis [0]
    0              ; Z-Axis [1]
    0              ; Velocity Magnitude (in [m/s]) [0]
    2              ; Cone Half Angle (in [deg]) [0]
    0.01           ; Outer Radius (in [m]) [0]
    fiber-mdot     ; Total Flow Rate (in [kg/s]) [9.999999999999999e-21]
```

- Using surface:

```
/define/models/dpm/injections/create-injection/fibers
    no           ; Particle type [inert]: Change current value? [no]
    yes          ; Injection type [single]: Change current value? [no]
    surface      ; Injection type [single]>
    no           ; Injection Material [anthracite]: Change current value? [no]
    injection () ; Surface(1) [()] injection ()
    no           ; Scale Flow Rate by Face Area [no]
    yes          ; Use Face Normal for Velocity Components [no]
    yes          ; Stochastic Tracking? [no]
    yes          ; Random Eddy Lifetime? [no]
    1            ; Number of Tries [1]
    0.15         ; Time Scale Constant [0.15]
    no           ; Modify Laws? [no]
    no           ; Set user defined initialization function? [no]
    no           ; Cloud Tracking? [no]
    no           ; Rosin Rammler diameter distribution? [no]
    fibers-diam  ; Diameter (in [m]) [0.0001]
    0            ; Velocity Magnitude (in [m/s]) [0]
    fibers-mdot  ; Total Flow Rate (in [kg/s]) [9.999999999999999e-21]

/define/materials/change-create/anthracite "fibers" yes constant 2800 no yes

/define/models/dpm/injections/properties/set/pick-injections-to-set
    no     ; List all available injections before picking one or more of them?
    fibers ; pick injections:(1) [()]
    ()     ; pick injections:(2) [()]
    no     ; Review list of picked injections?
```

```
/define/models/dpm/injections/injection-properties/set/physical-models
    drag-parameters nonspherical 0.1
```

15.10 Saving the case

```
/file/write-case-data "model-setup.cas.h5" yes
```

16 Solution from TUI

16.1 Report definitions

```
/solve/report-definitions/delete-all yes

/solve/report-definitions/add/mass-flows surface-massflowrate
    surface-names "injection-1" "injection-2" "extractions" ()
    per-surface yes ()

/solve/report-files/add/report-mass-flows
    file-name   "report-mass-flows-rfile.out"
    report-defs "mass-flows" () ()

/solve/report-files/edit/report-mass-flows
    frequency 10 ()

/solve/report-definitions/add/mass-extraction injection-escaped-mass
    boundary-zones "extraction" () ()
```

16.2 Residuals

```
/solve/monitors/residual/convergence-criteria
    1e-9 ; continuity residual convergence criterion [0.001]
    1e-6 ; x-velocity residual convergence criterion [0.001]
    1e-6 ; y-velocity residual convergence criterion [0.001]
    1e-6 ; z-velocity residual convergence criterion [0.001]
    1e-5 ; k residual convergence criterion [0.001]
    1e-5 ; omega residual convergence criterion [0.001]
```

16.3 Solver parameters

```
/solve/set/pseudo-time-method global yes 1 1.0 ()

/solve/set/p-v-controls 0.7 0.3
/solve/set/pseudo-time-method/relaxation-factors/density 0.5

/solve/set/discretization-scheme/k        0 ()
/solve/set/discretization-scheme/omega    0 ()
```

16.4 Initialize flow and iterate

```
/solve/initialize/initialize-flow
/solve/initialize/hyb-initialization yes
/solve/iterate 100
```

16.5 Parallel settings

```
; /parallel/check
; /parallel/partition/auto
; /parallel/timer/usage
```

17 Results from TUI

17.1 Figures in graphical mode only

You are probably loosing your time here; maybe skip straight to the next section and export results for post-processing with PyVista.

```
/display/display-states/create state-right
    "disable"      ; Front face transparent ("enable" "disable" "dont-save")
    "orthographic" ; Projection ("perspective" "orthographic" "dont-save")
    "enable"       ; Axes ("enable" "disable" "dont-save")
    "disable"      ; Ruler ("enable" "disable" "dont-save")
    "disable"      ; Title ("enable" "disable" "dont-save")
    "enable"       ; Boundary markers ("enable" "disable" "dont-save")
    "enable"       ; Anti-aliasing ("enable" "disable" "dont-save")
    "disable"      ; Reflections ("enable" "disable" "dont-save")
    "disable"      ; Static shadows ("enable" "disable" "dont-save"
    "disable"      ; Dynamic shadows  ("enable" "disable" "dont-save")
    "disable"      ; Grid plane ("enable" "disable" "dont-save")
    "on"           ; Headlights ("on" "off" "automatic" "dont-save")
    "automatic"    ; Lighting ("automatic" "flat" "gouraud" "phong" "dont-save" "off")
    "right"        ; View name ("active" "dont-save" "wf-view" "front" "back"
                   ;            "right" "left" "top" "bottom" "isometric" "view-0")
    "0"            ; (dont-save 0) ["0"]

; Produces an .stt file (do not add exension in the command!).
/display/display-states/write "journaling/display-states" state-right ()

/display/objects/create/mesh wire
    options
        edges? yes
        faces? no ()
    edge-type outline
    coloring uniform edges black ()
    display-state-name state-right ()

/display/objects/create/contour contour-total-pressure
    surfaces-list "plane-name" ()
    field total-pressure
    range-option autorange-off
        minimum -200
        maximum  200
        clip-to-range? no ()
    color-map
        title-elements "Variable Only"
        color          "field-pressure"
```

```
        position       0
        format         "%.0f" ()
    display-state-name state-right ()

/display/objects/create/scene scene-total-pressure
    graphics-objects
        add  "wire" ()
        add "contour-total-pressure" () ()
    display-state-name state-right ()
```

17.2 Exporting for external processing

Important: the creation of the following plane-surface is part of the solution step 16, but is included here for convenience.

```
/surface/plane-surface plane-symmetry
    yz-plane ; (yz-plane zx-plane xy-plane point-and-normal three-points)
    0.01     ; x (in [m])

/file/export/ascii "results.csv"
    plane-symmetry ()  ; Surfaces ()
    yes                ; Delimiter/Comma?
    total-pressure
    dynamic-pressure
    velocity-magnitude ()
    no

/file/export/cgns "results"
    surface-select     ; Scope: (full-domain volume-select surface-select)
    plane-symmetry ()  ; Surfaces ()
    no                 ; Cell-Centered [no]
    yes                ; Format/HDF5?, if HDF5 not chosen, ADF format is set [yes]
    total-pressure     ; CGNS scalar(1)> ...
    dynamic-pressure
    viscosity-ratio
    velocity-magnitude ()
    () ()

; XXX: `history` seems to be broken!
; XXX: options change depending on the export type!
/file/export/particle-history-data
    fieldview             ; cfdpost, fieldview, history, ensight, geometry
    "results-particles"   ; File name
    "fibers" ()           ; Injections ()
    particle-time-step () ; Variables ()
    20                    ; Skip tracks
    2000                  ; Coarsen path by
```

18 Getting started with PyFluent

- Intro to PyFluent
- PyFluent documentation

Install sequentially the following packages as described here:

```
pip install ansys-fluent-core
pip install ansys-fluent-parametric
pip install ansys-fluent-visualization
```

Note: `ansys-fluent-parametric` requires an outdated version of NumPy (<2.0), so you probably need to consider working in a virtual environment.

18.1 Journaling

```
(api-start-python-journal "journal.py")
```

18.2 First steps

```
import ansys.fluent.core as pyfluent

config = dict(
    ui_mode   = "gui",
    precision = "double",
    dimension = 3,
    processor_count = 1,
)
session = pyfluent.launch_fluent(mode = "solver", **config)

session.exit()
```

Part IV

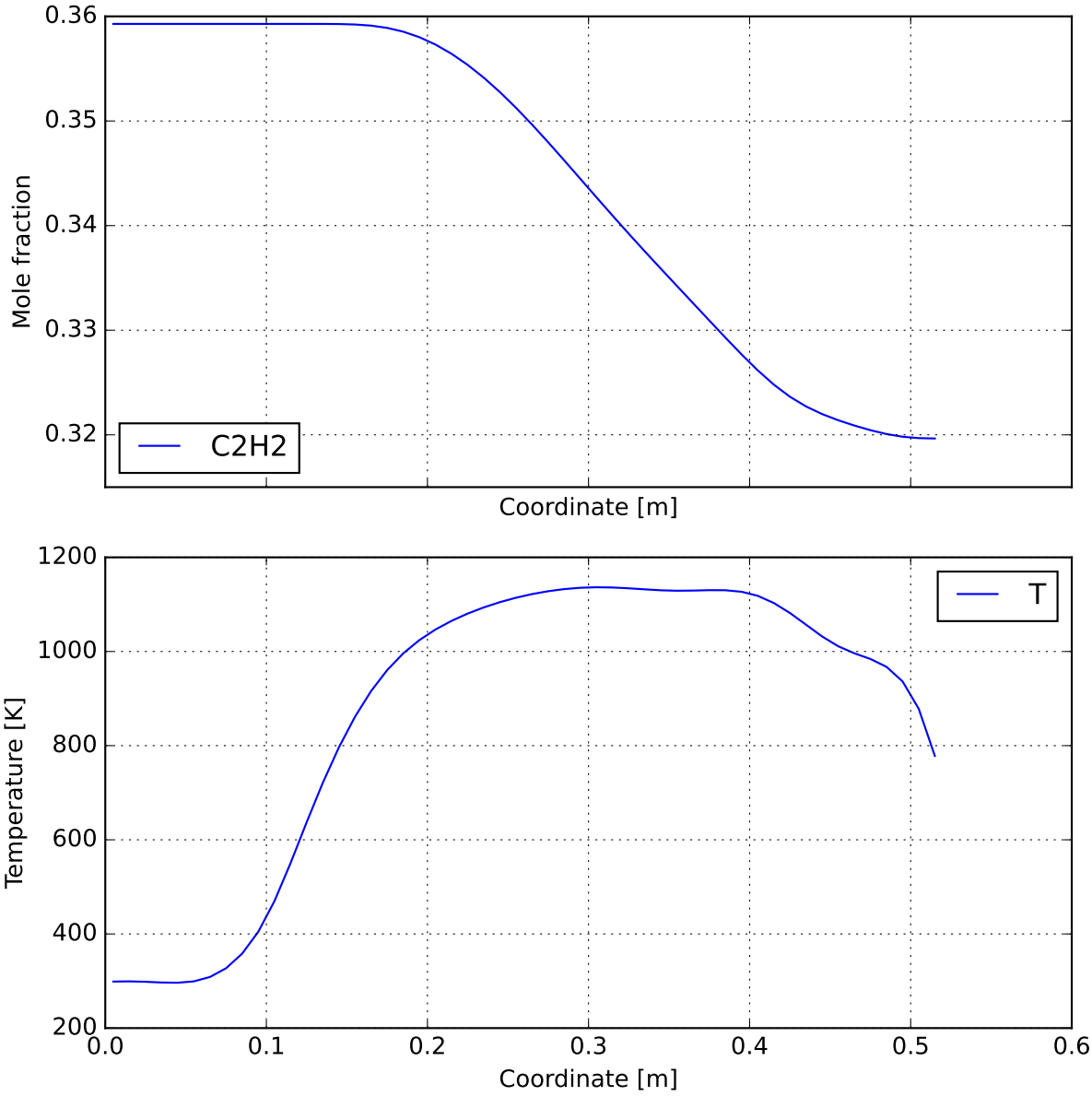
Reactor Engineering

19 Acetylene pyrolysis reactor

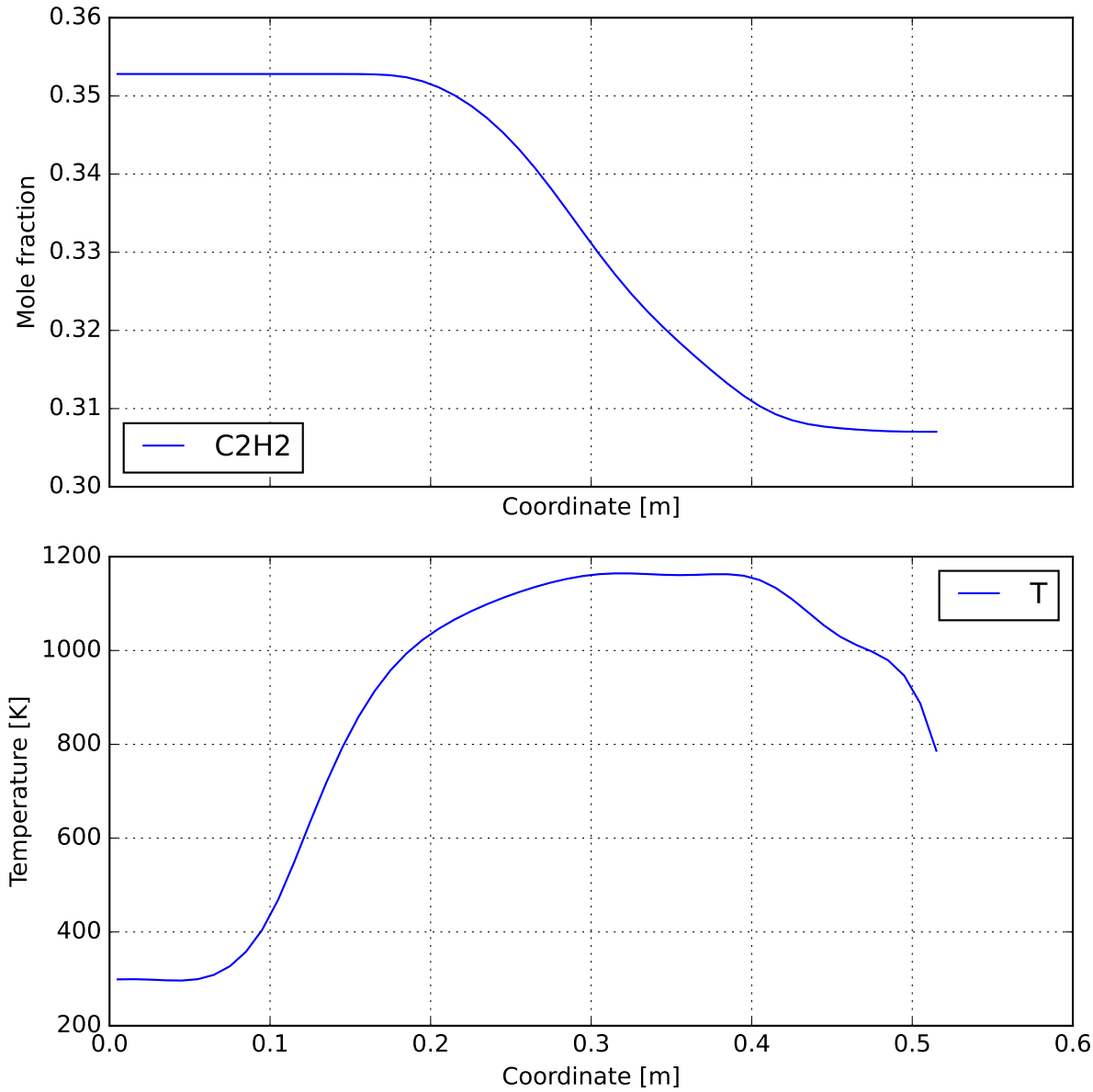
19.1 Toolbox

19.2 Reference case

```
Skipping cell: MJ_SOLVE_FINEST=True

CPU times: total: 781 ms
Wall time: 779 ms
```



```
CPU times: total: 1.89 s
Wall time: 1.89 s
```



19.3 Scan table

```
Working on case no. 1
Working on case no. 2
Working on case no. 3
Working on case no. 4
Working on case no. 5
Working on case no. 6
Working on case no. 7
Working on case no. 8
Working on case no. 9
Working on case no. 10
```

```
Working on case no. 11
Working on case no. 12
Working on case no. 13
Working on case no. 14
CPU times: total: 25.6 s
Wall time: 25.6 s

Working on case no. 1
Working on case no. 2
Working on case no. 3
Working on case no. 4
Working on case no. 5
Working on case no. 6
Working on case no. 7
Working on case no. 8
Working on case no. 9
Working on case no. 10
Working on case no. 11
Working on case no. 12
Working on case no. 13
Working on case no. 14
CPU times: total: 10.8 s
Wall time: 10.7 s

Skipping cell: MJ_SOLVE_FINEST=True
```

	N	P	Q	T	X_meas	X_dalmazsi_2017	err_dalmazsi_2017	err_graf_2007
0	1	50	222	773	0.352	0.352786	0.223212	2.032933
1	2	50	222	873	0.364	0.352540	-3.148330	-1.550413
2	3	50	222	973	0.364	0.350258	-3.775208	-2.602299
3	4	50	222	1073	0.346	0.339812	-1.788298	-1.188026
4	5	50	222	1123	0.312	0.326556	4.665512	6.179187
5	6	50	222	1173	0.307	0.307034	0.010994	4.120820
6	7	50	222	1273	0.288	0.290866	0.995060	6.743916
7	8	30	222	1173	0.323	0.329143	1.901911	5.859843
8	9	30	222	1223	0.314	0.318436	1.412680	7.828360
9	10	100	222	1173	0.249	0.242491	-2.613868	4.125649
10	11	100	222	1223	0.226	0.230372	1.934472	5.944962
11	12	100	222	1273	0.201	0.221251	10.075293	12.714431
12	13	100	378	1023	0.343	0.342777	-0.065090	0.254930
13	14	100	378	1123	0.298	0.305793	2.615003	6.255879

	err_dalmazsi_2017	err_graf_2007
0	3.53261	5.758262

20 Counter-current plug flow

20.1 Introduction

The goal of this tutorial is to discuss a sequence of increasingly complex plug-flow reactor (PFR) models. These are all built upon `PlugFlowChainCantera` which approximates a PFR as a cascade of perfect-stirred reactors (PSR), as illustrated here. Care must be taken when using such an approach because this zero-dimensional modeling does not formally represent a PFR, but approaches one - so grid convergence studies might be required to validate a model, what is beyond our scope here (and has been done during the conception of this report), which limits itself to studying the coupling of reactors. Nonetheless, this representation proves very interesting when assembling complex systems as it remains modular, allowing for extensibility of models.

In what follows we will be guided through the following models:

- A single PFR with possibility of distributed mass sources along its length; this case is often encountered in industry when modeling reactors coupled through a permeable membrane, or in a rotary kiln, where solids may eventually release gases, what is especially important in the field of biomatter gaseification.
- Next we will couple a pair of reactors as conceived in the first example, which will be allowed to exchange energy through a pseudo-convective model; there is no major difficulty at this point as the coupling can be stabilized through a relaxation routine. We investigate how the ordering of reactor solution affects convergence rate.
- At the last level of complexity studied here, we add a shared common wall around the pair of reactors of the previous step; one might argue that such a system would require two *wall* temperatures to be described at steady state; here we will assume that this wall is *rotating* around the pair of reactors in such a way that it remains in contact with a reactor during part of a cycle, and a *statistically significant mean temperature* is solely used as a process measurable.

Because writting long classes is not didatic for the purposes of a tutorial, we will use a *semi-object-oriented* programming approach here. That means, there will be minimal classes just holding the basic elements representing the state of a problem, and functions that resemble class methods (receiving the class instance as first argument) to perform the required operations.

Below we start by importing all required tools at once:

Because some cells take some time to execute, during development you can set this to `"true"` so that they are not run. You can recognize these cells by the presence of magic `%%skipper`. Setting this to `"false"` will not skip the cells (this is the logic of the booleans here, we are saying true to skip, not true to execute!).

20.2 Shared functionalities

This section is not commented, variables and functions should speak by themselves.

20.3 Single plug-flow reactor

For the single reactor `ReactorModel` we write a simple wrapper around `PlugFlowChainCantera`; this model holds an instance of a `solution` object conceived with the same mechanism as provided to the PFR, and a simple `source` structure for the distributed source terms.

Both the `solution` and `source` are used in `add_source` to inject a mass flow rate `mdot` at temperature `T`, and composition `X` (given in mole fractions) at a set of cell indices `where` along the reactor. We will see later how discretization is performed and thus how to link `where` to an actual axial coordinate along the reactor.

The underlining PFR model has already its own solution method (which is actually called `update`) that *seeks* a steady state for the reactor given a certaing `source`; sometimes it fail to converge and multiple calls may be required to find the steady state of a reactor. In `solve_reactor` below we wrap the aforementioned method and add some reporting.

20.3.1 Example - single PFR simulation

As this first case is already fully supported by the underlining library class, we are ready to solve a first example problem. Comments are provided in the function `sample_single` to guide you through the setup. A distributed power supply is added to the reactor as an analytical function as:

$$Q(z) = \begin{cases} A\left[1 - \exp\left(-\dfrac{z}{z_0}\right)\right] & z < \dfrac{L}{2} \\ 0 & \text{otherwise} \end{cases}$$

This is arbitrarily parametrized and is intended just to show that accessing `model.source.Q` allows to provide any heat source function to a reactor.

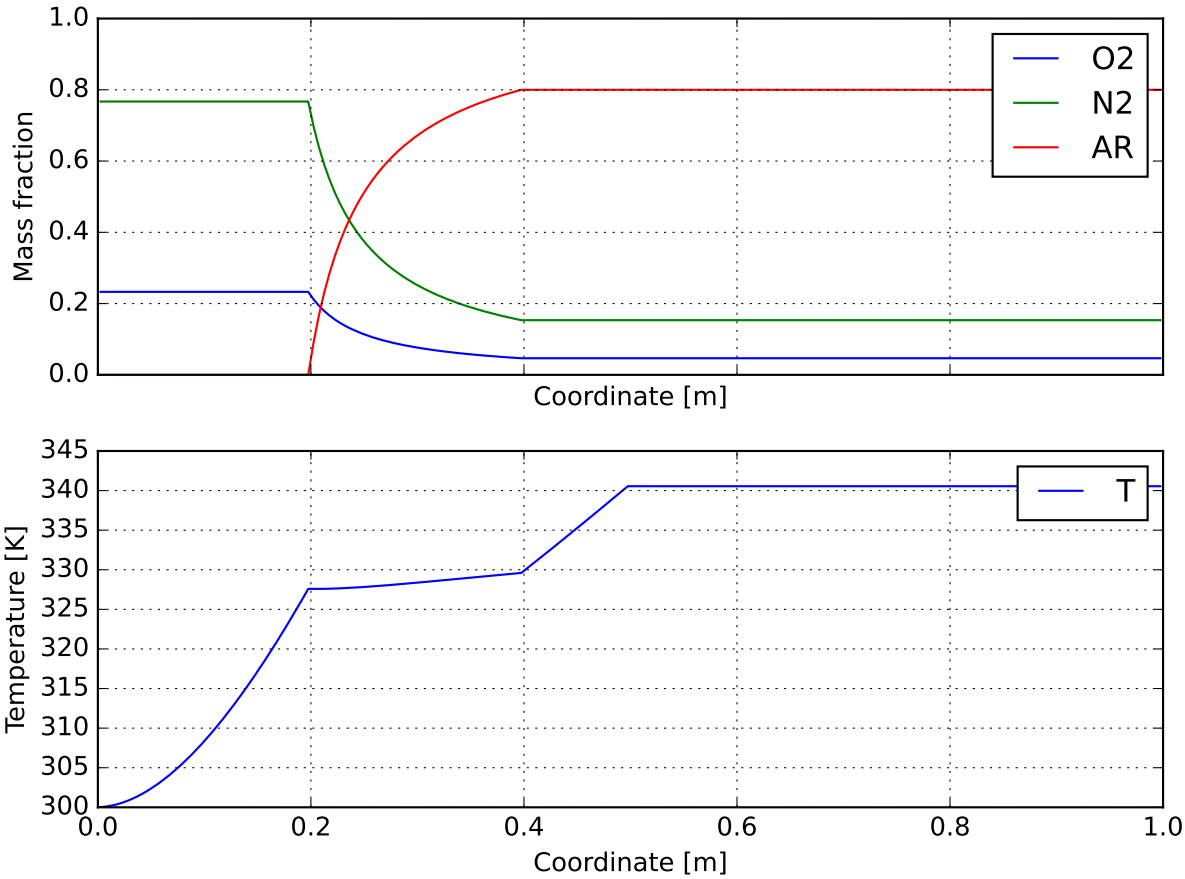
We said no convergence study was going to be performed; that was a lie. Below we inspect the evolution of final temperature in terms of reactor cell length in range $l \in [0.001; 0.1]$ m. Solution quickly approaches 340.6 K, and for practical purposes a discretization of 1 cm seems a good compromise between execution time and accuracy. Also notice that total supplied power may depend on discretization because of how it is provided.

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.035993e+03 |
| Final temperature    | K   | +3.404046e+02 |

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000357e+03 |
| Final temperature    | K   | +3.405258e+02 |

Skipping cell: MJ_SOLVE_FINEST=True
```

Below we show the report of first solution, which here is no better than an initial guess; sequential solution of reactor towards steady state is recommended in practice.

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000085e+03 |
| Final temperature    | K   | +3.395247e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000085e+03 |
| Final temperature    | K   | +3.405501e+02 |
```

Finally we illustate the composition and temperature profiles induced by the simultaneous mass and heat transfer:



You can also access the data in base reactor for post-processing as desired:

	0	1	2	3	4
z_cell	0.002500	0.007500	0.012500	0.017500	0.022500
V_cell	0.000039	0.000039	0.000039	0.000039	0.000039
Q_cell	1.177767	3.489592	5.744338	7.943414	10.088195
m_cell	0.000046	0.000046	0.000046	0.000046	0.000046
mdot_cell	0.050000	0.050000	0.050000	0.050000	0.050000
T	300.023321	300.092416	300.206156	300.363436	300.563179
density	1.171880	1.171610	1.171166	1.170553	1.169775
Y_O2	0.232909	0.232909	0.232909	0.232909	0.232909
Y_N2	0.767091	0.767091	0.767091	0.767091	0.767091
Y_AR	0.000000	0.000000	0.000000	0.000000	0.000000

20.4 Counter-current flow reactors

The tooling required to compute a pair of reactors is already provided by `ReactorModel`; all we have to do is wrap a pair of reactors in `CounterCurrentReactors`. For convenience we store a copy of the axial coordinate (which is the same for both reactors). Notice here the introduction of `U`, a global heat transfer coefficient between phases, `Q`, the associated heat flux, and a relaxation manager.

One cannot compute heat fluxes between reactors before having an initial temperature profile established (see it as an initial guess). Function `initialize` is provided for that end and must be called after mass flows have been setup.

There are two ways one can solve the relaxation problem here, each with its strengths and weaknesses; one could simply calculate calculate the flux, apply it to both reactors, and solve (`iter_direct`) or compose a series of flux and solution calculations (`iter_alternate`). When initial guess is not good enough, the alternating approach tends to deliver results in a more reliable fashion (because it is compatible with high relaxation factors), but it is slower. On the other hand, direct solution may converge faster as solution is already relaxed.

IMPORTANT: notice here that we get the reverse of `T2` to compute flux (as the reactors are in counter-current) and then `model.Q` is reversed againt to set heat flux to reactor 2. This class could easily be generalized by adding a flag on whether temperature and flux should be treated as is (co-current) or reversed.

Based on the previous discussion, a mixed approach can be proposed: relax the problem with the alternating solution and move to direct one after a certain criteria or maximum number of iterations has been achieved. This is emulated in `solve_pair` provided below.

A simple standard ploting function is provided for the counter-current reactor pair:

20.4.1 Example - pair of coupled PFR

This second example replaces the user-defined heat flow function by a convective heat exchange through a fin separating the reactions; this is modeled through a heat transfer coefficient `h` which is made constant here for illustration purposes only. To test the robustness of the approach, a relativelly high dilution flow is applied to a concentrated region in reactor 2 (hot stream).

After building the reactor pair, inlet and distributed conditions and the reactors are initialized, *.i.e.* solved individually, before iterative solution is performed. The user may require a final solution step for printing reactor quantities in a table.

For low relaxation values ($\alpha < 0.5$), both methods may eventually converge under the same number of iterations.

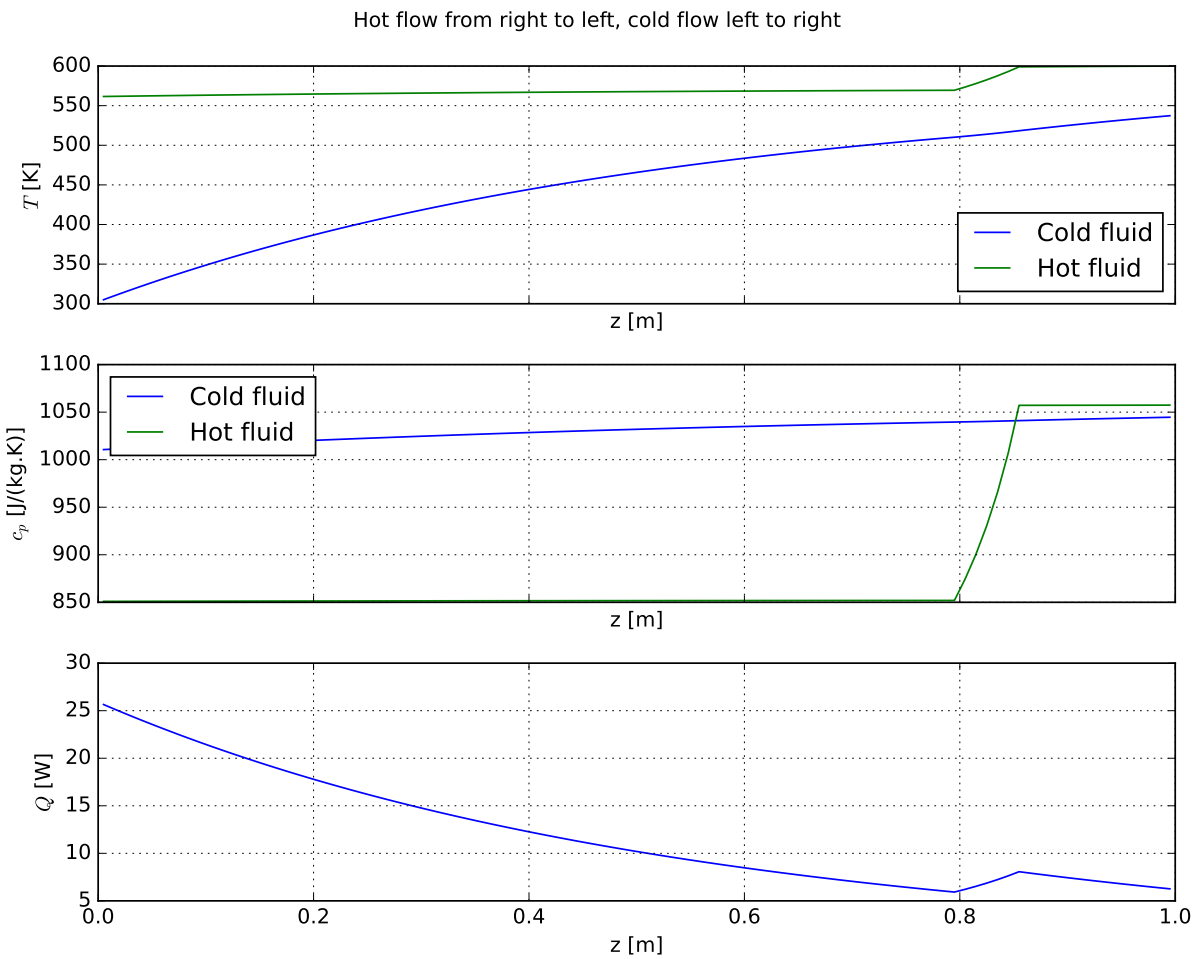
```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.216871e+03 |
| Final temperature    | K   | +5.372994e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -1.216871e+03 |
| Final temperature    | K   | +5.615588e+02 |

Skipping cell: MJ_SOLVE_FINEST=True
```

Refining the grid for the present case has an impact of a few degrees over final solution (but takes much longer to solve), as illustrated below. Below 2 mm there is little change in the final solution and this value is a practical limit for grid refinement here.

```
Skipping cell: MJ_SOLVE_FINEST=True
```

```
Skipping cell: MJ_SOLVE_FINEST=True
```

Finally we display the results; for the hot stream we observe a sharp decrease in temperature (and associated specific heat capacity) due to argon dilution. Cold stream progressivelly heats up along the channel length as expected.



20.5 Adding losses to the environment

So far we have developed the ideas required to reach the final goal of conceiving a fully coupled system with a pair of reactors and walls. Again we proceed with a wrapper class which countains our problem variables, but in this case we fully develop it as a single block. Also, this time documentation is provided in the class itself as that seems clearer to the author in this case, and provides some closure to the study.

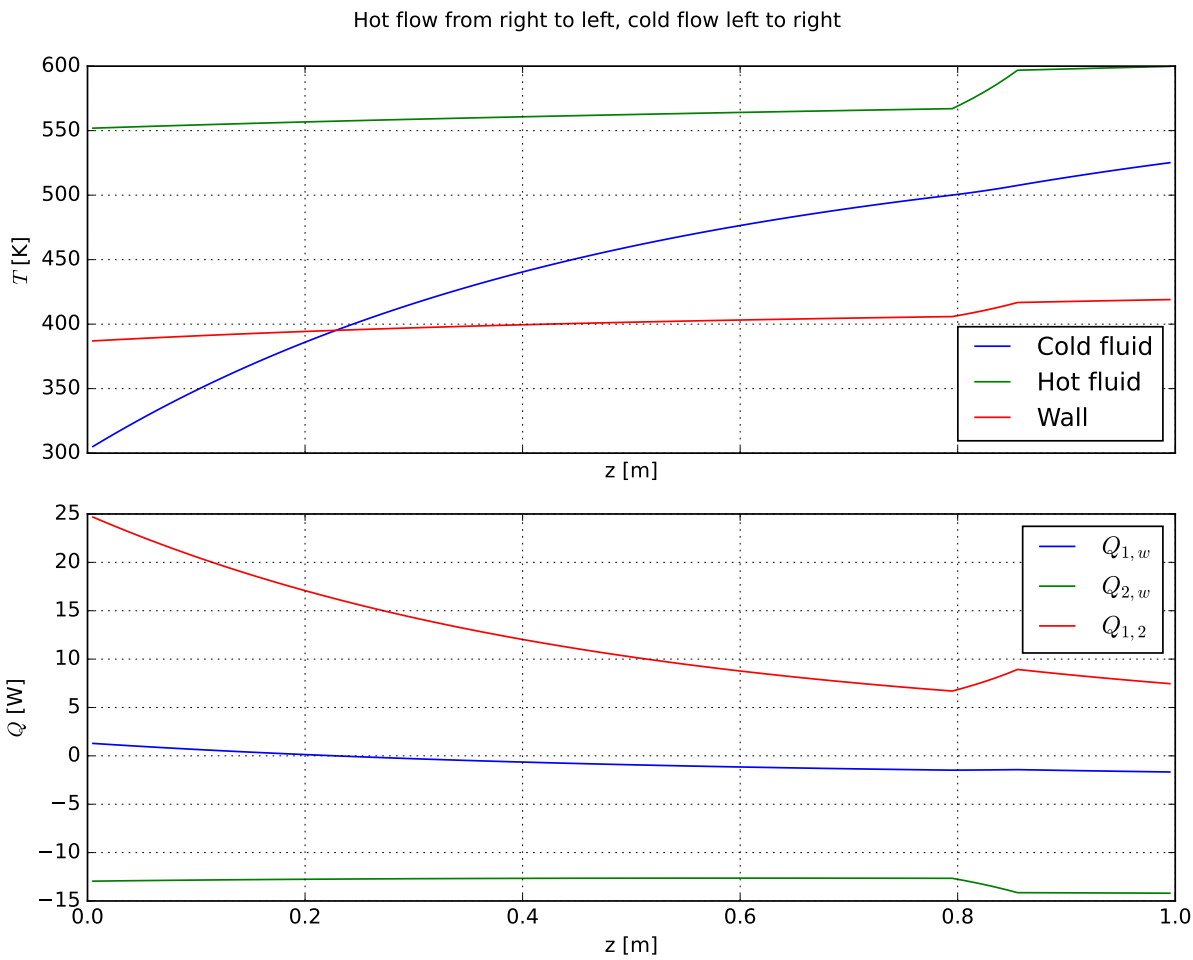
The main increment in problem solution here is the extra step of solving for the wall temperature, which can be found from the global flux definition:

$$\dot{Q}_w = \frac{T_w - T_e}{R_{tot}} \quad \text{where} \quad R_{tot} = \frac{1}{hA} + \frac{1}{2\pi lk} \log \frac{R_{ext}}{R_{int}}$$

We make use of the main takes of the previous sections, which have shown that alternating reactor solution order is not always an advantage; here the problem has been programmed to use direct solution. Relaxation is applied to reactor fluxes, but not to wall flux, which is computed from a balance of the relaxed quantities for the sake of energy conservation.

20.5.1 Example - environment coupling

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.003972e+03 |
| Final temperature    | K   | +4.963894e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +3.000000e+02 |
| Total external power | W   | -2.210504e+03 |
| Final temperature    | K   | +5.216989e+02 |

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |

Skipping cell: MJ_SOLVE_FINEST=True

Skipping cell: MJ_SOLVE_FINEST=True
```

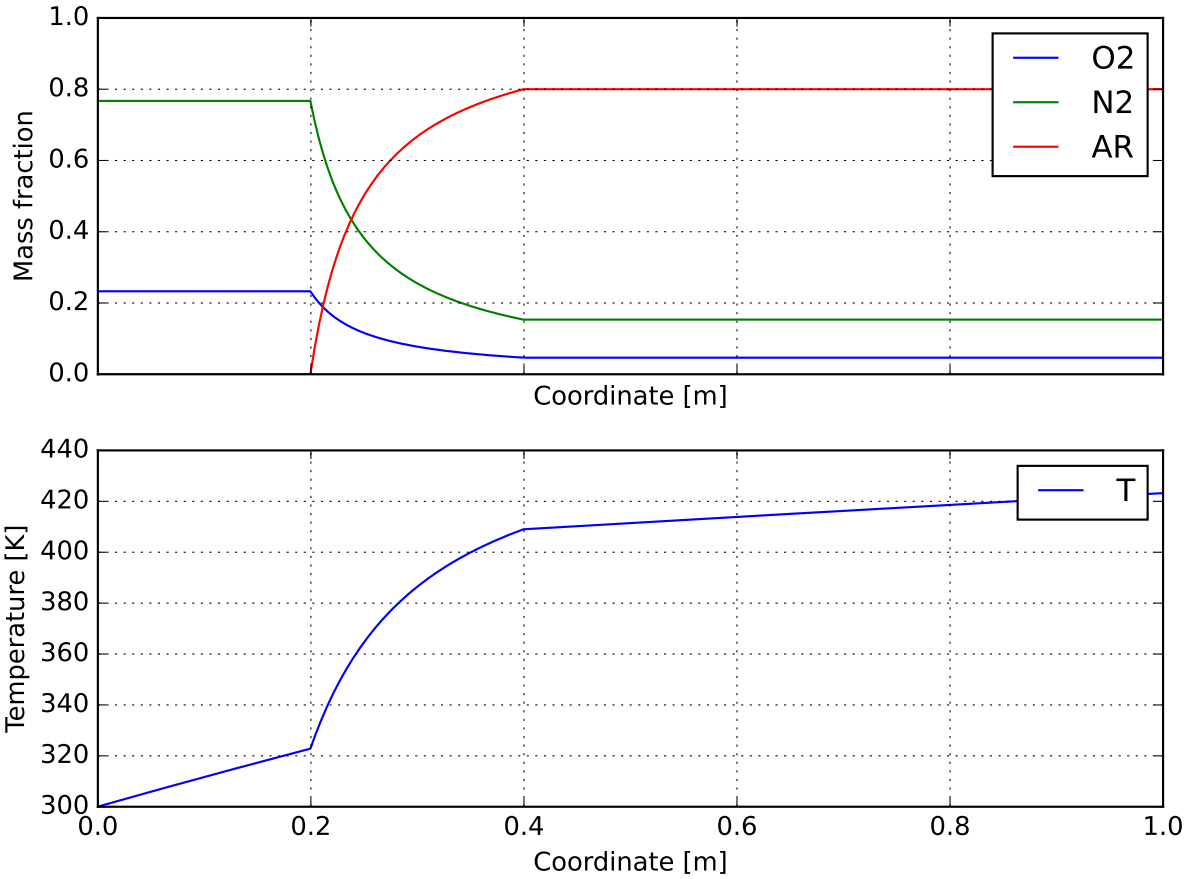


20.6 Registering flow for a single reactor

Below we develop further the idea into registering heat flows as an alternative to relaxation. The method may prove useful because heat flows are computed from current state of the reactors instead of an external guess.

20.6.1 Example - prototype with a single reactor

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +4.237303e+03 |
| Final temperature    | K   | +4.232112e+02 |
```



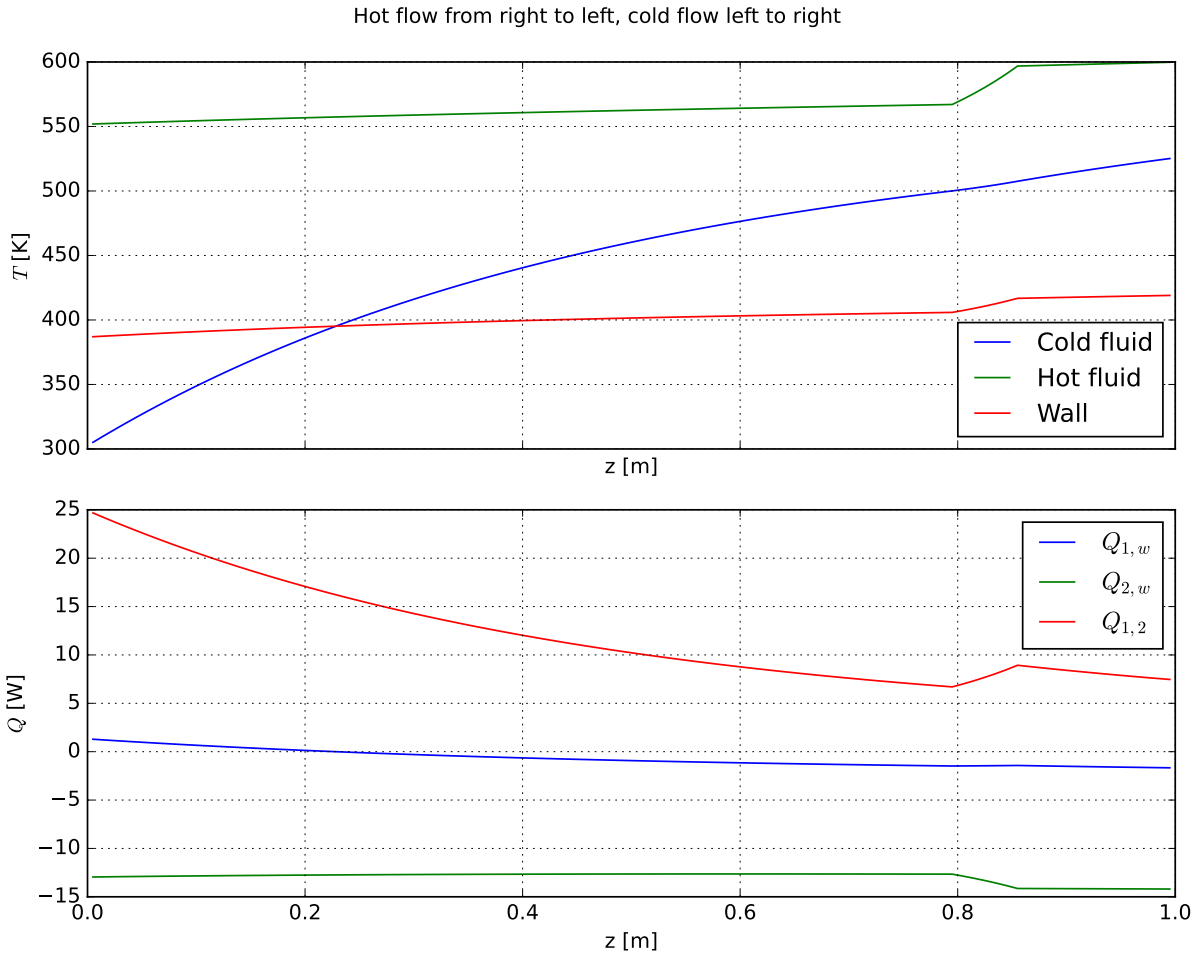
20.7 Registering flow for a pair of reactors

As a last level of improvement, we generalize the approach for a pair of reactors. After some experimentation it was concluded that falling back to the alternating approach with intermediate wall solution lead to the best performance, probably because of decreased system stifness. Relaxing the wall heat flow allows for faster convergence.

20.7.1 Example - extension to coupled system

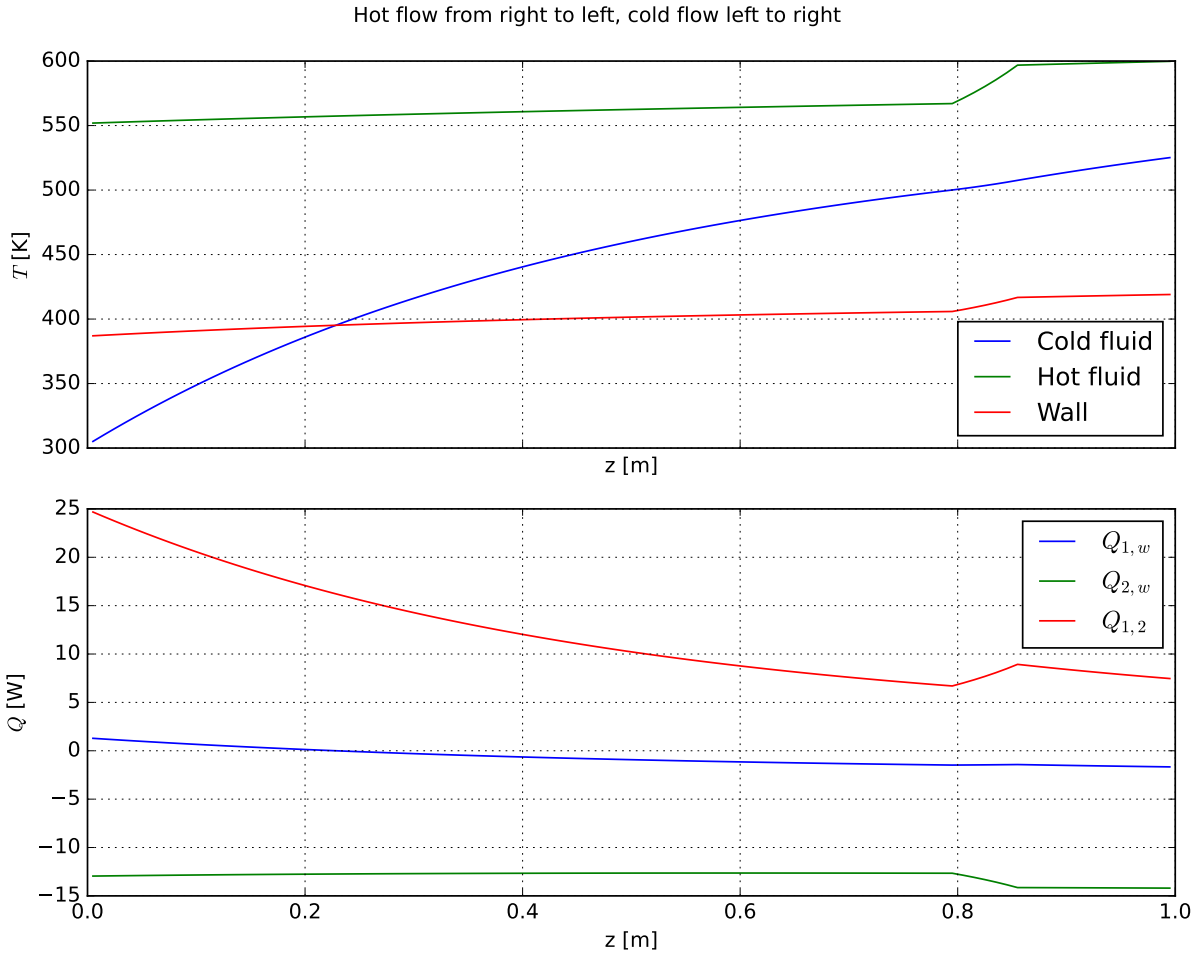
```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 17.8 s
Wall time: 17.8 s

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
```

```
| Final temperature     | K    | +5.251459e+02 |
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +1.600000e+02 |
| Total external power | W    | -2.516963e+03 |
| Final temperature     | K    | +5.519201e+02 |
CPU times: total: 19.4 s
Wall time: 19.5 s
```

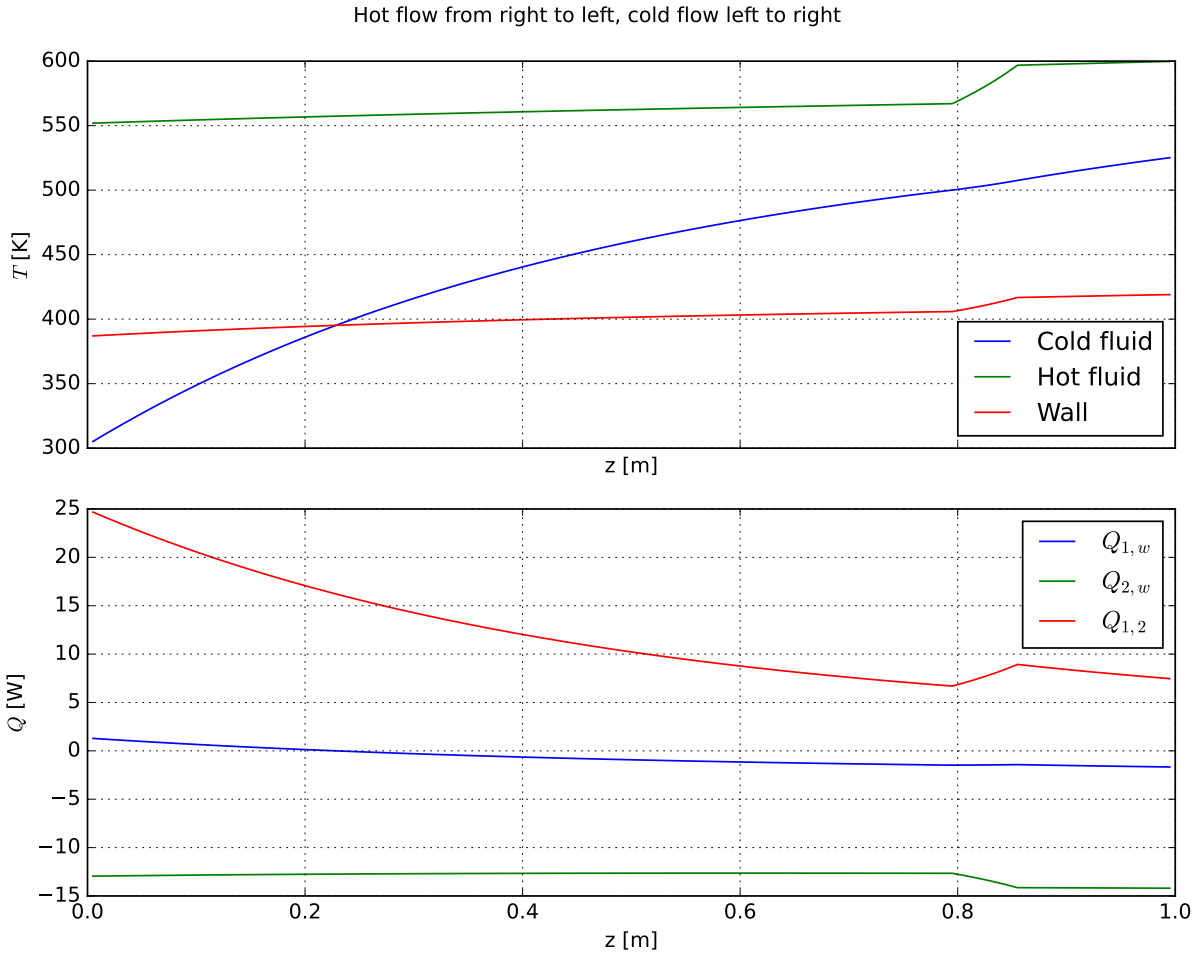



```
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +5.000000e+00 |
| Total external power | W    | +1.153462e+03 |
| Final temperature     | K    | +5.251459e+02 |
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +1.600000e+02 |
| Total external power | W    | -2.516963e+03 |
| Final temperature     | K    | +5.519201e+02 |
CPU times: total: 17.1 s
Wall time: 17.2 s

|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +5.000000e+00 |
| Total external power | W    | +1.153462e+03 |
| Final temperature     | K    | +5.251459e+02 |
```

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 18 s
Wall time: 18 s
```




```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 4.05 s
Wall time: 4.01 s
```


Hot flow from right to left, cold flow left to right
600
550
500
450
400
350
300
T [K]
z [m]
Cold fluid
Hot fluid
Wall
25
20
15
10
5
0
−5
−10
−15
Q [W]
0.0
0.2
0.4
0.6
0.8
1.0
z [m]
$Q_{1,w}$
$Q_{2,w}$
$Q_{1,2}$

21 Fitting NASA9 data

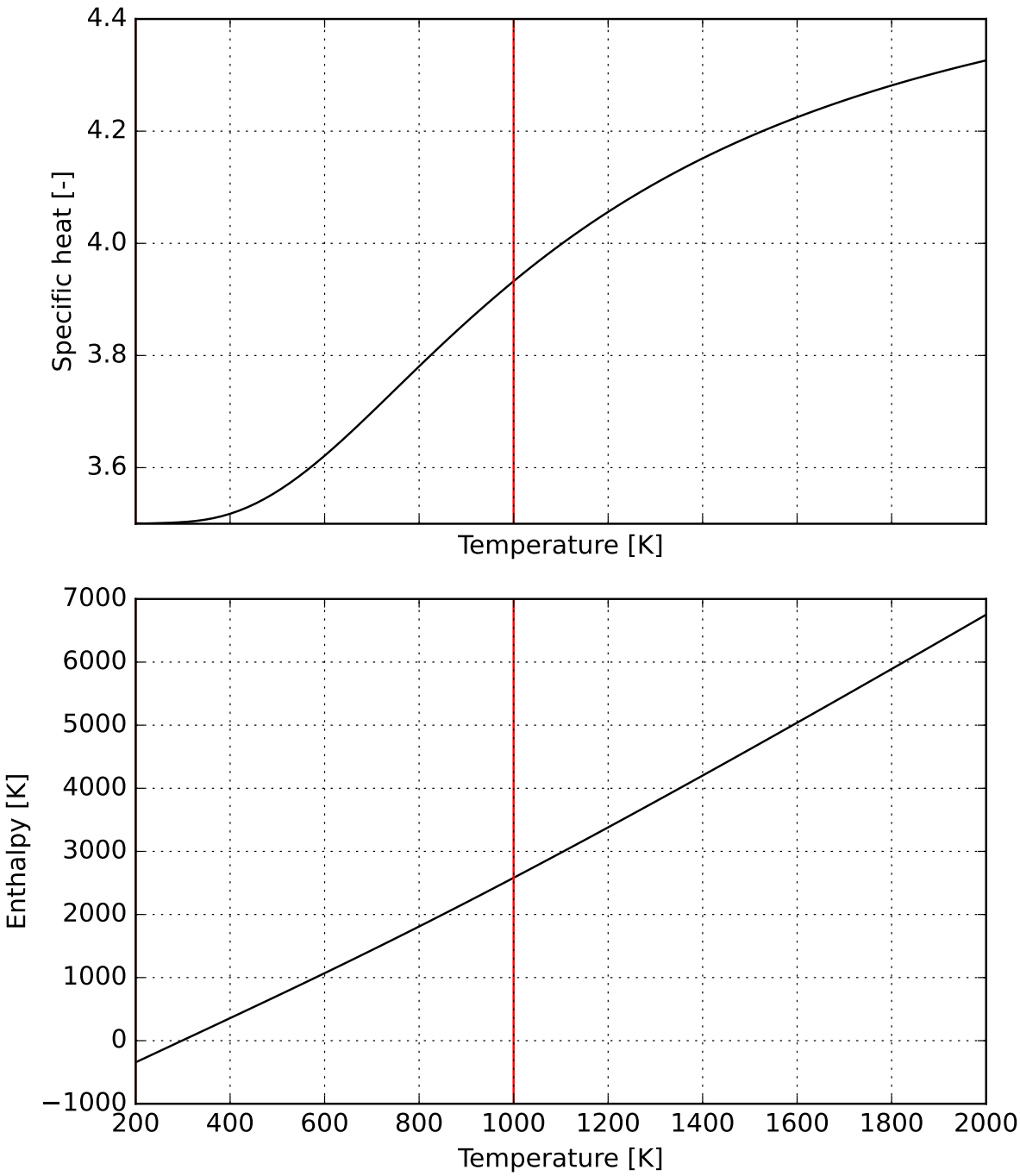
The goal of this document is to provide a better understanding on how to fit empirical data to NASA9 parameterization of thermodynamic properties; more details are provided in the original NASA report. A progressive level of complexity is used to provide better insights of how the species thermodynamics is encoded in this format.

21.1 Helper utilities

21.2 Nitrogen actual fit

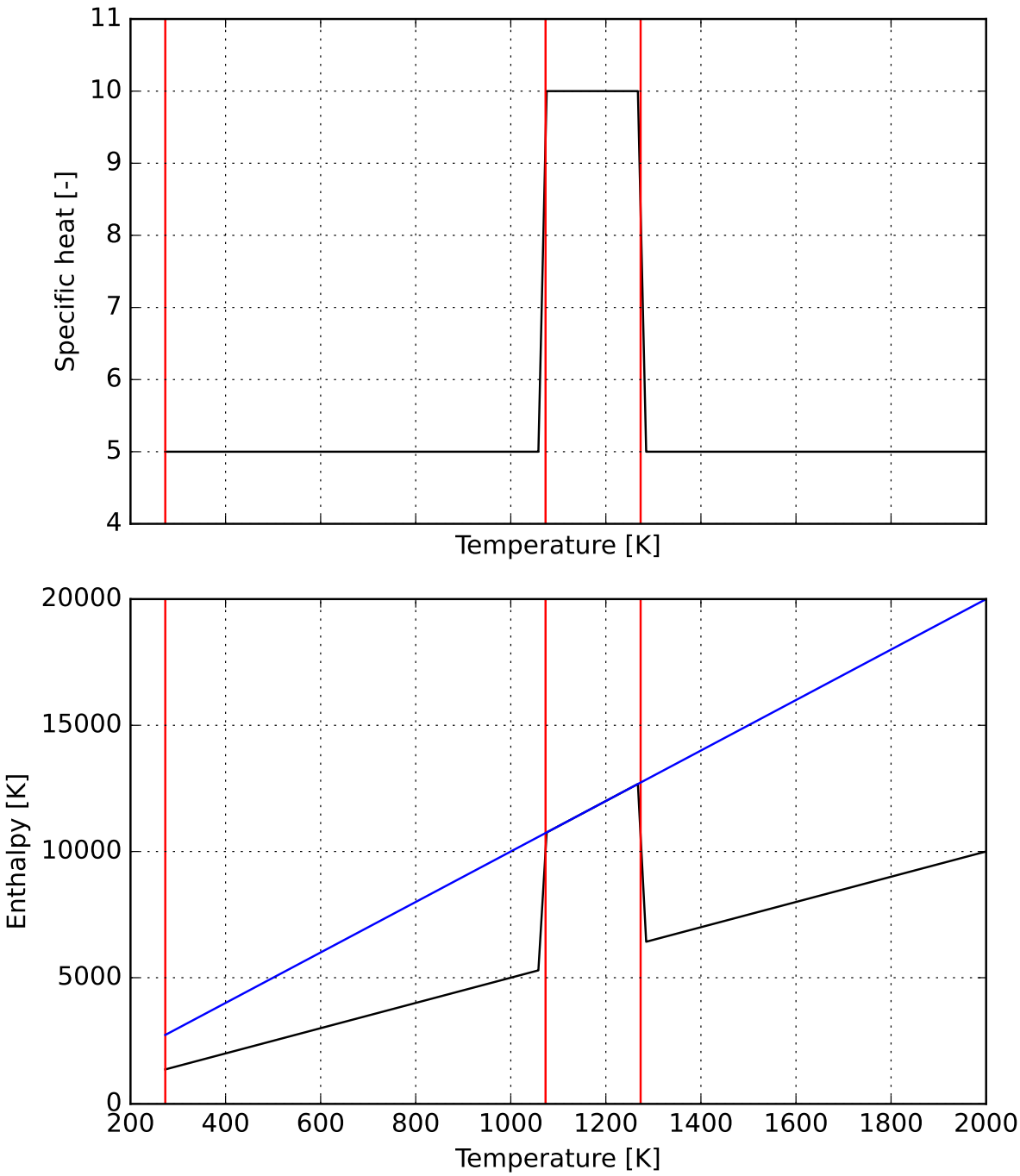
- Below we display in text format an actual NASA9 fit taken from `airNASA9.yaml` data file provided with Cantera; this is a fully parameterized species, making it a bit more difficult to visually understand the role of different coefficients
- The third coefficient a_2 is related to the specific heat intercept when $T = 0$ and the eighth coefficient a_7 represents the integration constant for providing enthalpy over a range
- Since NASA9 functions are provided in dimensionless form, it is worth dividing computed values by ideal gas constant, so that values are in the same *units* as the provided coefficients

The next cell illustrates the dimensionless specific heat and enthalpy over a typical industrial range.



21.3 Enforcing constant specific heat

- Next we create a dummy species (in Cantera composition needs to include *actual* atoms or you need to create a new database for elements) with all coefficients set to zero except those related to specific heat
- This way we establish a piecewise-constant specific heat function; notice that doing so will produce thermodynamic inconsistent steps in enthalpy



- First, notice that the temperature ranges are *closed-open* ranges, meaning the first point belongs to the range but the last not; we introduce `T_tol` to compute a *last* value of enthalpy over each range as close as possible to the next
- To ensure continuity of enthalpy function, we evaluate the gap `h_hi-h_lo` at each range end and add that to the correction of the previous range (as enthalpy is integrated from the reference temperature)
- Using `dh0 = s.dimless_h(298.15)` to compute the initial value is optional and simply enforces zero enthalpy at the reference state, but in principle it won't affect any computations as only *enthalpy changes* make sense physically

Note: as we are dealing with constant c_P here, we could have computed all the above enthalpy values straight from specific heat integration by hand, but this would be more complicated in general

cases and it was chosen to present the more general (and with least user interaction) approach. See below that the enthalpy at reference temperature fits nicely with the specific heat provided above.

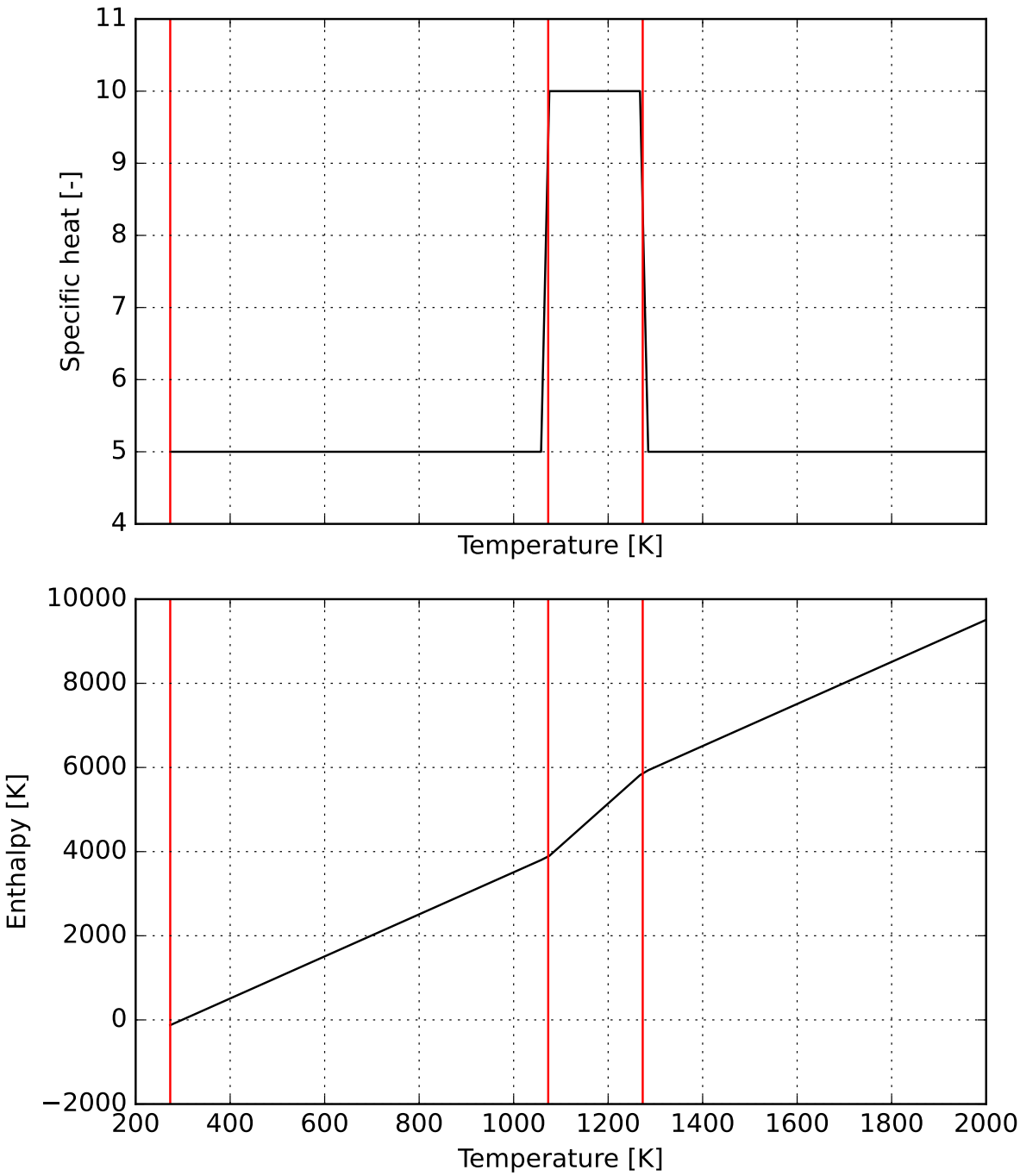
```
np.True_
```

Generalizing for all the steps, we could manually compute everything as follows:

```
((6856.5, np.float64(6856.500000000005)),
 (490.75, np.float64(490.75000000001455)))
```

21.4 Ensuring enthalpy continuity

- To wrap up, we use string interpolation to fix the integration constants required to ensure continuity of enthalpy function



21.5 Application to a real material

We start by providing actual physical values to be converted into NASA9 format.

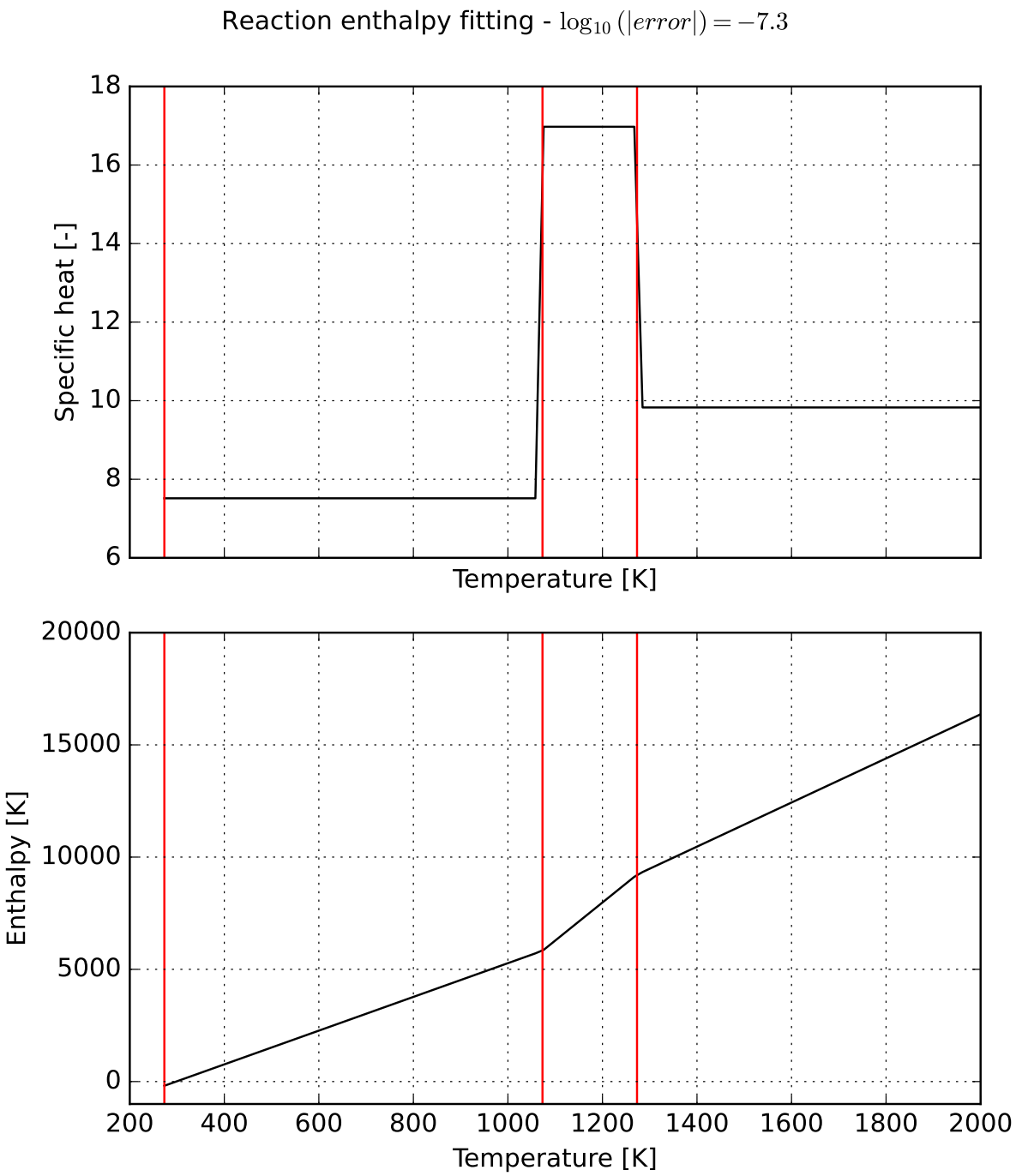
Calculations involve three phases (1) convert into units used by NASA9 format, (2) smearing of reaction enthalpy into specific heat, and (3) corrections required for continuity of enthalpy function.

Alternative for using a constant specific heat over the *whole range*:

Next we generate the YAML input that will be used in Cantera database file.

```
name: meal
composition: {Si: 1, O: 2}
thermo:
  model: NASA9
  temperature-ranges: [273.15, 1073.15, 1273.15, 6000.0]
  data:
  - [ 0.00000000e+00,  0.00000000e+00,  7.51537686e+00,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -2.24070961e+03,  0.00000000e+00]

  - [ 0.00000000e+00,  0.00000000e+00,  1.69746349e+01,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -1.23919123e+04,  0.00000000e+00]

  - [ 0.00000000e+00,  0.00000000e+00,  9.82780051e+00,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -3.29292018e+03,  0.00000000e+00]


name: meal
composition: {Si: 1, O: 2}
thermo:
  model: constant-cp
  cp0: 95.5200 J/mol/K
  h0: 0.0 J/mol®
```

Finally we inspect results before proceeding to use of newly fit data.



Important: NASA9 is the only parameterization supporting an arbitrary number of temperature ranges in Cantera. Given the number of parameters, it is expected to slow down simulations. For more details, please check the documentation.

21.6 Generalizing approach

> ⚠ Warning
>
> *Work in progress from here on... the code is right but the thermodynamics is pretty much sketchy!*

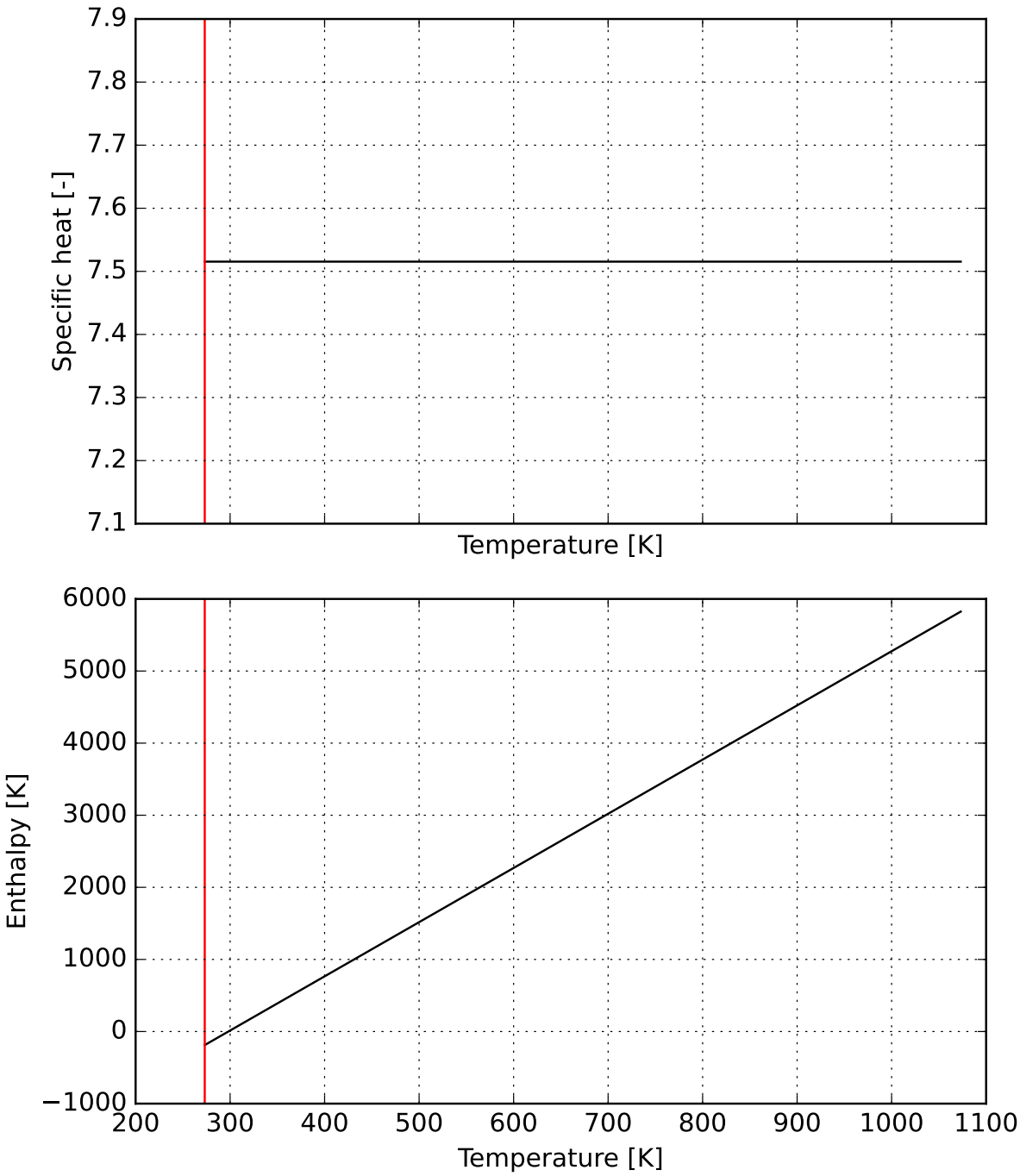
We start by declaring the symbols for temperature `t` and model coefficients `a`:

The next cell provides the representations of specific heat capacity and enthalpy as described here. We also provide the derivatives of theses functions computed algorithmically with help of `casadi.gradient`.

Below we illustrate the symbolic computational graph of operations for checking the functions:

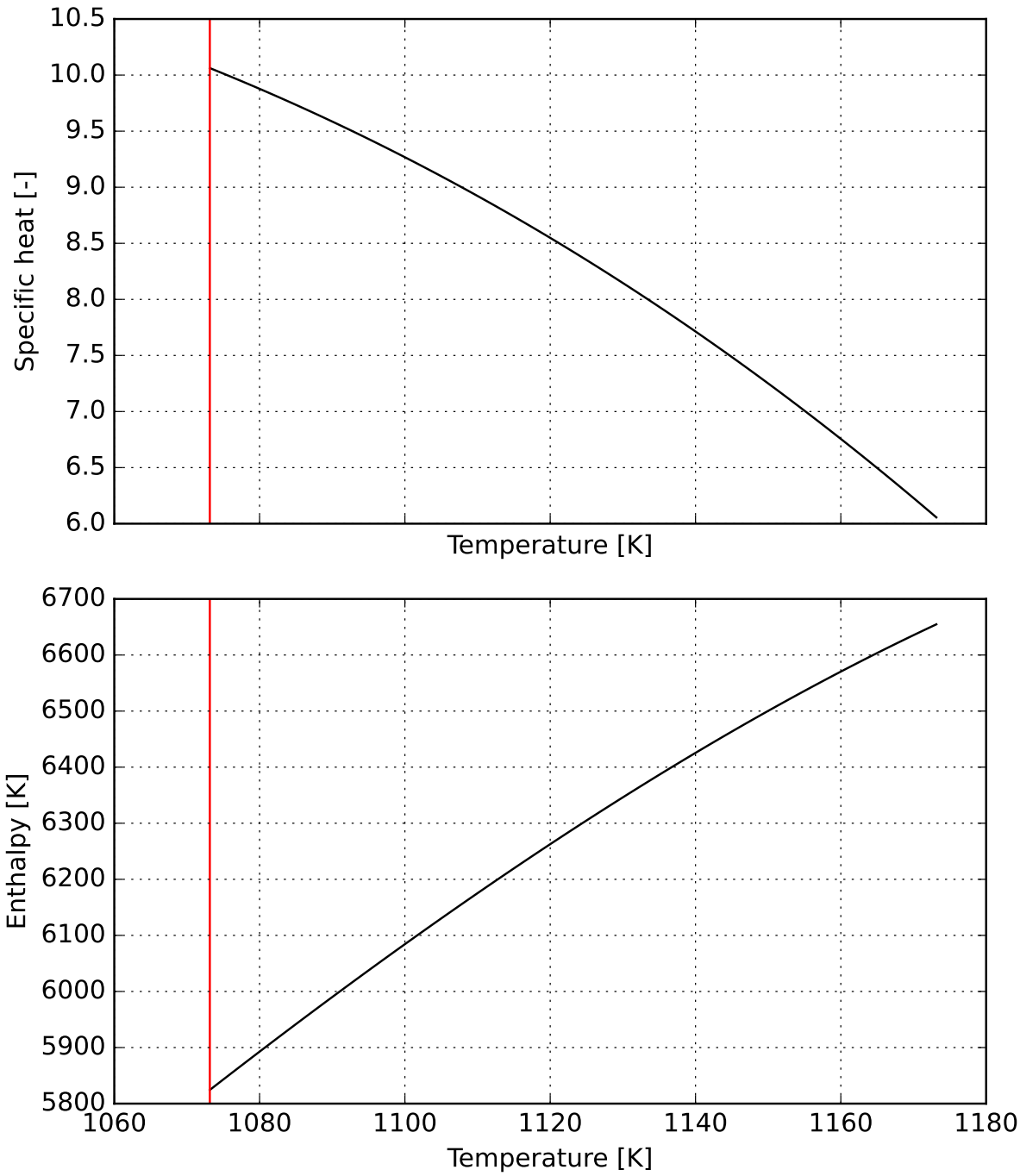
```
SX(((((a_0+(a_1*t))+(a_2*sq(t)))+(a_3*(t*sq(t))))+(a_4*sq(sq(t)))))
```

Below the reaction start we enforce a constant specific heat with zero enthalpy at reference temperature:



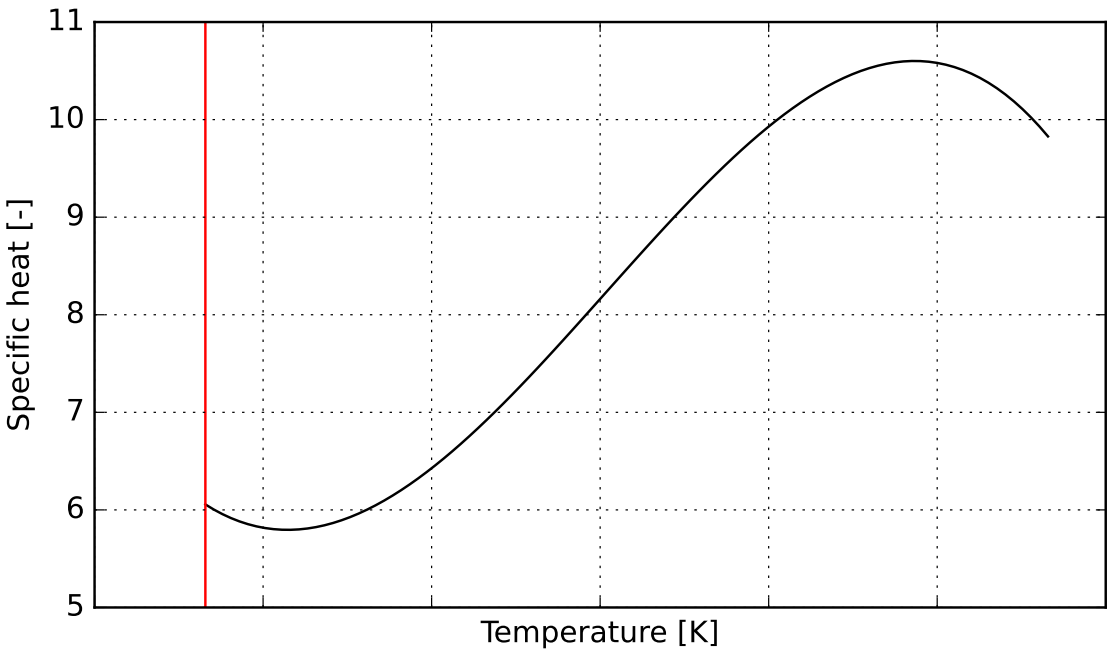
The reaction range is split in two parts, the first corresponding to a raising specific heat which reaches the maximum value.

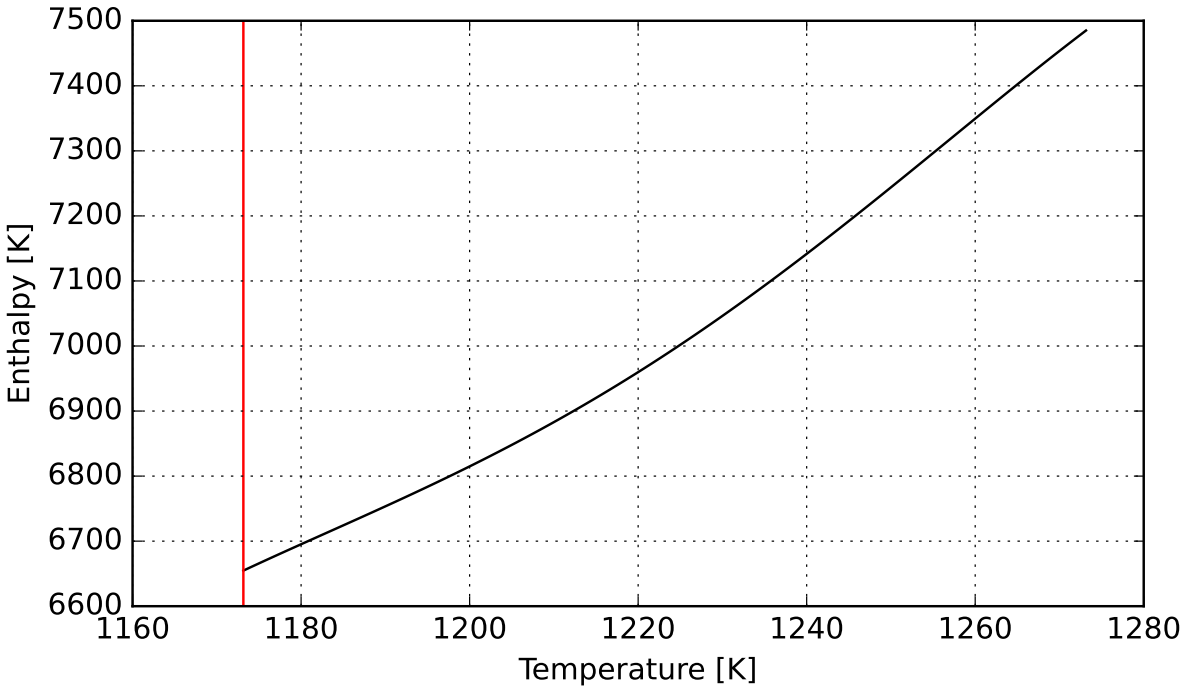
```
DM([-1.81899e-12, -1.25056e-12])
```

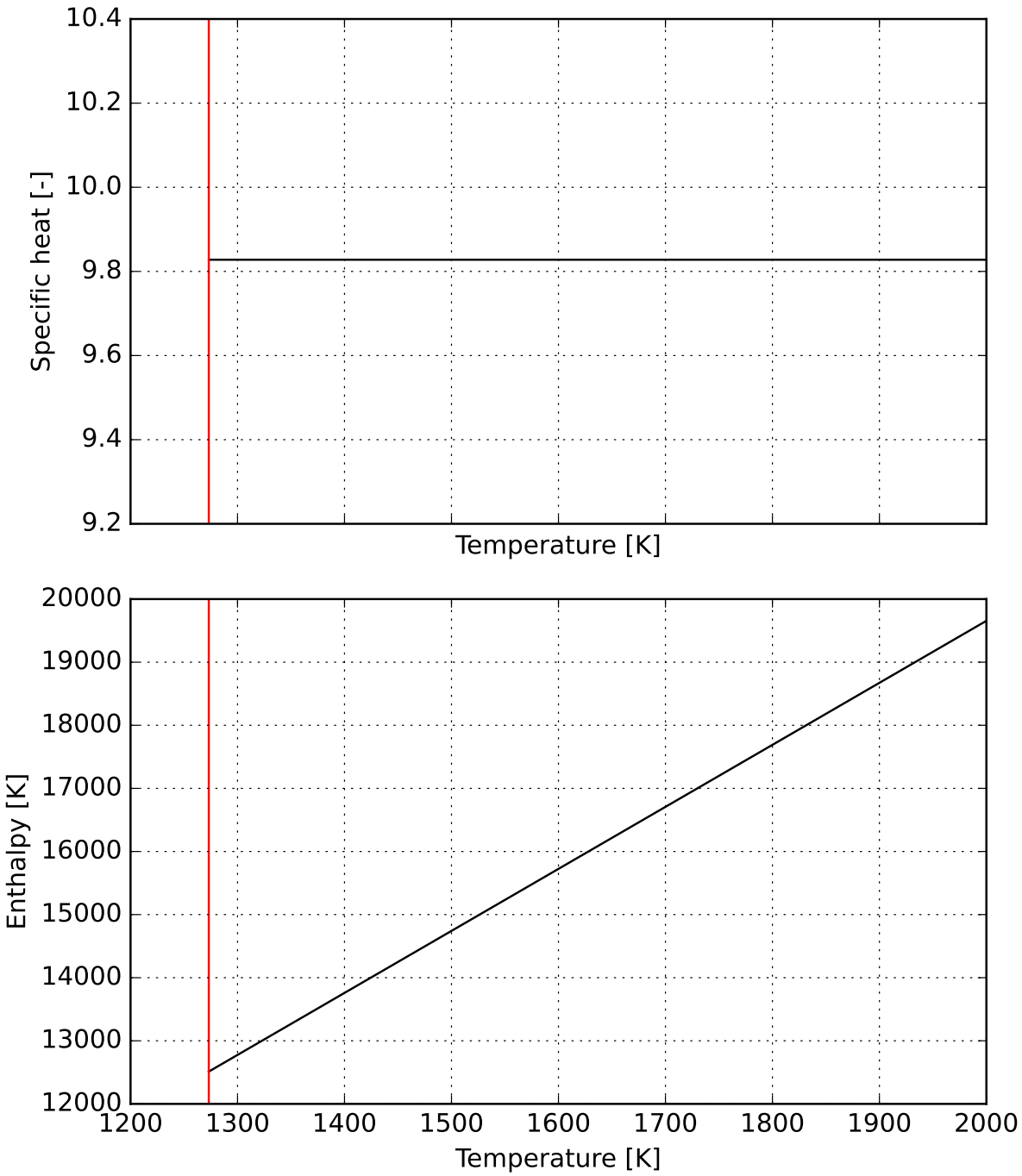


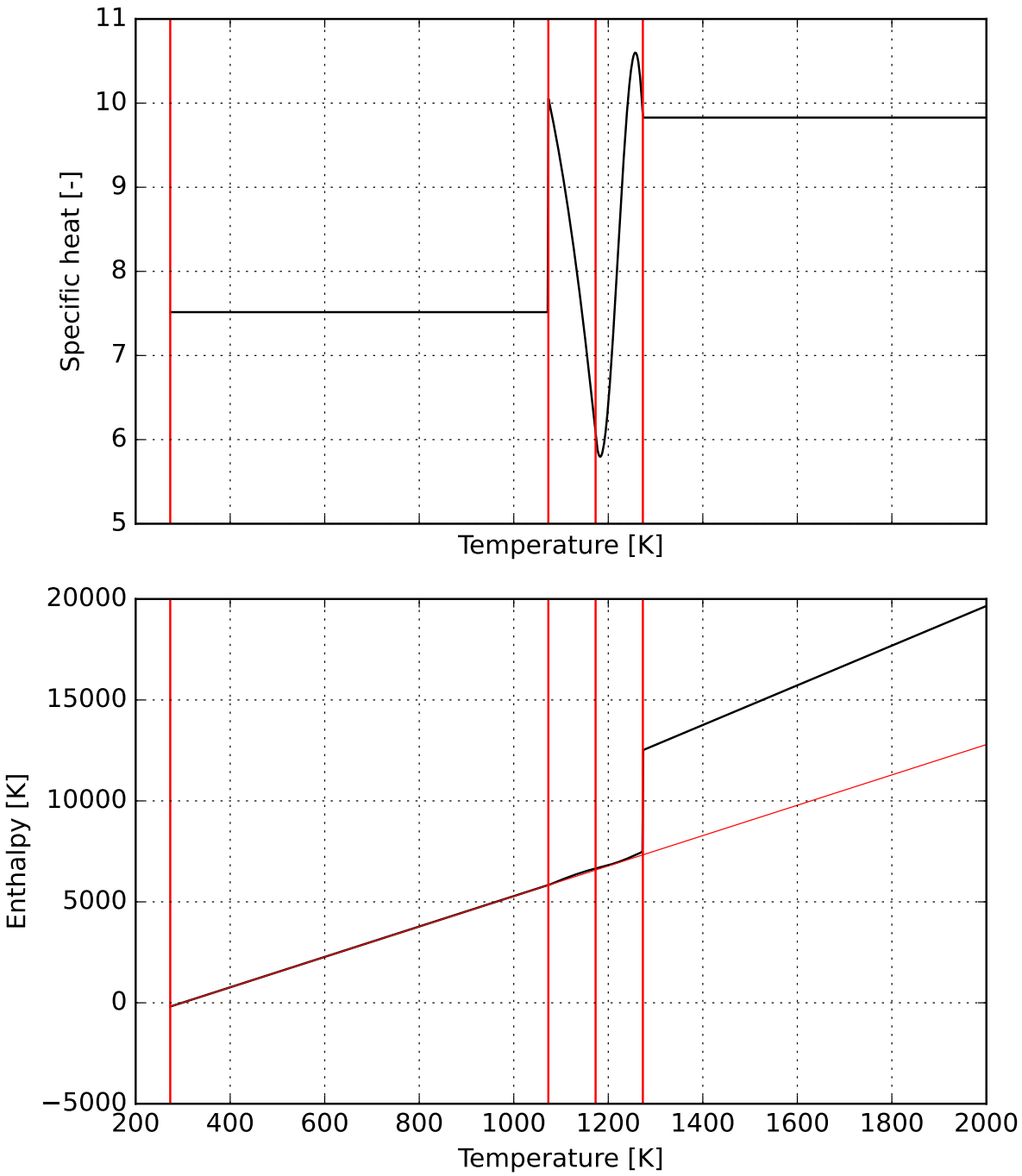
For the second part of the reaction we ensure continuity from the previous one

```
DM([-4.46699e-11, -1.13545e-11, 2.24354e-12, -1.10049e-09, -3.17539e-09])
```


11
10
9
8
7
6
5
Specific heat [-]
Temperature [K]


7500
7400
7300
7200
7100
7000
6900
6800
6700
6600
Enthalpy [K]
1160
1180
1200
1220
1240
1260
1280
Temperature [K]


10.4
10.2
10.0
9.8
9.6
9.4
9.2
Specific heat [-]
Temperature [K]
20000
19000
18000
17000
16000
15000
14000
13000
12000
Enthalpy [K]
1200
1300
1400
1500
1600
1700
1800
1900
2000
Temperature [K]


11
10
9
8
7
6
5
Specific heat [-]
Temperature [K]
20000
15000
10000
5000
0
−5000
Enthalpy [K]
200
400
600
800
1000
1200
1400
1600
1800
2000
Temperature [K]

Part V

Finite Volume Method

22 Finite Volume Method

In these notes we provide a Finite Volume Method relecture to the famous The 12 steps to Navier-Stokes equations, by Lorena A. Barba and Gilbert F. Forsyth. It is not a bare translation, but a different approach to the problems with the addition of new elements, some sort of *Twelve-steps to Navier Stokes rewind*.

Also the final goal has been changed and now it is not only as reaching the implementation of Navier-Stokes equations, but learning the key ideas of finite volumes applied to other fields of interest in Materials Science. Students who are undergoing Calculus training should already meet the minimum requirements to follow this course.

Focus will be given not only on the numerical methods, but also how a computer implementation should look like. This last point is the most distinctive point from the work upon which it is based. A vast amount of learning resources is available for scientific computing, but rare sources really provide guidance for conceiving quality and maintainable code. Here we try to contribute for better coding practices through a *think-before-coding* approach.

22.1 Diffusion equation

This part describes the implementation of diffusion using finite volume method. It is intended mainly to treat topics of interest in Materials Science, thus composition and spatial dependencies of diffusivity are a pre-requisite for any practical use case. A progressive level of complexity in the described physics will be considered. It assumes the reader is already familiar with basic concepts of diffusion in solids, much of which will be exposed in the text without further explanations.

The following lists the goals and status of current text:

- ☒ Simple Fick's Law with possible nonlinear behavior in one-dimension on chapter 23.
- ☐ Generalization of Fickean diffusion to higher dimensions and other coordinate systems.
- ☐ Irreversible Thermodynamics approach for chemical potentials.
- ☐ Anisotropy of diffusivities in three-dimensional space.

The interested reader may wish to consult the work by Mehrer (2007) for more details; any Materials Scientist is also recommended at some point to check the classic works of (Onsager 1931a, 1931b).

22.2 Convection equation

Upcoming in Chapter 24!

23 Diffusion in 1-D

23.1 Import toolbox

```
from abc import ABC
from sympy import symbols, init_printing
from sympy import Eq as SymEq, expand, solve
from pandas import DataFrame
import casadi as cs
import numpy as np

init_printing(use_latex="mathjax")
```

23.2 Introductory Fickean diffusion

Assume a simple concentration-dependent diffusion in the absence of external driving forces; in this introduction we chose to *model* the diffusive flux through Fick's constitutive (first) law stated below, where D is the diffusion coefficient that may depend on position x and/or concentration c, *i.e.* $D \equiv D(x, c)$. It states that diffusive flux of an species has the composition gradient as a driver force; the negative sign indicates that diffusion occurs from high to low concentration regions.

$$J = -D\frac{\partial c}{\partial x}$$

We can anticipate here that this can be the case for simple diluted systems, but interactions between species must be considered in concentrated solutions, what is generalized by the *Irreversible Thermodynamics* by Onsager (1931a) and Onsager (1931b). Here we emphasize again the word *model*: empirical observation has led scientists to *represent* reality through such an approximation; different models can be used for the same physics with their applicability limited to certain scenarios. Properly selecting an applicable model is the role of the researcher/engineer, what is outside our scope here; in what follows we will focus on *how to solve* balance equations for a given model.

Simply put, diffusion equation solves for a local mass balance; and mathematically, a local balance leads to a divergence operation. Following this idea, it can be shown that the time derivative of concentration c at one location is given by the divergence - which collapses to the spatial derivative - of the negative of mass flux given by Fick's (first) law (or in general any other constitutive law/model describing the mass flux); sometimes this form is called Fick's second law in the literature. Given the possible non-linearity introduced by diffusion coefficient D, the derivative in the right-hand side is not expanded. In Cartesian coordinate system, the governing diffusion equation in one dimension writes:

$$\frac{\partial c}{\partial t} = \frac{\partial}{\partial x}\left(D\frac{\partial c}{\partial x}\right)$$

This is the form of the diffusion equation that is discussed in what follows.

23.3 Internal discretization

To solve the diffusion equation numerically, we employ the finite volume method (FVM). The first step to establish a FVM scheme consists of integrating the governing equation over a control volume P from w (west face) to e (east face) and over a time interval from 0 to τ. The left-hand side is first integrated over time as it represents a *storage* term, meaning that it measures how much the average concentration changes in the control volume over the time-step. The right-hand side is a *flux* term, and using similar arguments its *natural* integration is performed over space. Also, the order of integration can be exchanged due to the smoothness assumptions considered here. For more details, please check Patankar (1980).

$$\int_w^e \int_0^\tau \frac{\partial c}{\partial t}\, dt\, dx = \int_0^\tau \int_w^e \frac{\partial}{\partial x}\left(D\frac{\partial c}{\partial x}\right)\, dx\, dt$$

This double integration allows us to convert the partial differential equation into a form suitable for numerical discretization. Evaluating the innermost integrals leads to:

$$\int_w^e (c_P - c_P^0)\, dx = \int_0^\tau \left[\left(D\frac{\partial c}{\partial x}\right)_e - \left(D\frac{\partial c}{\partial x}\right)_w\right] dt$$

Here, c_P^0 and c_P represent the concentration at the center of the control volume P at times 0 and τ, respectively. A convention of not adding a superscript to time τ is adopted here, unless required by the context. The right-hand side represents the net flux into the control volume through its east and west faces. Assuming the concentration is uniform within each control volume and the fluxes are constant over the time interval, we obtain:

$$(c_P - c_P^0)\,\frac{\delta}{\tau} = \left(D\frac{\partial c}{\partial x}\right)_e - \left(D\frac{\partial c}{\partial x}\right)_w$$

where δ is the width of the control volume. This equation represents a discrete time-step of the diffusion equation for a single control volume, where one still needs to specify a discretization of the gradients, *i.e.* how the fluxes are evaluated at a cell face. A simple introductory approach is to use forward differences, where the composition gradients are approximated as first differences between neighbors.

Before starting we declare a few classes grouping the required symbols for what follows:

```
class FvmVars:
    """ General FVM parameters and variables. """
    delta = symbols("delta")
    tau = symbols("tau")

class BcVars:
    """ Boundary condition parameters and variables. """
    h = symbols("h")
    c_inf = symbols("c_{\\infty}")

class CellVars:
    """ Cell-centered coefficients and variables. """
    c_0 = symbols("c_0")
    c_E = symbols("c_E")
    c_P = symbols("c_P")
```

```
    c_W = symbols("c_W")

    D_E = symbols("D_E")
    D_P = symbols("D_P")
    D_W = symbols("D_W")

    c_G = symbols("c_G")
    c_B = symbols("c_B")  # sort of cell (0-D cell)...

    D_G = symbols("D_G")
    D_B = symbols("D_B")

    @staticmethod
    def composition():
        """Return all cell compositions as a list."""
        return [
            CellVars.c_0,
            CellVars.c_E,
            CellVars.c_P,
            CellVars.c_W,
            CellVars.c_G,
            CellVars.c_B,
        ]

class FaceVars:
    """ Face-centered coefficients and variables. """
    D_e = symbols("D_e")
    D_w = symbols("D_w")
    D_g = symbols("D_g")
    D_b = symbols("D_b")

    Fo_e = symbols("alpha_e")
    Fo_w = symbols("alpha_w")
    Fo_g = symbols("alpha_g")
    Fo_b = symbols("alpha_b")
```

To approximate the aforementioned gradients, the next cell introduces the expressions for the fluxes at east and west faces. These helper functions ensure the order of arguments is consistent with the finite difference definition given above.

```
def diffusion_flux(D, c_left, c_right, delta=FvmVars.delta):
    """ Compute the (forward difference) diffusion flux between two points. """
    return -D * (c_right - c_left) / delta


def diffusion_flux_east(D=FaceVars.D_e, c_right=CellVars.c_E,
                        delta=FvmVars.delta):
    """ Compute the diffusion flux to the east cell. """
    return diffusion_flux(D, CellVars.c_P, c_right, delta=delta)


def diffusion_flux_west(D=FaceVars.D_w, c_left=CellVars.c_W,
```

```
                        delta=FvmVars.delta):
    """ Compute the diffusion flux to the west cell. """
    return diffusion_flux(D, c_left, CellVars.c_P, delta=delta)
```

For standardizing equations as organized as $F(x) = 0$ we override `sympy.Eq` with `StandardEquation` class:

```
class StandardEquation(SymEq):
    """ Standard LHS format of finite volume equation. """
    def __new__(cls, lhs, rhs=0, **kwargs):
        return SymEq.__new__(cls, expand(lhs), 0, **kwargs)

    def __truediv__(self, other):
        return StandardEquation(self.lhs / other)

    def __rtruediv__(self, other):
        return StandardEquation(other / self.lhs)

    def __mul__(self, other):
        return StandardEquation(self.lhs * other)

    def __rmul__(self, other):
        return StandardEquation(other * self.lhs)

    def subs(self, *args, **kwargs):
        """ Substitute into the equation LHS. """
        return StandardEquation(self.lhs.subs(*args, **kwargs))

    def solve_for(self, var):
        """ Solve the equation for the specified variable. """
        return solve(self, var)

    def coefficients(self, variables=None, op=lambda x: x):
        """ Extract coefficients for the specified variables. """
        variables = CellVars.composition() if variables is None else variables
        variables = [v for v in variables if v in self.lhs.free_symbols]
        return {var: op(self.lhs.coeff(var)) for var in variables}

    def tabulate(self, **kwargs):
        """ Helper to display a table of coefficients for the problem. """
        coefs = self.coefficients(**kwargs).items()
        return DataFrame(coefs, columns=["Variable", "Coefficient"])


# Alias:
Eq = StandardEquation
```

Below we compose the diffusion equation with help of SymPy and the above extensions. Notice that when compositing the right-hand side `rhs` we multiply by -1 because we are integrating the negative of the fluxes.

```
J_w = diffusion_flux_west()
J_e = diffusion_flux_east()
```

```
lhs = (CellVars.c_P - CellVars.c_0) * FvmVars.delta / FvmVars.tau
rhs = -1 * (J_e - J_w)

eq = Eq(lhs - rhs)
eq
```

$$-\frac{D_e c_E}{\delta} + \frac{D_e c_P}{\delta} + \frac{D_w c_P}{\delta} - \frac{D_w c_W}{\delta} - \frac{c_0 \delta}{\tau} + \frac{c_P \delta}{\tau} = 0$$

Here, c_E and c_W are the concentrations at the eastern and western neighboring control volumes, respectively, and D_e and D_w are the effective diffusion coefficients at the east and west faces of the control volume. Patankar (1980) shows that the evaluation of these diffusion coefficients is not arbitrary and for conservation (here the continuity of flux across the interface) to be respected one must take the harmonic mean of neighboring cells:

$$D_k = 2\frac{D_i D_j}{D_i + D_j}$$

Or as code:

```
def harmonic_mean(a, b):
    """ Compute the harmonic mean of two values. """
    return 2 * a * b / (a + b)
```

A final step to get a linear form of the problem is multiplying both sides by τ/δ and rearranging terms. This form clearly shows the contribution of each neighboring concentration to the rate of change at point P. The coefficient of c_P is negative, representing the outflow from the central node, while the coefficients of c_E and c_W are positive, representing inflow from the neighboring nodes.

$$-\frac{D_e c_E \tau}{\delta^2} + \frac{D_e c_P \tau}{\delta^2} + \frac{D_w c_P \tau}{\delta^2} - \frac{D_w c_W \tau}{\delta^2} - c_0 + c_P = 0$$

Introducing dimensionless coefficients α_k, the Fourier number for mass transfer (more generally referred to as $\mathcal{F}o_m$), we obtain the final discrete form. It represents the dimensionless diffusion number, which determines the stability and accuracy of the numerical scheme. The subscript k refers to either the east (e) or west (w) face.

$$\alpha_k = \frac{\tau D_k}{\delta^2}$$

This definition is provided below for helping with the replacements that follow.

```
def fourier_number(D, tau=FvmVars.tau, delta=FvmVars.delta):
    """ Compute the Fourier number for the given parameters. """
    return D * tau / delta**2


SUBS_FOURIER = [
    (fourier_number(FaceVars.D_e), FaceVars.Fo_e),
    (fourier_number(FaceVars.D_w), FaceVars.Fo_w),
    (fourier_number(FaceVars.D_g), FaceVars.Fo_g),
    (fourier_number(FaceVars.D_b), FaceVars.Fo_b),
]
```

Performing the replacements with help of SymPy:

```
eq = eq.subs(SUBS_FOURIER)
eq
```

$$-\alpha_e c_E + \alpha_e c_P + \alpha_w c_P - \alpha_w c_W - c_0 + c_P = 0$$

Notice that so far nothing has been said with respect to the instant at which the right-hand side is evaluated. The obvious solution would be to take $c_0 \equiv c_P^0$ as it is currently known, but for numerical stability reasons we will continue here with an implicit scheme so that all values other than c_0 are defined to be evaluated at time τ, *i.e.* $c_K \equiv c_K^\tau$, in phase with the notation convention introduced above.

Because it is more convenient for symbolic manipulations with SymPy, the problem is currently written as $F(c) = 0$, *i.e.* all elements are on the left-hand side. Care must be taken to properly interpret the signs of the coefficients below.

```
coefs = eq.coefficients()
coefs
```

$$\{c_0 : -1,\ c_E : -\alpha_e,\ c_P : \alpha_e + \alpha_w + 1,\ c_W : -\alpha_w\}$$

Because it is more readable, a helper utility is provided to tabulate them:

```
eq.tabulate()
```

	Variable	Coefficient
0	c_0	-1
1	c_E	-alpha_e
2	c_P	alpha_e + alpha_w + 1
3	c_W	-alpha_w

23.4 Boundary conditions

So far all we have derived are the internal balances of the system, we still lack proper boundary conditions. It should be evident by now that we have formulated a tridiagonal matrix for representing the 1-D system: there are 3 unknowns at step τ (c_W, c_P, and c_E) and a currently known *field* c_0.

In what follows we will workout the three families of boundary conditions: Dirichlet (type 1), Neumann (type 2), and the more general Robin (type 3). Following this sequence will create the required intuition for deriving the progressively more complex conditions. In practice, for diffusion in solids often only the implementation of Robin conditions is required, as it will be discussed later. Our goal at presenting types 1 and 2 is simply to introduce the tooling required for working out type 3 boundary condition; they will not be treated in full detail, leaving the full discussion for later.

The implementation of boundary conditions requires special treatment of the control volumes at the domain boundaries. The general form for a boundary control volume is:

$$(c_P - c_P^0)\,\frac{\delta}{\tau} = \text{flux}_{\text{interior}} - \text{flux}_{\text{boundary}}$$

It is worth here repeating the partial development derived above for an analogy:

$$\left(c_P^\tau - c_P^0\right)\frac{\delta}{\tau} = D_e\frac{c_E - c_P}{\delta} - D_w\frac{c_P - c_W}{\delta}$$

Please, notice that from now on we make use of class `StandardProblem` for the end of representing a canonical discrete equation. It encodes the results obtained in the previous section and is also repeated below:

```
class StandardProblem:
    """ Standard format of finite volume diffusion problem in 1-D. """
    @staticmethod
    def get_lhs(**kw):
        """ Standard format of inner LHS of problem. """
        return (CellVars.c_P - CellVars.c_0) * FvmVars.delta / FvmVars.tau

    @staticmethod
    def get_rhs(**kw):
        """ Standard format of inner RHS of problem."""
        J_e = kw.get("J_e", diffusion_flux_east())
        J_w = kw.get("J_w", diffusion_flux_west())
        return -1 * (J_e - J_w)

    @staticmethod
    def full_expression(subs_lhs=[], subs_rhs=[], subs_fourier=False, **kw):
        """ Canonical form of problem set as `f(c)=0`. """
        lhs = StandardProblem.get_lhs(**kw).subs(subs_lhs)
        rhs = StandardProblem.get_rhs(**kw).subs(subs_rhs)

        eq = StandardEquation(lhs - rhs)
        eq = eq * FvmVars.tau / FvmVars.delta

        if subs_fourier:
            eq = eq.subs(SUBS_FOURIER)

        return eq
```

```
StandardProblem.full_expression()
```

$$-\frac{D_e c_E \tau}{\delta^2} + \frac{D_e c_P \tau}{\delta^2} + \frac{D_w c_P \tau}{\delta^2} - \frac{D_w c_W \tau}{\delta^2} - c_0 + c_P = 0$$

23.5 Type 1 BC - Dirichlet

Let's start by the Dirichlet, or fixed concentration, boundary condition (type 1), for which the concentration at the boundary face is prescribed, *i.e.* $c_\mathrm{B} = c_\mathrm{prescribed}$. Notice that before using this we need to make a choice regarding the computational grid (mesh); you could have half-volumes on boundaries, fictitious volumes (sometimes called ghost cells), or immersed nodes over the boundary.

Partial control volumes on boundaries are not desirable as they introduce implementation complexities in code and lack conservation (especially in higher dimensions), see a discussion by Maliska (2004) (section 3.7) for details. In what follows we discuss immersed nodes and extended domains with *ghost* cells. This last solution implies considerably more unknowns in 3-D problems, for which different variants of FVM will be discussed at a more advanced point.

Immersed boundary: Assume the last cell on *west* boundary is indexed as P; over its *west* boundary we find a node where prescribed composition c_B is enforced - and the corresponding diffusion coefficient D_b. Notice that this node does not belong to the cell, nor represents a half-cell, but lies over its boundary. To keep it simple, let's assume c_B is constant (time-dependency would slightly complicate the development and it is worth postponing it to the algorithm development discussion), so keeping our fully implicit scheme leads to We start by computing the mass flux from the boundary node to the first cell center; this flux is computed using half-cell length $\delta/2$ for the derivative approximation.

```
J_w = diffusion_flux_west(FaceVars.D_b, CellVars.c_B, FvmVars.delta/2)
J_w
```

$$-\frac{2D_b\left(-c_B + c_P\right)}{\delta}$$

This expression of flux is used as replacement for the *west* boundary in the full equation:

```
eq = StandardProblem.full_expression(J_w=J_w, subs_fourier=True)
eq
```

$$-2\alpha_b c_B + 2\alpha_b c_P - \alpha_e c_E + \alpha_e c_P - c_0 + c_P = 0$$

The coefficients for the first row of the matrix problem are summarized below:

```
eq.tabulate()
```

	Variable	Coefficient
0	c_0	-1
1	c_E	-alpha_e
2	c_P	2*alpha_b + alpha_e + 1
3	c_B	-2*alpha_b

There are two main features in this equation: the coefficient of cell P now depends on the prescribed composition through α_b and a modification is applied to the right-hand side at boundaries where solution is prescribed. That's a good reminder that the array of coefficients must correspond to the number of interfaces, not cells - more on this later.

The immersed boundary approach presented above is simple. Nonetheless, it requires a different calculation of boundary nodes with respect to the volume, but that comes with the advantage that no additional equation has been added to the system. Let's now explore the extended domain alternative.

Ghost cell: As above, assume the last cell on *west* boundary is indexed as P. Consider another *ghost* cell to its left being denoted G (fictitious cell in place of W). The node c_B where composition is enforced and the same details presented above still apply here. Here we repeat the immersed node flux computation and do the same with respect to *ghost* cell G.

Our goal is to identify an equation for c_G so that it is solved along with the interior points and ensures the same flux as the immersed node. To identify the compatible *ghost* composition we equate the fluxes by both definitions and solve for c_G. The following steps repeat what we have seem so far and should be self-explanatory.

```
J_w1 = diffusion_flux_west(FaceVars.D_b, CellVars.c_B, FvmVars.delta/2)
J_w2 = diffusion_flux_west(FaceVars.D_g, CellVars.c_G, FvmVars.delta)

Eq(J_w1 - J_w2).tabulate()
```

	Variable	Coefficient
0	c_P	-2*D_b/delta + D_g/delta
1	c_G	-D_g/delta
2	c_B	2*D_b/delta

Care must be taken for interpreting the above result; here c_G takes the diagonal position and c_P is the first *east* cell in the first row of the problem's matrix. There is no previous state (no coefficient on c_0), but a source term based on c_B that must be placed on the right-hand side for solution. This is the equation we are looking for to extend the system.

Since this equation has D_g as a coefficient, an initial guess for c_G is required. The boundary B being the same interface for evaluation of D_g, for both fluxes to equate one needs to make sure the same diffusivity is evaluated there. Thus we make $D_b \equiv D_g$, what seems physically reasonable but so far not supported by any mathematical argument:

```
c_G_sol = Eq(J_w1 - J_w2).solve_for(CellVars.c_G)[0]
c_G_sol = c_G_sol.subs(FaceVars.D_b, FaceVars.D_g)
c_G_sol
```

$$\frac{2D_g c_B - D_g c_P}{D_g}$$

This simply says that the composition is linearly extrapolated, with the distances from cell centers to the boundary being used as weighting factors. It ensures the problem is initialized the same as the immersed boundary as we made $D_g = D_b$.

Common mistake

One could think about solving for c_G and using the result in the standard balance equation, but doing so falls back to the immersed boundary approach. Replacing the solution in J_w2 simply gets back to the initial point and does not produce one extra equation!

```
J_w = J_w2.subs(CellVars.c_G, c_G_sol)

eq = StandardProblem.full_expression(J_w=J_w, subs_fourier=True)
eq.tabulate()
```

	Variable	Coefficient
0	c_0	-1
1	c_E	-alpha_e
2	c_P	alpha_e + 2*alpha_g + 1
3	c_B	-2*alpha_g

23.6 Type 2 BC - Neumann

Next, let's handle Neumann boundary condition (type 2). For a fixed flux boundary condition, the flux at the boundary is prescribed at q_B. A special case arises when one imposes zero flux (impermeable boundary for mass transfer, adiabatic boundary for heat transfer) where $q_B = 0$. It is worthless

formulating the problem for this case as the equation is trivial, so we will assume a general q_B in what follows.

$$-D\frac{\partial c}{\partial x}\bigg|_B = q_B$$

When dealing with Neumann boundary conditions one must take care of the sign of fluxes, especially in 1-D. In higher dimensions, when applying Gauss theorem to convert the volume integral into a surface integral one normally works with vector calculus formulations; here we have dropped the vector notation when integrating over dx, but the concept of surface orientation must be recalled at this point. At the *west* boundary the flux into domain is positive and conversely, at the *east* boundary the flux out of domain is positive.

$$\text{(west)} \qquad \left(c_P^\tau - c_P^0\right)\frac{\delta}{\tau} = D_e\frac{c_E - c_P}{\delta} + q_B$$

$$\text{(east)} \qquad \left(c_P^\tau - c_P^0\right)\frac{\delta}{\tau} = -q_B - D_w\frac{c_P - c_W}{\delta}$$

We will not elaborate more as the most relevant condition represented by this class, the impermeable condition, can be represented with a type 3 boundary condition by setting the mass transfer coefficient h to zero.

23.7 Type 3 BC - Robin

Finally, we generalize the above discussion for a Robin boundary condition (type 3). Also known as a convective or mixed boundary condition. In this case the flux is proportional to the concentration difference between the boundary and a reference external composition c_∞. This often models mass transfer between a reacting fluid and a solid, but the discussion of boundary layer formation is out of the present scope and we summarize it simply by the mass transfer coefficient h.

$$-D\frac{\partial c}{\partial x}\bigg|_B = h(c_B - c_\infty)$$

Fallback to other kinds

In fact, this is not the canonical form of such a boundary condition, but the reasonable engineering statement. Below we transform the equation to a format more commonly found the in mathematical literature, where all elements should be interpreted as being evaluated at boundary B.

$$\alpha c + \beta\frac{\partial c}{\partial x} = g$$

where

$$\alpha = 1 \quad \text{and} \quad \beta = \frac{D}{h} \quad \text{and} \quad g = c_\infty$$

Under this form we can easily jump to conclusions with respect to its relationship to the other types of boundary conditions. First, simply by setting $\beta = 0$ we find ourselves with a type 1

condition where $c = \alpha^{-1} g$ - or in engineering notation $c_B = c_\infty$; this happens as mass transfer efficiency is ideal, as $\beta \to 0$ when $h \to \infty$. In computational practice it is enough to enforce that $h \gg D$ to approach a Dirichlet boundary condition.
A type 2 condition would be established with $\alpha = 0$, but that is not possible straight from the way we have posed the equation. By multiplying everything by h and setting its value to zero we get to an impermeable Neumann condition, the only one we consider here for the mass transfer context (because in general it is not simple to enforce a mass uptake by a material).
That said, type 3 condition is *all we need* for a first solid state diffusion solver. As for the type 2 condition, we need to take care of signs here.

The (scalar) convention we used so far implies a convective flux that is positive *outwards* the domain, $J = h(c_b - c_\infty) > 0$ if $c_b > c_\infty$, *i.e.*, the chemical potential in the solid is higher than the environment. The *west* boundary being oriented towards the $-y$ axis, we need to account for its sign; in later chapters we will see that the boundary condition reasoning is actually simpler to follow in higher dimensions as we implicitly treat the face normal vectors through the inner product.

```
def convection_flux(cb, h=BcVars.h, c_inf=BcVars.c_inf):
    """ Compute the convection flux between environment and surface. """
    return h * (c_inf - cb)

q_bw = -convection_flux(CellVars.c_B)
q_be = +convection_flux(CellVars.c_B)
```

In what follows we derive the immersed boundary equation for type 3 condition. The extension to *ghost* cells is trivial and can be worked out with the ideas presented in section Section 23.5. We solve for c_B at both sides of the domain by equating the convective flux to the one established internally with the immersed boundary node:

```
J_w = diffusion_flux_west(FaceVars.D_b, CellVars.c_B, FvmVars.delta/2)
J_e = diffusion_flux_east(FaceVars.D_b, CellVars.c_B, FvmVars.delta/2)

c_Bw = Eq(J_w - q_bw).solve_for(CellVars.c_B)[0]
c_Be = Eq(J_e - q_be).solve_for(CellVars.c_B)[0]
```

This leads to

c_Bw

$$\frac{2D_b c_P - c_\infty \delta h}{2D_b - \delta h}$$

and

c_Be

$$\frac{2D_b c_P - c_\infty \delta h}{2D_b - \delta h}$$

If we take, *e.g.*, the *west* boundary equation with the previous definition of flux evaluated from the immersed boundary node, we retrieve the corresponding equation for the system. Notice that c_B is a known value at the current iteration, so in solution it can be moved to the right-hand side and act as a flux term. This gives a grasp of the solution algorithm that makes the subject for the next section.

```
eq = StandardProblem.full_expression(J_w=J_w, subs_fourier=True)
eq.tabulate()
```

	Variable	Coefficient
0	c_0	-1
1	c_E	-alpha_e
2	c_P	2*alpha_b + alpha_e + 1
3	c_B	-2*alpha_b

23.8 Arbitrary grid spacing

For establishing the equations for an arbitrary grid spacing in 1-D, we move forward to a more general derivation. Instead of expressing the problem over a single coordinate, we introduce the control volume V that will be revisited again when solving for 3-D geometries. For convenience the minus sign is moved outside the integral in what follows.

$$\int_V \int_0^\tau \frac{\partial c}{\partial t}\, dt\, dV = -\int_0^\tau \int_V \nabla \cdot \vec{J}\, dV\, dt$$

The inner integration on the right-hand side is treated first. Using the divergence theorem, we convert the volume integral of the divergence of flux into a surface integral over the control volume's faces. Since in FVM we are dealing with a finite number of faces, we can express the surface integral as a sum over all faces f of the control volume, where J_f is the flux through face f and A_f is the area of that face.

$$\int_V \nabla \cdot \vec{J}\, dV = \int_S \vec{J} \cdot \vec{n}\, dS = \sum_f J_f A_f$$

Substituting this result back into the original equation and integrating the left-hand side over time gives:

$$\int_V \left(c_P - c_P^0\right)\, dV = -\int_0^\tau \sum_f J_f A_f\, dt$$

And finally integrating the last level:

$$\left(c_P - c_P^0\right) \frac{V}{\tau} = -\vec{J}_f \cdot \vec{A}_f$$

In 1-D, there are only two faces: the west face (w) and the east face (e). Thus, the summation over faces simplifies to just these two contributions:

$$\left(c_P - c_P^0\right) \frac{V}{\tau} = -J_w(-\hat{i})A_w(-\hat{i}) - J_e(+\hat{i})A_e(+\hat{i})$$

Notice that as we are dealing with the actual surfaces, the signs of the terms will be naturally handled by the flux definitions. The final step consists of substituting the expressions for the diffusive fluxes

at each face, which can be approximated using finite differences. Below we account already for the face orientation (sign). For the west face:

$$-J_w(-\hat{i})A_w(-\hat{i}) = D_w \frac{c_W - c_P}{\delta_w} A_w$$

And for the east face:

$$-J_e(+\hat{i})A_e(+\hat{i}) = D_e \frac{c_E - c_P}{\delta_e} A_e$$

Substituting these flux expressions back into the integrated equation gives:

$$(c_P - c_P^0)\,\frac{V}{\tau} = D_w \frac{c_W - c_P}{\delta_w} A_w D_e \frac{c_E - c_P}{\delta_e} A_e$$

Rearranging terms leads to the final discrete equation for an arbitrary grid spacing in 1-D implicit formulation:

$$-\frac{D_w A_w}{\delta_w} c_W + \left(\frac{V}{\tau} + \frac{D_w A_w}{\delta_w} + \frac{D_e A_e}{\delta_e}\right) c_P - \frac{D_e A_e}{\delta_e} c_E = \frac{V}{\tau} c_P^0$$

or assuming a unit cross-sectional area A so that $V = A\delta_P$, we have:

$$-\frac{D_w \tau}{\delta_w \delta_P} c_W + \left(1 + \frac{D_w \tau}{\delta_w \delta_P} + \frac{D_e \tau}{\delta_e \delta_P}\right) c_P - \frac{D_e \tau}{\delta_e \delta_P} c_E = c_P^0$$

For the boundary conditions we apply the same approach as before:

$$-J_b(-\hat{i})A_b(-\hat{i}) = D_b \frac{c_B - c_P}{\frac{1}{2}\delta_P} A_b = hA_b(c_\infty - c_B)$$

Before simplifying this expression to find out c_B it is worth introducing the Sherwood number $\mathcal{S}h$ defined as follows, where L represents a characteristic length of the problem and D the diffusion coefficient; here $L = \delta_P/2$ as half-cell length was used in flux computations and

$$\mathcal{S}h = \frac{hL}{D} \quad \text{and} \quad \mathcal{S}h_b = \frac{h\delta_P}{2D_b}$$

leading to the final boundary node concentration

$$c_B = \frac{c_P + \mathcal{S}h_b\, c_\infty}{1 + \mathcal{S}h_b}$$

23.9 Solution algorithm

Using a segregated solution of non-linear boundaries or immersed nodes:

$$\begin{bmatrix} A_{(0)} & A_E & 0 & & & ... & 0 & 0 \\ A_W & A_P & A_E & & & ... & 0 & 0 \\ 0 & A_W & A_P & A_E & 0 & ... & 0 & 0 \\ 0 & 0 & \ddots & \ddots & \ddots & \ddots & \vdots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & 0 & 0 \\ 0 & 0 & ... & 0 & A_W & A_P & A_E & 0 \\ 0 & 0 & ... & 0 & 0 & A_W & A_P & A_E \\ 0 & 0 & ... & 0 & 0 & 0 & A_W & A_{(n)} \end{bmatrix} \cdot \begin{bmatrix} c^{\tau}_{(0)} \\ c^{\tau}_{(1)} \\ c^{\tau}_{(2)} \\ c^{\tau}_{(3)} \\ \vdots \\ c^{\tau}_{(N-3)} \\ c^{\tau}_{(N-2)} \\ c^{\tau}_{(N-1)} \end{bmatrix} = \begin{bmatrix} c^{0}_{(0)} \\ c^{0}_{(1)} \\ c^{0}_{(2)} \\ c^{0}_{(3)} \\ \vdots \\ c^{0}_{(N-3)} \\ c^{0}_{(N-2)} \\ c^{0}_{(N-1)} \end{bmatrix} + \begin{bmatrix} B_{(0)} \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ B_{(N-1)} \end{bmatrix}$$

23.10 Python prototype

24 Convection in 1-D

Upcoming

Part VI

Montagne

25 Cartographie

This page is intentionally written in French.

25.1 OruxMaps

Pour ajouter des cartes IGN sur OruxMaps il faut créer des entrées au fichier `onlinemapsources.xml` qui se trouve sous `Android/data/com.orux.oruxmapsDonate/files/oruxmaps/mapfiles` dans les configurations du logiciel. Ces cartes doivent avoir un identifiant (uid) différent chacune. La numérotation des `uid` est arbitraire (bonne pratique à suivre: utiliser une séquence de valeurs). Si vous le fusionnez avec un autre, faire attention à adapter les `uid` au besoin.

Références:

- Forum OruxMaps
- Tyber Randos

```
<onlinemapsource uid="1"><!-- Modifier `uid` selon besoin! -->
    <name>Geoplateforme IGN SCAN25</name>
    <url><![CDATA[https://data.geopf.fr/private/wmts?Service=WMTS&apikey=ign_scan_ws&LAYER=C
    <website><![CDATA[<a href="https://www.ign.fr/geoplateforme" target="_blank">Géoplatefor
    <minzoom>2</minzoom>
    <maxzoom>16</maxzoom>
    <projection>MERCATORESFERICA</projection>
    <servers></servers>
    <httpparam name="User-Agent">{om}</httpparam>
    <cacheable>1</cacheable>
    <downloadable>1</downloadable>
    <maxtilesday>0</maxtilesday>
    <maxthreads>0</maxthreads>
    <xop></xop>
    <yop></yop>
    <zop></zop>
    <qop></qop>
    <sop></sop>
</onlinemapsource>
```

25.2 Liens utiles

- GPX Studio
- GPS Visualizer

25.3 Visualisation des traces GPX

```python
from majordome_cartography import GpxManager
from majordome_cartography import display_track

def process_dump(fname: str) -> None:
    gpx_mgnr = GpxManager.from_file(f"{fname}.gpx")
    gpx_mngr = gpx_mgnr.sanitize(dump=f"{fname}-sanitized.gpx")

    track_map = display_track(gpx_mngr, colored=True, vmin=750, vmax=1000)
    track_map.save(f"{fname}.html")

process_dump("gpx/2026-03-07-aretes-du-gerbier")
process_dump("gpx/2026-03-08-mont-aiguille")
```

26 Liens utiles

This page is intentionally written in French.

26.1 Géneral

- Sports de Nature

26.2 Cartographie

OpenStreetMap:

- OpenStreetMap Wiki
- OpenStreetMap Wiki - FR:Projet Plein Air
- OpenStreetMap live edits
- Forum OSM France
- A summary of things happening in the OpenStreetMap world

Cartes pour Garmin:

- OpenTopopMap
- numeriquement
- mkgmap

Other sites:

- Freizeitkarte
- OpenTopoMap
- UtagawaVTT
- CyclOSM
- OpenWhateverMap
- Waymarked Trails
- Jeux de données - data.gouv.fr
- MyOSMatic
- MapRoulette

26.3 Mobilité douce

- Partir Ici
- SNCF Rhône Alpes - Réservation vélo

26.4 Cyclisme

Vélo de montagne:

- Sites FFC
- Cotations de difficulté
- Autour du Pilat
- Gravel Passion

Vélo de route:

- Cols-Cyclisme
- Inforoutes FR
- Inforoutes 04
- Inforoutes 06

26.5 Trail running

- Finishers
- Premier Marathon

26.6 Neige et glace

- Data Avalanche
- CRISTAL
- Ice Climbing Cogne
- ANENA
- Skitourenguru

26.7 Autour de Chamonix

- Mont-Blanc Natural Resort

References

Maliska, Clovis R. 2004. *Transferência de Calor e Mecânica Dos Fluidos Computacional*. 2nd ed. Rio de Janeiro: LTC.

Mehrer, Helmut. 2007. *Diffusion in Solids : Fundamentals, Methods, Materials, Diffusion-Controlled Processes*. Vol. 155. Springer Series in Solid-State Sciences. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-71488-0.

Onsager, Lars. 1931a. "Reciprocal Relations in Irreversible Processes. i." *Physical Review* 37: 405–26.

———. 1931b. "Reciprocal relations in irreversible processes. II." *Physical Review* 38: 2265–79.

Patankar, Suhas V. 1980. *Numerical Heat Transfer and Fluid Flow*. Taylor & Francis. https://doi.org/10.1201/9781482234213.